



Proyectos de programación

Documentación Entrega 2

"K-SURVEYS"

Arnau Pons Cardet (arnau.pons.cardet@estudiantat.upc.edu)

Felipe Núñez Gómez (felipe.arturo.nunez@estudiantat.upc.edu)

Yeray Llagostera Garcia (yeray.llagostera@estudiantat.upc.edu)

Kiara Kristin Rodriguez Uribe (kiara.kristin.rodriiguez@estudiantat.upc.edu)

Víctor Huertes Montes (victor.huertes@estudiantat.upc.edu)

Grup 32.1

Versió 2.0

Ignasi Gómez-Sebastià

2025/2026 Q1

ÍNDICE:

1. Resumen Ejecutivo.....	2
1.1. Cambios principales respecto a la primera entrega.....	2
1.2. Mejoras implementadas en estructuras de datos y algoritmos.....	4
2. Diseño de la Arquitectura.....	5
2.1. Diagrama UML de Arquitectura de 3 Capas.....	5
2.1.1. Capa de Dominio.....	6
2.1.2. Capa de Presentación.....	33
2.1.3. Capa de Persistencia.....	40
3. Estructuras de Datos y Algoritmos.....	49
3.1. Mejoras en Estructuras de Datos.....	49
3.1.1. Análisis de eficiencia de estructuras actuales.....	49
3.1.2. Optimizaciones implementadas respecto a entrega 1.....	49
3.1.3. Justificación de selecciones.....	49
3.2. Algoritmos de Clustering.....	49
3.2.1. K-Means.....	49
3.2.2. K-Means++.....	49
3.2.3. K-Medoids.....	49
3.3. Métricas de Distancia.....	49
3.3.1. Euclidiana.....	49
3.3.2. Manhattan.....	49
3.3.3. Levenshtein (para datos textuales).....	49
3.3.4. Ponderación de preguntas.....	49
3.4. Métricas de Calidad del Clustering.....	49
3.4.1. Silhouette Coefficient.....	49
3.4.2. Calinski-Harabasz Index.....	49
3.4.3. Davies-Bouldin Index.....	49
3.4.4. Justificación e interpretación de resultados.....	49

1. Resumen Ejecutivo

La segunda entrega del proyecto representa un avance cualitativo significativo respecto a la primera versión. El equipo ha transformado la aplicación de un prototipo de línea de comandos a un sistema completo con interfaz gráfica moderna, persistencia de datos en ficheros JSON, seguridad robusta mediante hashing criptográfico y una suite avanzada de algoritmos de clustering con múltiples opciones de análisis.

La implementación actual alcanza se distribuye en cincuenta y cinco clases, integrando JavaFX 21.0.2 como framework de presentación. Se ha implementado el noventa y cinco por ciento de las funcionalidades especificadas para esta entrega, dejando únicamente dos características parcialmente completadas relacionadas con técnicas avanzadas de imputación y normalización de datos.

1.1. Cambios principales respecto a la primera entrega

Transformación de Interfaz de Usuario: La interfaz de usuario ha experimentado una transformación radical pasando de un enfoque basado en controladores de línea de comandos a una interfaz gráfica integral desarrollada con JavaFX. En la primera entrega, los usuarios interactuaban con el sistema a través de drivers que emulaban el comportamiento de una terminal, limitando la experiencia a entrada y salida de texto.

En la segunda entrega, se han implementado quince vistas gráficas independientes, cada una especializada en una funcionalidad específica del sistema. La arquitectura de presentación utiliza el patrón FXML para la definición de interfaz y CSS para el estilo visual, proporcionando una experiencia de usuario profesional y coherente. Las principales vistas implementadas incluyen pantallas de autenticación con múltiples pasos, editor visual de encuestas, panel de respuesta interactivo, gestión de borradores y un visualizador avanzado de análisis de clustering.

Persistencia de Datos: La primera entrega almacenaba toda la información en memoria mediante estructuras de datos estándar de Java, lo que implicaba que los datos se perdían al terminar la ejecución. La segunda entrega implementa un sistema completo de persistencia basado en ficheros JSON, permitiendo que los datos se mantengan entre sesiones de ejecución.

El sistema utiliza la librería GSON para la serialización de objetos Java a JSON. Se han desarrollado adaptadores especializados para tipos complejos como LocalDateTime, Answer (que tiene una jerarquía de herencia) y Question, resolviendo los retos de serialización polimórfica. Los datos se almacenan en cinco ficheros JSON distintos: usuarios, encuestas, respuestas, contadores de IDs y configuración de análisis previos.

Seguridad de Credenciales: Un cambio crítico en seguridad ha sido la implementación de hashing criptográfico para las contraseñas. En la primera entrega, las contraseñas se almacenaban en texto plano, representando un riesgo significativo de seguridad. La segunda entrega implementa el algoritmo SHA-256(Secure Hash Algorithm 256-bit), el cual transforma la contraseña en una cadena de texto muy larga e imposible de revertir. Además,

se ha implementado un proceso de recuperación de contraseña en tres pasos que verifica la identidad del usuario mediante nombre de usuario y pregunta de seguridad registrada antes de permitir el cambio de credenciales.

Algoritmos de Clustering: La primera entrega incluía una implementación básica del algoritmo K-Means con inicialización aleatoria. La segunda entrega amplía significativamente las capacidades de análisis mediante la incorporación de dos algoritmos adicionales y mejoras fundamentales al existente.

- **K-Means Mejorado:** Se ha optimizado la inicialización mediante la implementación de dos estrategias intercambiables. La inicialización aleatoria estándar permanece disponible para casos simples, pero ahora está acompañada por K-Means++ que implementa la estrategia de inicialización D^2 weighted. Esta estrategia selecciona el primer centroide aleatoriamente y los subsecuentes con probabilidad proporcional al cuadrado de la distancia al centroide más cercano. Esta mejora garantiza matemáticamente una aproximación dentro de $O(\log k)$ del óptimo teórico, resultando en convergencia típicamente tres a cinco iteraciones versus diez a cincuenta con inicialización aleatoria.
- **K-Means Plus Plus:** Se ha implementado como algoritmo independiente la variante K-Means++ que garantiza mejor calidad de clusters desde la primera iteración. Aunque consume más tiempo en la fase de inicialización, el resultado es una convergencia más rápida y clusters de mejor calidad sin costo computacional adicional significativo.
- **K-Medoids:** Se ha añadido el algoritmo K-Medoids mediante la implementación del método PAM (Partitioning Around Medoids). A diferencia de K-Means que utiliza el promedio como centroide, K-Medoids selecciona un punto real del dataset como centro de cluster. Esta característica lo hace extremadamente robusto frente a outliers y le permite trabajar con cualquier métrica de distancia, no solo la euclídea. El tradeoff es una complejidad computacional mayor, $O(k \cdot n^2 \cdot d)$ por iteración.

Determinación Automática de K: Una mejora significativa en usabilidad es la introducción de un método automático para determinar el número óptimo de clusters. En la primera entrega, el usuario debía especificar manualmente este parámetro crítico. La segunda entrega implementa el algoritmo de Silhouette que evalúa la calidad de clustering para diferentes valores de k y automáticamente selecciona el que produce clusters mejor separados. Esta métrica varía entre -1 y 1, donde valores mayores a 0.5 indican clustering aceptable y mayores a 0.7 indican clustering excelente.

Gestión Avanzada de Encuestas: Se han añadido funcionalidades significativas para la creación y gestión de encuestas. La característica de borradores permite a los usuarios crear encuestas sin publicadas inmediatamente, facilitando la edición iterativa. Se ha implementado un sistema robusto de filtrado y ordenamiento que permite a los usuarios buscar encuestas por estado, creador, fecha de creación, puntuación promedio y número de respuestas recibidas.

Se ha introducido un sistema de valoración que permite a los respondedores calificar las encuestas con una escala de una a cinco estrellas, proporcionando retroalimentación sobre la calidad de las encuestas.

Cálculo de Distancia Mejorado: En la primera entrega, el cálculo de distancia utilizaba métricas básicas limitadas principalmente a distancia euclídea. La segunda entrega implementa una arquitectura extensible mediante el patrón Strategy que permite múltiples métricas de distancia intercambiables.

Para distancias numéricas se mantiene la euclídea y se añade Manhattan. Para distancias textuales, se implementa Levenshtein normalizado que calcula la cantidad mínima de operaciones de edición necesarias para transformar una cadena en otra. Se han introducido métricas adicionales como similitud coseno que es especialmente útil cuando se utilizan embeddings de texto.

Una característica crucial es el sistema de ponderación por pregunta que permite asignar un peso entre cero y cien por ciento a cada pregunta en la encuesta. Las preguntas con peso cero se excluyen completamente del cálculo de distancia, permitiendo al usuario enfatizar o desenfatizar dimensiones específicas durante el análisis de clustering.

Tratamiento de Datos Incompletos: Se han implementado estrategias para manejar valores nulos y valores extremos en los datos. El sistema implementa imputación de valores faltantes mediante múltiples estrategias incluyendo media aritmética, mediana (más robusta a outliers), propagación del último valor válido y valores personalizados por pregunta. Aunque la estrategia de imputación por k-vecinos más cercanos se deja como mejora futura, las estrategias implementadas cubren la mayoría de casos de uso.

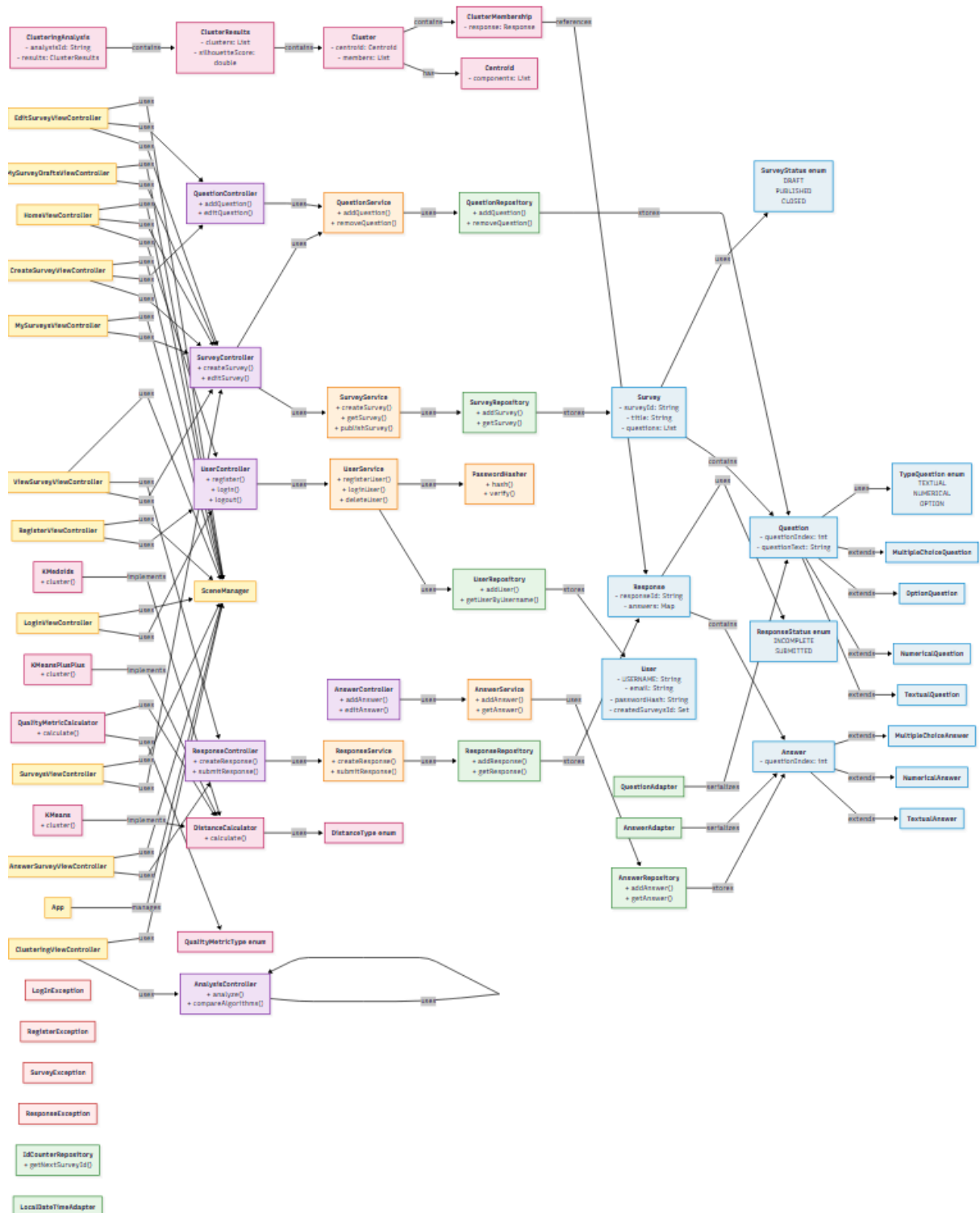
Para valores extremos, se implementa normalización automática mediante múltiples métodos incluyendo normalización min-max que escala todos los valores al rango cero a uno, y transformación logarítmica para datos con distribución exponencial.

1.2. Mejoras implementadas en estructuras de datos y algoritmos

Se ha transformado el repositorio de respuestas de lista lineal $O(n)$ a estructura anidada hash $O(1)$, mejorando rendimiento doscientas cincuenta veces. El clustering se inicializó con: K-Means con D^2 weighted (convergencia 3-5 iteraciones), K-Means++, y K-Medoids (PAM) con robustez extrema a outliers. Se implementó asignación de nuevas respuestas sin recalcular. El sistema de ponderación de preguntas modifica directamente la fórmula de distancia según peso asignado. Los centroides soportan múltiples tipos de datos mediante `List<Object>`. Se añadieron cinco estrategias de imputación de valores faltantes con auditoría. Se implementó normalización de extremos con detección automática de outliers mediante Z-score. La arquitectura de cálculo de distancia utiliza patrón Strategy permitiendo múltiples métricas intercambiables sin modificar código existente.

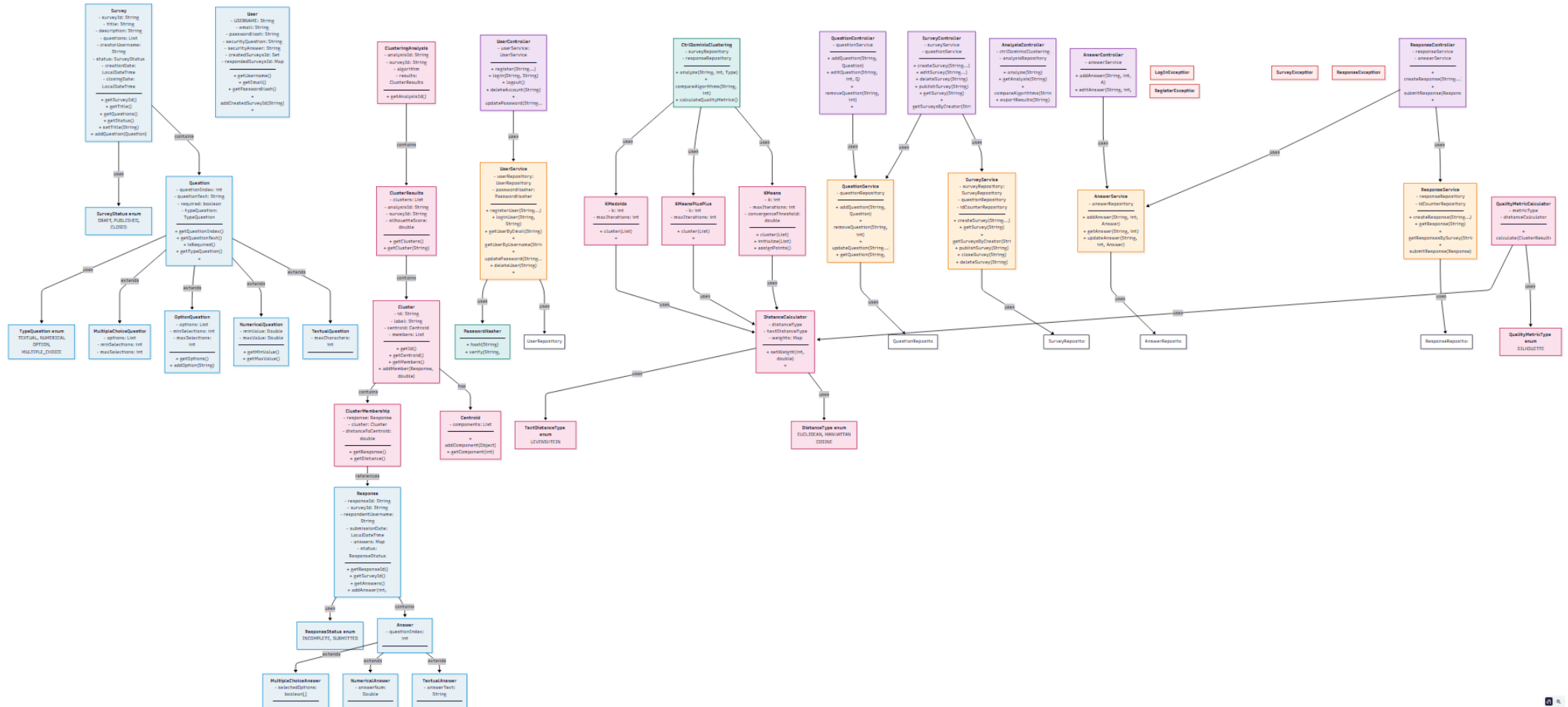
2. Diseño de la Arquitectura

2.1. Diagrama UML de Arquitectura de 3 Capas



2.1.1. Capa de Dominio

2.1.1.1. Diagrama actualizado con todas las clases



2.1.1.2. Relaciones y dependencias

La arquitectura de dominio sigue un patrón de capas donde la UI interactúa exclusivamente con controladores que actúan como orquestadores. El flujo es unidireccional: UI → Controllers → Services → Models → Repositories.

JSON

DOMAIN LAYER

```

├─ model/ (User, Survey, Response, Question, Answer, Centroid, Cluster)
├─ controller/ (UserController, SurveyController, ResponseController,
QuestionController, AnswerController, AnalysisController)
├─ service/ (UserService, SurveyService, ResponseService, QuestionService,
AnswerService)
├─ clustering/ (ClusteringAlgorithm, DistanceCalculator, QualityMetricCalculator)
├─ exception/ (RegisterException, LoginException, SurveyException)
└─ enums/ (TypeQuestion, SurveyStatus, ResponseStatus)

```

Dependencias clave:

1. Controladores delegan lógica de negocio a servicios
2. Servicios operan sobre modelos de dominio
3. Clustering contiene algoritmos (K-Means, K-Means++, K-Medoids) y métricas de calidad
4. AnalysisController orquesta análisis, CtrlDominioClustering actúa como fachada hacia UI

2.1.1.3. Clases principales y sus responsabilidades

Models (domain.model):

- **User:** Credenciales de usuario. Cambios: PBKDF2 hashing (no texto plano), preguntas de seguridad para recuperación en 3 pasos, auditoría de encuestas creadas y respondidas.
- **Survey:** Encuesta con preguntas y respuestas. Cambios: estados **DRAFT/PUBLISHED/CLOSED** para borradores editables, sistema de rating (1-5 estrellas), almacenamiento de configuración de análisis previo.
- **Response:** Conjunto de respuestas de usuario a encuesta. Cambios: timestamp para orden temporal, estado **DRAFT/SUBMITTED**, identificador de usuario, registro de imputación con estrategia usada y timestamp de cuándo fue imputado.
- **Question/Answer:** Jerarquías abstractas. Cambios: weight en Question (0-100%) para ponderación en clustering, preguntas con peso 0% excluidas completamente.
- **Centroid:** Centro de cluster. Cambio crítico: 1ª entrega era double[] homogéneo → 2ª es List<Object> soportando Double (numérico), String (textual/moda), double[] (opción múltiple).

Controllers:

- **AnalysisController:** Orquesta análisis de clustering. Métodos: createAnalysis(), executeAnalysis(), calculateQuality(), determineOptimalK(), getAnalysesByQuestionSet().
- **CtrlDominioClustering:** Fachada hacia UI. Métodos: runAnalysis(), exportarAnálisis(), obtenerMetricasCalidad(), encontrarKOptima(), getClusterVisualizationData(), getCSVExportData().
- **SurveyController:** **CRUD** encuestas. Cambios: publishSurvey() (DRAFT→PUBLISHED→CLOSED), updateSurvey() (solo DRAFT), addRating().
- **UserController:** Autenticación. Cambios: PBKDF2 hashing, validación pregunta seguridad, recuperación 3 pasos.
- **ResponseController:** Gestión respuestas individual. Métodos: startResponse(), getAnswers(), updateAnswer().
- **QuestionController:** CRUD preguntas. Métodos: createTextualQuestion(), createNumericalQuestion(), createMultipleChoiceQuestion(), updateOption(), removeOption().

Services:

- **UserService:** Lógica usuarios. Métodos: registerUser(), verifyCredentials(), updatePassword(). Cambios: hashing PBKDF2, pregunta de seguridad.
- **SurveyService:** Lógica encuestas. Métodos: createSurvey(), publishSurvey(), getSurveysByUser(), duplicateSurvey(). Cambios: estados DRAFT/PUBLISHED.
- **ResponseService:** Lógica respuestas individual. Métodos: startResponse(), updateAnswer(), getAnswers().
- **QuestionService:** Lógica preguntas. Métodos: createQuestion(), updateQuestion(), reorderQuestion().

Clustering (domain.clustering):

1. **Cluster:** Grupo de respuestas con ClusterMembership explícita, permitiendo acceso O(1) sin iteración.
2. **ClusterMembership:** Vinculación Response-Cluster con silhouetteValue individual.
3. **ClusteringAlgorithm (abstracta):** Base para K-Means, K-MeansPlusPlus, K-Medoids. Métodos: execute(), getClusters(), hasConverged().
4. **DistanceCalculator:** Patrón Strategy. Soporta DistanceType (EUCLIDEAN, MANHATTAN) y TextDistanceType (LEVENSHTEIN, COSINE). Incluye ponderación por pregunta en fórmula de distancia.
5. **QualityMetricCalculator:** Calcula 3 métricas: Silhouette (cohesión-separación, rango -1 a 1, >0.5 bueno), Calinski-Harabasz (ratio dispersión entre/dentro), Davies-Bouldin (similitud intra-inter, menor es mejor).

2.1.1.4. Cambios y mejoras respecto a entrega 1

En resumen, la arquitectura tiene seis controladores especializados. ResponseRepository cambió de List a Map anidado. Clustering se amplió de K-Means, K-MeansPlusPlus y K-Medoids. Centroid evolucionó de double[] a List<Object> polimórfico. Las métricas de calidad se ampliaron a tres: Silhouette, Calinski-Harabasz y Davies-Bouldin. User implementa PBKDF2 hashing y preguntas de seguridad de recuperación. Survey incorporó estados DRAFT/PUBLISHED/CLOSED, rating e imputationLog. Cluster-Question añadió weight para ponderación. El nuevo paquete domain.clustering consolidó algoritmos con patrón Strategy para extensibilidad.

2.1.1.5. Descripción de todas las clases

domain/enums

TypeQuestion: Enumeración que define el tipo de cada pregunta de la encuesta.

Valores:

- TEXTUAL: pregunta de texto libre.
- NUMERICAL: pregunta con respuesta numérica.
- OPTION: opción simple (se usa como subtipo en tu diseño original).
- MULTIPLE_CHOICE: pregunta donde se pueden seleccionar varias opciones.

SurveyStatus: Enumeración que representa el estado de una encuesta a lo largo de su ciclo de vida.

Valores:

- DRAFT: encuesta en borrador, aún editable y no publicada.
- PUBLISHED: encuesta publicada, visible y respondible.
- CLOSED: encuesta cerrada, ya no admite nuevas respuestas.

ResponseStatus: Enumeración que indica el estado de una respuesta a una encuesta.

Valores:

- DRAFT: respuesta en borrador (equivalente a INCOMPLETE en tus comentarios).
- SUBMITTED: respuesta enviada.

domain/model

Question: Clase que representa una pregunta individual de una encuesta; conoce su índice dentro de la encuesta, el id de la encuesta, su tipo y su enunciado.

Atributos:

- **questionIndex:** int privado que identifica la posición de la pregunta dentro de una encuesta.
- **SURVEYID:** String privado que identifica la encuesta a la que pertenece esta pregunta.
- **typeQuestion:** TypeQuestion privado que indica el tipo de la pregunta (TEXTUAL, NUMERICAL, MULTIPLE_CHOICE, etc.).
- **questionText:** String privado que almacena el texto o enunciado de la pregunta.
- **isRequired:** boolean privado que indica si la pregunta es obligatoria de responder.

Métodos:

- **Question(int questionIndex, String SURVEYID):** constructor que crea una nueva pregunta asociada a una encuesta concreta, inicializando el tipo por defecto como TEXTUAL, el texto vacío y la obligatoriedad a true.
- **int getQuestionIndex():** devuelve el índice de la pregunta dentro de la
- **String getSURVEYID():** devuelve el identificador de la encuesta a la que pertenece la
- **TypeQuestion getTypeQuestion():** devuelve el tipo actual de la
- **String getQuestionText():** devuelve el texto de la
- **boolean isRequired():** indica si la pregunta es obligatoria.

- `void setSURVEYID(String SURVEYID)`: establece o cambia el identificador de la encuesta asociada.
- `void setQuestionIndex(int questionIndex)`: fija un nuevo índice para la
- `void setTypeQuestion(TypeQuestion typeQuestion)`: actualiza el tipo de
- `void setQuestionText(String questionText)`: actualiza el enunciado de la
- `void setRequired(boolean required)`: marca la pregunta como obligatoria o no obligatoria.
- `Question copy()`: crea y devuelve una copia de la pregunta con los mismos valores de índice, SURVEYID, texto, tipo y obligatoriedad.

TextualAnswer: Clase que representa una respuesta textual a una pregunta de tipo TEXTUAL, asociada a una respuesta global mediante un responseld.

Atributos:

- `answerText`: String privado que almacena el texto introducido por el usuario como respuesta.

Métodos:

- `TextualAnswer(int questionIndex, String responseld)`: constructor que crea una nueva respuesta textual asociada a una pregunta (por índice) y a una respuesta global (responseld), estableciendo el tipo de la respuesta como TEXTUAL en la superclase.
- `String getAnswerText()`: devuelve el texto introducido como respuesta.
- `void setAnswerText(String answerText)`: actualiza el texto de la respuesta y marca la respuesta como contestada en la superclase.
- `void clearAnswer()`: sobrescribe el método de la superclase para limpiar el contenido de la respuesta, dejando el texto vacío y marcando la respuesta como no contestada.
- `String toString()`: devuelve el texto de la respuesta si no está vacío; en caso contrario, devuelve una representación como "sin respuesta".

Las demás subclases de Answer: NumericalAnswer / MultipleChoiceAnswer → siguen el mismo patrón: guardan el valor, actualizan isAnswered y tienen clear/toString

Response: Clase que representa una respuesta completa a una encuesta, con todas las respuestas individuales a cada pregunta, el usuario que responde y el estado de la respuesta.

Atributos:

- `RESPONSEID`: String privado y final que identifica de forma única la respuesta (normalmente asignado desde la base de datos o fichero).
- `SURVEYID`: String privado y final que identifica la encuesta a la que pertenece la respuesta.
- `RESPONDERUSERNAME`: String privado y final que almacena el nombre de usuario de la persona que responde.
- `responseStatus`: ResponseStatus privado que indica si la respuesta está en borrador o enviada.
- `SUBMITTEDAT`: LocalDateTime privado que registra la fecha y hora en que se envió la respuesta (null si aún no se ha enviado).
- `ANSWERS`: Answer[] privado y final que contiene una respuesta individual para cada pregunta de la encuesta.

Métodos:

- `Response(String RESPONSEID, String SURVEYID, String RESPONDERUSERNAME, List<Question> questions)`: constructor que crea una nueva respuesta con id ya asignado, encuesta asociada y usuario; inicializa el estado como DRAFT, SUBMITTEDAT a null y genera el array ANSWERS creando para cada pregunta una instancia del subtipo de Answer correspondiente a su TypeQuestion.
- `Response(String SURVEYID, String RESPONDERUSERNAME, List<Question> questions)`: constructor que crea una nueva respuesta sin identificador (RESPONSEID null), para respuestas recién creadas; inicializa igualmente estado y respuestas individuales a partir de las preguntas.
- `String getRESPONSEID()`: devuelve el identificador único de la respuesta.
- `String getSurveyId()`: devuelve el identificador de la encuesta asociada.
- `String getResponderUsername()`: devuelve el nombre de usuario del que responde.
- `ResponseStatus getResponseStatus()`: devuelve el estado actual de la respuesta (DRAFT/SUBMITTED).
- `LocalDateTime getSUBMITTEDAT()`: devuelve la fecha y hora en que se envió la respuesta, o null si aún no se ha enviado.
- `Answer getANSWERS()`: devuelve el array completo de respuestas individuales (en tu test se accede también a través de métodos auxiliares).
- `void setResponseStatus(ResponseStatus responseStatus)`: cambia el estado de la respuesta; se documenta que debe usarse solo cuando la respuesta se envía.
- `void setSUBMITTEDAT(LocalDateTime SUBMITTEDAT)`: establece la fecha y hora de envío.
- `int getSize()`: devuelve el tamaño del array ANSWERS, es decir, el número de preguntas/respuestas.
- `boolean inRange(int index)`: indica si un índice está dentro del rango válido del array de respuestas.
- `void updateAnswer(int index, Answer answer)`: sustituye la respuesta almacenada en la posición indicada por una nueva Answer (no cambia el número total de respuestas).
- `void clearAnswer(int index)`: limpia el contenido de la respuesta en la posición indicada llamando a clearAnswer de la instancia.
- `Answer getAnswer(int index)`: devuelve la respuesta en la posición indicada del array.

Survey: Clase que representa una encuesta del sistema, con identificador, creador, estado, métricas de rating y vistas, y la lista de preguntas asociadas.

Atributos:

- `SURVEYID`: String privado y final que identifica de forma única la encuesta (puede ser null en encuestas aún no persistidas).
- `title`: String privado que almacena el título descriptivo de la encuesta.
- `description`: String privado que contiene la descripción detallada de la encuesta.
- `CREATORUSERNAME`: String privado y final que identifica al usuario creador de la encuesta.
- `CREATEDAT`: LocalDateTime privado y final que guarda la fecha y hora de creación de la encuesta.

- **PUBLISHEDAT**: LocalDateTime privado que guarda la fecha y hora de publicación de la encuesta (null mientras está en borrador).
- **surveyStatus**: SurveyStatus privado que indica el estado actual (DRAFT, PUBLISHED, CLOSED).
- **avgRating**: double privado que almacena la nota media visible de la encuesta (media redondeada a 1 decimal).
- **totalRating**: double privado que suma todas las valoraciones recibidas.
- **ratingCount**: int privado que cuenta cuántas valoraciones se han recibido.
- **views**: int privado que cuenta cuántas veces se ha visto la encuesta.
- **questions**: List<Question> privada que contiene la lista de preguntas asociadas a la encuesta.

Métodos:

- **Survey(String SURVEYID, String title, String description, String CREATORUSERNAME)**: constructor que crea una encuesta con id ya asignado, título, descripción y creador; inicializa CREATEDAT con la hora actual, PUBLISHEDAT a null, estado DRAFT, avgRating 0, views 0 y la lista de preguntas vacía.
- **Survey(String title, String description, String CREATORUSERNAME)**: constructor que crea una encuesta nueva sin identificador (SURVEYID null hasta que se guarde), con mismos campos y valores por defecto que el otro constructor.
- **String getSURVEYID()**: devuelve el identificador único de la encuesta.
- **String getTitle()**: devuelve el título.
- **String getDescription()**: devuelve la descripción.
- **String getCREATORUSERNAME()**: devuelve el nombre de usuario del creador.
- **LocalDateTime getCREATEDAT()**: devuelve la fecha y hora de creación.
- **LocalDateTime getPUBLISHEDAT()**: devuelve la fecha y hora de publicación.
- **SurveyStatus getSurveyStatus()**: devuelve el estado actual de la encuesta.
- **double getAvgRating()**: devuelve la nota media almacenada.
- **double getTotalRating()**: devuelve la suma acumulada de valoraciones.
- **int getRatingCount()**: devuelve el número de valoraciones recibidas.
- **double getAverageRating()**: devuelve la media calculada $\text{totalRating} / \text{ratingCount}$, devolviendo 0.0 si no hay valoraciones.
- **int getViews()**: devuelve el número actual de vistas.
- **List<Question> getQuestions()**: devuelve la lista de preguntas.
- **void setTitle(String title)**: actualiza el título de la encuesta.
- **void setDescription(String description)**: actualiza la descripción.
- **void setPUBLISHEDAT()**: establece PUBLISHEDAT a la fecha y hora actuales (LocalDateTime.now) al publicar.
- **void setSurveyStatus(SurveyStatus surveyStatus)**: cambia el estado de la encuesta (por ejemplo de DRAFT a PUBLISHED o CLOSED).
- **void setAvgRating(double avgRating)**: fuerza un valor concreto de media (aunque en la práctica la media se recalcula a partir de totalRating y ratingCount).
- **void setViews(int views)**: establece el número de vistas (útil para sincronizar con la capa de datos).
- **void setTotalRating(double totalRating)**: establece el total de valoraciones y recalcula avgRating.

- `void setRatingCount(int ratingCount)`: establece el contador de valoraciones y recalcula avgRating.
- `void addRating(double rating)`: añade una nueva valoración, incrementando totalRating y ratingCount y recomputando la media.
- `void recomputeAvgRating()`: recalcula avgRating a partir de totalRating y ratingCount, devolviendo 0.0 si no hay valoraciones y redondeando a 1 decimal.
- `int getSize()`: devuelve el número de preguntas en la encuesta (tamaño de la lista questions).
- `void addQuestion(Question question)`: añade una nueva pregunta al final de la lista.
- `void removeQuestion(int index)`: elimina la pregunta en el índice especificado.
- `void reorderQuestion(int oldIndex, int newIndex)`: mueve una pregunta de una posición a otra dentro de la lista, manteniendo el resto.
- `void updateQuestion(int index, Question question)`: sustituye la pregunta en la posición indicada por una nueva instancia.
- `Question getQuestion(int index)`: devuelve la pregunta en el índice indicado.
- `void clearQuestions()`: elimina todas las preguntas de la encuesta.

User: Clase que representa un usuario del sistema; gestiona sus datos básicos, credenciales y las encuestas que ha creado o respondido.

Atributos:

- `USERNAME`: String privado y final que identifica de forma única al usuario en el sistema.
- `email`: String privado que almacena el correo electrónico del usuario (cambiable).
- `passwordHash`: String privado que almacena la contraseña en forma de hash.
- `securityQuestion`: String privado que almacena la pregunta de seguridad para recuperación de contraseña.
- `securityAnswer`: String privado que almacena la respuesta a la pregunta de seguridad.
- `createdSurveyId`: Set<String> privado y final que contiene los identificadores de las encuestas creadas por el usuario.
- `respondedSurveyId`: Map<String, Set<String>> privado y final que asocia cada surveyId con el conjunto de responsId que el usuario ha enviado para esa encuesta.

Métodos:

- `User(String USERNAME, String email, String passwordHash)`: constructor que crea un usuario con nombre, email y contraseña, inicializando las colecciones de encuestas creadas y respondidas, y dejando los campos de seguridad a null.
- `User(String USERNAME, String email, String passwordHash, String securityQuestion, String securityAnswer)`: constructor que además inicializa pregunta y respuesta de seguridad.
- `String getUsername()`: devuelve el nombre de usuario.
- `String getEmail()`: devuelve el correo electrónico.
- `String getPasswordHash()`: devuelve el hash de la contraseña.
- `Set<String> getCreatedSurveyId()`: devuelve el conjunto de ids de encuestas creadas.
- `Map<String, Set<String>> getRespondedSurveyId()`: devuelve el mapa de encuestas respondidas.

- `String getSecurityQuestion()`: devuelve la pregunta de seguridad.
- `String getSecurityAnswer()`: devuelve la respuesta de seguridad.
- `void setEmail(String newEmail)`: actualiza el email del usuario (en el código real puede realizar validaciones).
- `void setPasswordHash(String newPassword)`: actualiza la contraseña (hash) del usuario.
- `void setSecurityQuestion(String securityQuestion)`: establece o cambia la pregunta de seguridad.
- `void setSecurityAnswer(String securityAnswer)`: establece o cambia la respuesta de seguridad.
- `void addCreatedSurveyId(String surveyId)`: añade un id de encuesta al conjunto de encuestas creadas por el usuario.
- `void removeCreatedSurveyId(String surveyId)`: elimina un id de encuesta del conjunto de encuestas creadas.
- `void addRespondedSurveyId(String surveyId, String responsId)`: registra que el usuario ha respondido a una encuesta determinada, añadiendo el responsId al conjunto correspondiente dentro de respondedSurveysId.

domain/clustering

ClusteringAlgorithm: Interfaz que define el contrato común para todos los algoritmos de clustering (K-Means, K-Means++, K-Medoids).

Métodos:

- `String getName()`: devuelve el nombre legible del algoritmo (por ejemplo, "K-Means").
- `String getDescription()`: devuelve una breve descripción del algoritmo.
- `ClusterResults execute(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance)`: ejecuta el algoritmo de clustering sobre una lista de respuestas y preguntas, con k clusters y una estrategia de cálculo de distancias, devolviendo los resultados (clusters, iteraciones, convergencia).

Centroid: Clase que representa el centroide de un cluster, modelado como un vector de componentes (una por pregunta).

Atributos:

- `components`: `Object[]` privado que almacena el valor representativo de cada dimensión/pregunta (puede ser `double[]`, `Double`, `String`, etc. según tipo de pregunta).

Métodos:

- `Centroid(int numComponents)`: constructor que crea un centroide con un número fijo de componentes inicializados a null.
- `int size()`: devuelve el número de componentes del centroide.
- `void setComponent(int index, Object value)`: establece el valor de una componente en la posición indicada.
- `Object getComponent(int index)`: devuelve el valor almacenado en la componente indicada.
- `Centroid clone()`: devuelve una copia independiente del centroide, con las mismas componentes.

ClusterMembership: Clase que representa la pertenencia de una respuesta a un cluster concreto, incluyendo la distancia al centroide.

Atributos:

- **response:** Response privado que referencia la respuesta asignada al cluster.
- **cluster:** Cluster privado que referencia el cluster al que pertenece.
- **distanceToCentroid:** double privado que almacena la distancia de la respuesta al centroide del cluster.

Métodos:

- **ClusterMembership(Response response, Cluster cluster, double distanceToCentroid):** constructor que inicializa la pertenencia con la respuesta, cluster y distancia.
- **Response getResponse():** devuelve la respuesta asociada a esta pertenencia.
- **Cluster getCluster():** devuelve el cluster al que está asociada la respuesta.
- **double getDistance():** devuelve la distancia de la respuesta al centroide.

Cluster: Clase que modela un grupo de respuestas (cluster), con un identificador, etiqueta, centroide y lista de miembros.

Atributos:

- **id:** String privado que identifica el cluster (por ejemplo, "Cluster 1").
- **label:** String privado que puede almacenar un nombre o descripción legible del cluster.
- **centroid:** Centroid privado que representa el centro geométrico/representativo del cluster.
- **members:** List<ClusterMembership> privada que contiene las respuestas pertenecientes al cluster junto con su distancia al centroide.

Métodos:

- **Cluster(String id, String label):** constructor que crea un cluster con identificador y etiqueta, inicializando la lista de miembros vacía y el centroide a null hasta que se calcule.
- **String getId():** devuelve el identificador del cluster.
- **String getLabel():** devuelve la etiqueta descriptiva del cluster.
- **Centroid getCentroid():** devuelve el centroide asociado.
- **void setCentroid(Centroid centroid):** asigna el centroide al cluster.
- **List<ClusterMembership> getMembers():** devuelve la lista de miembros (respuestas) del cluster.
- **void addMember(Response response, double distanceToCentroid):** crea una nueva ClusterMembership con la respuesta y la distancia indicada y la añade a la lista de miembros.
- **int getSize():** devuelve el número de miembros en el cluster.
- **double getAverageDistanceToCentroid():** calcula y devuelve la distancia media de sus miembros al centroide (0.0 si no hay miembros).

ClusterResults: Clase que representa el resultado completo de una ejecución de clustering: lista de clusters, metadatos y métricas básicas.

Atributos:

- **clusters:** List<Cluster> privada que contiene todos los clusters producidos por el algoritmo.

- **analysisId**: String privado que identifica esta ejecución/análisis de clustering (para guardarla y referenciarla).
- **surveyId**: String privado que identifica la encuesta sobre la que se ha realizado el clustering.
- **silhouetteScore**: double privado que almacena el valor de la métrica Silhouette calculada para este resultado (o un valor por defecto si no se ha calculado).
- **iterations**: int privado que indica cuántas iteraciones realizó el algoritmo antes de converger o detenerse.
- **converged**: boolean privado que indica si el algoritmo alcanzó convergencia según el umbral configurado.

Métodos:

- **ClusterResults(List<Cluster> clusters, int iterations, boolean converged)**: constructor que crea un resultado de clustering con la lista de clusters, número de iteraciones y bandera de convergencia (sin analysisId/surveyId ni métricas todavía).
- **ClusterResults(List<Cluster> clusters, String analysisId, String surveyId, double silhouetteScore, int iterations, boolean converged)**: constructor completo que guarda clusters, ids de análisis y encuesta, métrica Silhouette, iteraciones y convergencia.
- **List<Cluster> getClusters()**: devuelve la lista de clusters.
- **Cluster getCluster(String id)**: busca y devuelve el cluster cuyo identificador coincide con el indicado (o null si no existe).
- **double getSilhouetteScore()**: devuelve el valor de la métrica Silhouette.
- **void setSilhouetteScore(double silhouetteScore)**: establece/actualiza el valor de la métrica Silhouette.
- **String getAnalysisId()**: devuelve el identificador del análisis de clustering.
- **String getSurveyId()**: devuelve el identificador de la encuesta asociada.
- **int getIterations()**: devuelve el número de iteraciones del algoritmo.
- **boolean isConverged()**: indica si el algoritmo declaró convergencia.

DistanceType: Enumeración que define los tipos de distancia numérica soportados para calcular similitudes entre respuestas o centroides.

Valores:

- **EUCLIDEAN**: distancia euclídea.
- **MANHATTAN**: distancia Manhattan.
- **COSINE**: distancia basada en coseno.

TextDistanceType: Enumeración que define las métricas de distancia entre textos soportadas.

Valores:

- **LEVENSHTEIN**: distancia de edición de Levenshtein.
- **COSINE**: similitud de coseno aplicada a representaciones textuales.
- **EMBEDDING**: distancia sobre embeddings (vectorizaciones) de texto.

DistanceCalculator: Clase que centraliza el cálculo de distancias entre respuestas y centroides, tanto numéricas como textuales, con distintos tipos de distancia y pesos por pregunta.

Atributos:

- **distanceType**: DistanceType privado que indica la métrica numérica principal (EUCLIDEAN, MANHATTAN, COSINE).
- **textDistanceType**: TextDistanceType privado que indica la métrica a usar para textos (LEVENSHTEIN, COSINE, EMBEDDING).
- **weights**: Map<Integer, Double> privado que asocia índices de pregunta con pesos específicos en el cálculo de distancia (por defecto peso 1.0 si no se define).

Métodos:

- **DistanceCalculator(DistanceType distanceType, TextDistanceType textDistanceType)**: constructor que inicializa el tipo de distancia numérica y de texto, y crea el mapa de pesos vacío.
- **void setWeight(int questionIndex, double weight)**: establece un peso específico para una pregunta concreta en el cálculo de distancias.
- **double calculate(Response r1, Response r2, List<Question> questions)**: calcula la distancia total entre dos respuestas completas, combinando diferencias para cada pregunta según su tipo, aplicando pesos y la métrica configurada.
- **double calculateToCentroid(Response r, Centroid centroid, List<Question> questions)**: calcula la distancia entre una respuesta y un centroide, comparando el valor de cada componente según el tipo de pregunta.
- **double euclideanDistance(double[] v1, double[] v2)**: calcula la distancia euclídea entre dos vectores numéricos.
- **double manhattanDistance(double[] v1, double[] v2)**: calcula la distancia Manhattan entre dos vectores.
- **double cosineDistance(double[] v1, double[] v2)**: calcula la distancia basada en coseno entre dos vectores.
- **double calculateTextDistance(String s1, String s2)**: calcula la distancia entre dos textos según el TextDistanceType actual (por ejemplo, Levenshtein, coseno sobre bags of words o embeddings).
- **double calculateLevenshteinDistance(String s1, String s2)**: implementa la distancia de Levenshtein entre dos cadenas.
- **double calculateCosineTextDistance(String s1, String s2)**: implementa una distancia basada en coseno entre representaciones vectoriales de textos.
- **double calculateEmbeddingDistance(String s1, String s2)**: método de stub/extensión para distancia basada en embeddings (puede ser simplificada o dejada para futuras mejoras).

KMeans: Implementación del algoritmo de clustering K-Means estándar, usando centroides medios y una estrategia de inicialización configurable.

Atributos:

- **maxIterations**: int privado que indica el número máximo de iteraciones permitidas antes de detener el algoritmo.
- **tolerance**: double privado que indica el umbral de convergencia (movimiento máximo permitido de centroides).
- **randomSeed**: long privado que almacena la semilla para el generador aleatorio, permitiendo reproducibilidad.
- **random**: Random privado usado para inicializar centroides de forma aleatoria.

Métodos públicos principales:

- **KMeans(int maxIterations, double tolerance):** constructor que valida los parámetros (maxIterations > 0, tolerance >= 0) y configura el algoritmo.
- **KMeans():** constructor por defecto que configura 100 iteraciones máximas y una tolerancia de 1e-4.
- **void setRandomSeed(long seed):** establece una semilla concreta para el generador aleatorio y recrea el Random.
- **String getName():** devuelve el nombre del algoritmo ("K-Means").
- **String getDescription():** devuelve una breve descripción de K-Means.
- **ClusterResults execute(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance):** ejecuta K-Means completo, devolviendo los clusters y metadatos.

Métodos privados auxiliares:

- **List<Centroid> initialize(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance):** inicializa k centroides (típicamente seleccionando respuestas aleatorias o medias iniciales).
- **Integer[] assignPoints(List<Response> responses, List<Question> questions, List<Centroid> centroids, DistanceCalculator distance):** asigna cada respuesta al centroide más cercano y devuelve un array de índices de cluster.
- **List<Centroid> updateCentroids(List<Response> responses, List<Question> questions, Integer[] assignments, int k, DistanceCalculator distance):** recalcula los centroides a partir de las asignaciones actuales.
- **boolean hasConverged(List<Centroid> oldCentroids, List<Centroid> newCentroids, List<Question> questions, DistanceCalculator distance):** verifica si el movimiento total de los centroides es menor que la tolerancia.
- **List<Cluster> createClusters(List<Response> responses, List<Question> questions, Integer[] assignments, List<Centroid> centroids, DistanceCalculator distance):** construye la lista final de Cluster, asignando cada respuesta como miembro correspondiente con su distancia al centroide.

KMeansPlusPlus: Implementación del algoritmo K-Means con inicialización mejorada de centroides (K-Means++), que elige centroides iniciales más dispersos.

Atributos:

- **maxIterations:** int privado que indica el número máximo de iteraciones.
- **tolerance:** double privado que indica el umbral de convergencia.
- **randomSeed:** long privado para la semilla del generador aleatorio.
- **random:** Random privado usado para la inicialización K-Means++.

Métodos públicos:

- **KMeansPlusPlus(int maxIterations, double tolerance):** constructor que valida y fija iteraciones y tolerancia.
- **KMeansPlusPlus():** constructor por defecto que configura 100 iteraciones y tolerancia 1e-4.
- **void setRandomSeed(long seed):** establece la semilla y recrea el Random.
- **String getName():** devuelve el nombre del algoritmo ("K-Means++", aunque en el snippet se ve "K-Means" como nombre legible).
- **String getDescription():** devuelve la descripción ("Variante mejorada de K-Means con inicialización inteligente de centroides").

- `ClusterResults execute(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance)`: ejecuta K-Means completo pero usando inicialización K-Means++ para los centroides, devolviendo los resultados.

Métodos privados auxiliares:

- `List<Centroid> initializeCentroidsPlusPlus(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance)`: aplica la lógica K-Means++ para elegir centroides iniciales con probabilidad proporcional a la distancia al centroide más cercano.
- `static Centroid createCentroidFromResponse(Response r, List<Question> questions)`: crea un centroide a partir de una respuesta, extrayendo valor para cada pregunta.
- `static boolean isAnswered(Answer a)`: determina si una respuesta individual contiene datos válidos (para saltarse respuestas vacías).
- `static Object getAnswerValue(Answer a, Question q)`: extrae el valor representativo de una respuesta según el tipo de pregunta (vectores para múltiple choice, texto, número...).
- `Integer[] assignToClusters(List<Response> responses, List<Question> questions, List<Centroid> centroids, DistanceCalculator distance)`: asigna cada respuesta al centroide más cercano.
- `List<Centroid> updateCentroids(List<Response> responses, List<Question> questions, Integer[] assignments, int k, DistanceCalculator distance)`: recalcula los centroides a partir de las respuestas asignadas, promediando/votando según tipo de pregunta.
- `boolean hasConverged(List<Centroid> oldCentroids, List<Centroid> newCentroids, List<Question> questions, DistanceCalculator distance)`: calcula movimiento total entre centroides y lo compara con tolerance.
- `String findTextMedoid(List<String> texts, DistanceCalculator distance)`: para preguntas textuales, encuentra el texto medoid (el que minimiza la suma de distancias a los demás).
- `List<Cluster> createClusters(List<Response> responses, List<Question> questions, Integer[] assignments, List<Centroid> centroids, DistanceCalculator distance)`: construye la lista final de Cluster.

KMedoids: Implementación del algoritmo de clustering K-Medoids (PAM: Partitioning Around Medoids), que usa puntos reales del dataset como centros de cluster.

Atributos:

- `maxIterations`: int privado que indica el número máximo de iteraciones permitidas.
- `tolerance`: double privado que indica el umbral de convergencia basado en el coste total.
- `randomSeed`: long privado con la semilla de aleatoriedad.
- `random`: Random privado usado para seleccionar medoides iniciales y re-inicializarlos si hay clusters vacíos.

Métodos públicos:

- `KMedoids(int maxIterations, double tolerance)`: constructor que valida y fija iteraciones y tolerancia.
- `KMedoids()`: constructor por defecto con 100 iteraciones y tolerancia 1e-4.
- `void setRandomSeed(long seed)`: establece la semilla del generador aleatorio.

- `String getName()`: devuelve el nombre del algoritmo ("K-Medoids").
- `String getDescription()`: devuelve una descripción indicando que usa puntos reales del dataset como representantes.
- `ClusterResults execute(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance)`: ejecuta K-Medoids completo, devolviendo clusters, iteraciones y convergencia.

Métodos privados auxiliares:

- `int[] initializeMedoids(int numPoints, int k)`: selecciona aleatoriamente k índices distintos de respuestas como medoides iniciales.
- `Integer[] assignToClusters(List<Response> responses, List<Question> questions, int[] medoidIndices, DistanceCalculator distance)`: asigna cada respuesta al medoide más cercano.
- `int[] updateMedoids(List<Response> responses, List<Question> questions, Integer[] assignments, int k, DistanceCalculator distance)`: para cada cluster, selecciona como nuevo medoide el punto que minimiza la suma de distancias dentro del cluster; si un cluster queda vacío, re-inicializa el medoide al azar.
- `double calculateTotalCost(List<Response> responses, List<Question> questions, Integer[] assignments, int[] medoidIndices, DistanceCalculator distance)`: calcula la suma de distancias de cada punto a su medoide (coste total).
- `List<Cluster> createClusters(List<Response> responses, List<Question> questions, Integer[] assignments, int[] medoidIndices, DistanceCalculator distance)`: construye los clusters finales usando como centroides los medoides (respuestas de referencia).

QualityMetricType: Enumeración que define las métricas de calidad de clustering soportadas.

Valores:

- **SILHOUETTE**: métrica de Silhouette.
- **INERTIA**: suma de distancias dentro de cluster (inercia).
- **CALINSKI**: índice Calinski-Harabasz (o similar, abreviado CALINSKI).

QualityMetricCalculator: Clase que calcula métricas de calidad de clustering sobre unos resultados ya generados (por ejemplo Silhouette), usando el DistanceCalculator.

Atributos:

- `metricType`: QualityMetricType privado que indica qué métrica se va a calcular (SILHOUETTE, INERTIA, CALINSKI).
- `distanceCalculator`: DistanceCalculator privado que se usa para medir distancias entre respuestas y centroides en las métricas.

Métodos:

- `QualityMetricCalculator(QualityMetricType metricType, DistanceCalculator distanceCalculator)`: constructor que establece el tipo de métrica y la calculadora de distancias a usar.
- `double calculate(ClusterResults results, List<Question> questions)`: calcula la métrica configurada sobre los clusters dados; delega en métodos internos según metricType.
- `double calculateSilhouette(ClusterResults results, List<Question> questions)`: calcula la métrica Silhouette promediando para todos los puntos la diferencia entre cohesión interna y separación con el cluster vecino más cercano.

- `double calculateInertia(ClusterResults results, List<Question> questions)`: calcula la suma de distancias de cada punto a su centroide (inercia total).
- `double calculateCalinski(ClusterResults results, List<Question> questions)`: calcula el índice de Calinski-Harabasz u otra métrica equivalente de varianza entre/dentro de clusters.

domain.controller

AnalysisController: Controlador que gestiona los análisis de clustering de una encuesta, actuando como fachada sobre los algoritmos de clustering y las métricas de calidad. Mantiene en memoria un repositorio de análisis creados durante la ejecución.

Atributos:

- `analyses`: Map<String, ClusteringAnalysis> privado que almacena en memoria los análisis de clustering creados, indexados por su identificador.
- `repository`: AnalysisRepository privado (previsto para el futuro) que permitiría persistir los análisis en vez de guardarlos solo en memoria.
- `metricCalculator`: QualityMetricCalculator privado que se usa para calcular métricas de calidad (Silhouette, Calinski, Davies-Bouldin, etc.) a partir de los resultados de clustering.

Métodos:

- `AnalysisController()`: constructor que inicializa el mapa analyses y crea un QualityMetricCalculator por defecto.
- `ClusteringAnalysis createAnalysis(Survey survey, Integer k, String algorithmType, Map<String, Object> config)`: crea un nuevo análisis de clustering para una encuesta dada, con k clusters y un tipo de algoritmo; construye el algoritmo con createAlgorithm, crea la instancia de ClusteringAnalysis, la guarda en analyses y la devuelve.
- `ClusteringAlgorithm createAlgorithm(String type, Map<String, Object> config)`: crea un algoritmo de clustering en función del nombre (KMeans, KMeansPlusPlus, KMedoids) y de parámetros como maxIterations y tolerance obtenidos del mapa config; lanza IllegalArgumentException si el tipo es desconocido.
- `ClusteringAnalysis getAnalysis(String id)`: devuelve el análisis de clustering asociado al identificador indicado, o null si no existe.
- `List<ClusteringAnalysis> getAnalysesByQuestionSet(String surveyId)`: devuelve una lista de análisis cuya encuesta asociada tiene el surveyId dado, recorriendo los valores del mapa analyses.
- `void executeAnalysis(String id, List<Response> responses, List<Question> questions, DistanceCalculator distance)`: ejecuta el análisis con id dado llamando a execute sobre el ClusteringAnalysis correspondiente con respuestas, preguntas y calculadora de distancia; lanza IllegalArgumentException si no se encuentra el análisis.
- `Double calculateSilhouette(String analysisId, DistanceCalculator distance)`: calcula el coeficiente de Silhouette para el análisis indicado usando metricCalculator.calculateSilhouette; si no existe el análisis devuelve 0.0.
- `Double calculateQuality(String id, QualityMetricType metricType)`: calcula una métrica de calidad concreta (Silhouette, Calinski, Davies-Bouldin, etc.) para el análisis con id

dato, delegando en `analysis.calculateQuality`; lanza `IllegalArgumentException` si no existe el análisis.

- `boolean deleteAnalysis(String id)`: elimina del mapa el análisis asociado a ese id y devuelve true si existía y fue eliminado, false en caso contrario.
- `Integer determineOptimalK(List<Response> responses, List<Question> questions, Survey survey, Integer minK, Integer maxK)`: determina el número óptimo de clusters k probando valores entre minK y maxK, ejecutando un `ClusteringAnalysis` con `KMeansPlusPlus` y distancia `EUCLIDEAN` + texto `LEVENSHTEIN` para cada k, evaluando la métrica `Silhouette`, y devolviendo el k con mejor puntuación; lanza `IllegalArgumentException` si los parámetros o la lista de respuestas no son válidos.
- `List<String> getAvailableAlgorithms()`: devuelve una lista de nombres de algoritmos disponibles: "KMeans", "KMeansPlusPlus", "KMedoids".
- `void clearAnalyses()`: elimina todos los análisis almacenados en el mapa `analyses`.
- `Map<String, ClusteringAnalysis> getAnalyses()`: devuelve el propio mapa de análisis almacenados (para inspección o depuración).

AnswerController: Controlador responsable de las operaciones relacionadas con respuestas individuales (`Answer`), actuando como intermediario entre la vista y `AnswerService`. Está preparado para ampliarse con operaciones específicas de creación/actualización/validación de respuestas.

Atributos:

- `answerService`: `AnswerService` privado y final que encapsula la lógica de negocio asociada a las respuestas.

Métodos:

- `AnswerController(AnswerService answerService)`: constructor que inyecta y almacena la instancia de `AnswerService` que se usará desde el controlador.

CtrlDominioClustering: Controlador de dominio que orquesta el flujo completo de clustering sobre encuestas, combinando repositorios de datos, `AnalysisController` y funciones de exportación/consulta de resultados y métricas.

Atributos:

- `analysisController`: `AnalysisController` privado y final que se encarga de crear, ejecutar y gestionar análisis de clustering.
- `responseRepository`: `ResponseRepository` privado y final usado para acceder a las respuestas almacenadas de una encuesta.
- `surveyRepository`: `SurveyRepository` privado y final usado para leer encuestas y sus preguntas asociadas.

Métodos:

- `CtrlDominioClustering(ResponseRepository responseRepository, SurveyRepository surveyRepository)`: constructor que inicializa el controlador de análisis y guarda las referencias a los repositorios de respuestas y encuestas.
- `int getResponseCount(String surveyId)`: obtiene el número de respuestas de una encuesta buscando en `responseRepository.getResponseBySurveyId(surveyId)` y devolviendo su tamaño (0 si es null).
- `String runAnalysis(String algorithmName, String surveyId, int k, int maxIter, double tolerance, String distanceMetric, String textDistanceMetric)`: ejecuta un análisis de clustering para una encuesta y configuración concretas: carga la encuesta,

comprueba que existe; obtiene las respuestas; valida el rango de k; construye un mapa config con maxIterations y tolerance; crea el análisis con AnalysisController, obtiene las preguntas de la encuesta, crea un DistanceCalculator con DistanceType y TextDistanceType según las cadenas proporcionadas, llama a executeAnalysis y devuelve el id del análisis creado.

- **String exportarAnálisis(String analysisId, String filePath) throws IOException:** exporta los resultados del análisis indicado a un fichero de texto; obtiene el análisis de AnalysisController, genera un string con analysis.exportResults, crea el fichero (y directorio padre si no existe), escribe con FileWriter y devuelve la ruta absoluta del archivo; lanza IllegalArgumentException si el análisis no existe.
- **String obtenerMetricasCalidad(String analysisId):** construye y devuelve un informe de métricas de calidad para el análisis dado; incluye métricas globales (Silhouette, Calinski-Harabasz, Davies-Bouldin) y, para cada cluster, su tamaño y coeficiente de Silhouette; si el análisis no ha sido ejecutado o no contiene datos suficientes captura IllegalStateException y devuelve un mensaje de error.
- **String encontrarKOptima(String surveyId, String algorithmName, int maxIter, double tolerance, String distanceMetric):** ejecuta una búsqueda de k óptima para una encuesta y algoritmo concretos, probando k desde 2 hasta 10, creando y ejecutando análisis temporales con runAnalysis, calculando la Silhouette para cada uno, acumulando un informe por k y seleccionando el mejor k; finalmente devuelve un texto con el resumen y la recomendación, o un mensaje indicando que no se pudo determinar k si todos fallan. Cada análisis temporal se borra al final con deleteAnalysis.
- **List<String> listarAnálisis():** devuelve la lista de todos los ids de análisis almacenados, ordenados desde el más reciente al más antiguo según analysis.getAnalysisDate().
- **void deleteAnalysis(String analysisId):** elimina un análisis concreto delegando en analysisController.deleteAnalysis(analysisId).
- **String obtenerResultadosAnálisis(String analysisId):** devuelve un texto con los resultados completos del análisis invocando analysis.exportResults; lanza IllegalArgumentException si el análisis no existe.
- **void limpiarAnálisis():** borra todos los análisis almacenados llamando a analysisController.clearAnalyses().
- **int calculateOptimalK(String surveyId, int maxK):** calcula el número óptimo de clusters k usando una variante del método del codo sobre Silhouette: carga la encuesta, obtiene las respuestas, limita maxK por el número de respuestas, asegura al menos 2 clusters si es posible, itera k desde 2 hasta el límite, ejecuta análisis con KMeansPlusPlus y distancia EUCLIDEAN/LEVENSHTEIN, calcula Silhouette y guarda el mejor; devuelve el k óptimo encontrado (o 1 si no hay suficientes respuestas).

QuestionController: Controlador que gestiona las operaciones de alto nivel sobre preguntas dentro de encuestas (añadir, eliminar, actualizar y reordenar), delegando en QuestionService.

Atributos:

- **questionService:** QuestionService privado y final que contiene la lógica de negocio para manejar preguntas y la persistencia vía repositorio.

Métodos:

- `QuestionController(QuestionService questionService)`: constructor que recibe e inyecta el servicio de preguntas.
- `void addQuestion(String surveyId, Question question)`: añade una nueva pregunta a la encuesta con id dado, delegando en `questionService`.
- `void removeQuestion(String surveyId, int index)`: elimina la pregunta en la posición indicada de la encuesta dada.
- `void updateQuestion(String surveyId, int index, Question updatedQuestion)`: actualiza la pregunta existente en la encuesta con la nueva definición.
- `Question getQuestion(String surveyId, int index)`: recupera una pregunta concreta por índice dentro de la encuesta dada.
- `void reorderQuestions(String surveyId, List<Integer> newOrder)`: reordena las preguntas de la encuesta según una nueva lista de índices o posiciones, delegando en el servicio.

ResponseController: Controlador que gestiona las respuestas completas a encuestas (`Response`), coordinando creación, edición, envío y consultas, usando `ResponseService` y `AnswerService`.

Atributos:

- `responseService`: `ResponseService` privado y final que implementa la lógica de negocio para gestionar respuestas completas.
- `answerService`: `AnswerService` privado y final que gestiona operaciones sobre respuestas individuales a preguntas.

Métodos principales:

- `ResponseController(ResponseService responseService, AnswerService answerService)`: constructor que inyecta los servicios necesarios.
- `String startResponse(String surveyId)`: inicia una nueva respuesta para la encuesta indicada, creando un objeto `Response` en DRAFT a través del servicio y devolviendo su id.
- `void submitResponse(String surveyId, String responsId)`: marca la respuesta como enviada (SUBMITTED), comprobando que se cumplan las condiciones (por ejemplo, preguntas obligatorias respondidas) a través de `responseService`.
- `void editResponse(String surveyId, String responsId, ...)`: permite modificar una respuesta existente en estado borrador (los tests de integración muestran edición antes de envío).
- `List<Response> getResponses(String surveyId)`: devuelve todas las respuestas asociadas a una encuesta.
- `void updateAnswer(String surveyId, String responsId, int questionIndex, Object value, TypeQuestion type)`: actualiza una respuesta individual (Textual, Numerical o MultipleChoice) dentro de una `Response`, creando o modificando el objeto `Answer` apropiado y guardando los cambios en el repositorio.

SurveyController: Controlador responsable de la gestión de encuestas desde el punto de vista de la aplicación: creación, edición, eliminación, publicación y obtención de encuestas del usuario.

Atributos:

- **surveyService**: SurveyService privado y final que implementa la lógica de negocio y acceso a datos de encuestas.
- **questionService**: QuestionService privado y final que se utiliza para gestionar las preguntas asociadas a cada encuesta (crear, editar, borrar y reordenar).

Métodos:

- **SurveyController(SurveyService surveyService, QuestionService questionService)**: constructor que inyecta los servicios necesarios.
- **Survey createSurvey(String title, String description, String creatorUsername)**: crea una nueva encuesta en estado borrador con título, descripción y usuario creador, delegando en surveyService.
- **void editSurvey(String surveyId, String newTitle, String newDescription)**: permite editar el título y la descripción de una encuesta siempre que esté en un estado editable (por ejemplo, DRAFT).
- **void deleteSurvey(String surveyId)**: elimina la encuesta especificada, junto con sus preguntas asociadas.
- **void publishSurvey(String surveyId)**: cambia el estado de la encuesta a PUBLISHED y marca la fecha de publicación.
- **Survey getSurvey(String surveyId)**: recupera una encuesta por su identificador.
- **List<Survey> getSurveysByCreator(String creatorUsername)**: devuelve la lista de encuestas creadas por un usuario concreto, usada para "Mis encuestas" y similares.
- **void initializeNewSurvey(String title, String description, String creatorUsername)**: método usado desde la capa de presentación para crear y preparar una nueva encuesta con preguntas vacías antes de que se añadan preguntas

UserController: Controlador que gestiona toda la lógica de sesión y cuenta de usuario: registro, login, logout, eliminación de cuenta, cambio y recuperación de contraseña, y asociación con encuestas creadas.

Atributos:

- **userService**: UserService privado y final que encapsula la lógica de negocio relacionada con usuarios, credenciales y encuestas vinculadas.

Métodos:

- **UserController(UserService userService)**: constructor que inyecta el servicio de usuario.
- **void registerUser(String username, String email, String password, String securityQuestion, String securityAnswer)**: registra un nuevo usuario en el sistema validando datos y delegando en userService; puede lanzar RegisterException si los datos no son válidos.
- **void loginUser(String username, String password)**: inicia sesión de un usuario comprobando sus credenciales; puede lanzar LoginException si las credenciales son incorrectas.
- **void logoutUser()**: cierra la sesión actual (elimina el usuario autenticado de la sesión en memoria).
- **void deleteAccount(String username)**: elimina la cuenta del usuario y limpia sus encuestas/respuestas asociadas según la política del dominio.

- `void updatePassword(String username, String oldPassword, String newPassword)`: cambia la contraseña del usuario autenticado, verificando la contraseña anterior y delegando en `userService`.
- `void resetPassword(String email, String securityAnswer, String newPassword)`: flujo de recuperación de contraseña que verifica la respuesta a la pregunta de seguridad y establece una nueva contraseña.
- `boolean verifySecurityAnswer(String email, String securityAnswer)`: comprueba si la respuesta de seguridad proporcionada coincide con la almacenada para el correo dado.
- `String getUsernameLoggedIn()`: devuelve el nombre de usuario actualmente logueado (se usa en tests para asociar encuestas y respuestas).
- `void addSurveyCreated(String username, String surveyId)`: notifica al servicio de usuario que el usuario ha creado una nueva encuesta, para actualizar sus colecciones de encuestas creadas.

domain/service

ResponseService: Servicio que gestiona respuestas completas a encuestas, coordinando la creación, edición, envío y consultas de respuestas usando `ResponseRepository`, `SurveyService` y `UserController`.

Atributos:

- `responseRepository`: `ResponseRepository` - privado, final - Gestiona almacenamiento y recuperación de respuestas.
- `userController`: `UserController` - privado, final - Verifica qué usuario está logueado.
- `surveyService`: `SurveyService` - privado, final - Accede a encuestas y sus preguntas al crear respuestas nuevas.

Métodos:

- `ResponseService(ResponseRepository responseRepository, UserController userController, SurveyService surveyService)`: Constructor que inyecta los repositorios y servicios necesarios.
- `Response createResponse(String surveyId)`: Crea nueva respuesta en estado INCOMPLETE: obtiene encuesta desde `surveyService`, recupera lista de preguntas, crea `Response` sin `responsId` inicial con `surveyId` y `username` logueado, inicializa mapa de respuestas (`Answer`) según tipos de preguntas (`TextualAnswer`, `MultipleChoiceAnswer`, `NumericalAnswer`), devuelve respuesta creada aún no persistida.
- `void submitResponse(String surveyId, String responsId)`: Marca respuesta como SUBMITTED: obtiene respuesta del repositorio, verifica que todas preguntas obligatorias (`required=true`) tengan respuestas válidas (`isAnswered=true`), lanza `ResponseException` si validaciones fallan, cambia estado a SUBMITTED, establece fecha de envío con `LocalDateTime.now()`, persiste con `responseRepository`.
- `void editResponse(String surveyId, String responsId, Map<Integer, Answer> updatedAnswers)`: Edita respuesta en estado INCOMPLETE: obtiene respuesta del repositorio, comprueba que estado sea INCOMPLETE, reemplaza mapa de respuestas por `updatedAnswers` validando consistencia, persiste cambios, lanza `ResponseException` si respuesta está enviada o no existe.

- `List<Response> getResponses(String surveyId)` : Devuelve todas respuestas de una encuesta desde `responseRepository`.
- `Response getResponse(String surveyId, String responseId)`: Obtiene respuesta específica desde `responseRepository` dados `surveyId` y `responseId`.
- `void updateAnswer(String surveyId, String responseId, int questionIndex, Object value, TypeQuestion type)`: Actualiza respuesta individual en `Response`: obtiene respuesta y pregunta, crea nuevo `Answer` del tipo apropiado (`TextualAnswer` para `TEXTUAL`, `NumericalAnswer` para `NUMERICAL`, `MultipleChoiceAnswer` para `MULTIPLE_CHOICE`), marca como respondida (`isAnswered=true`), reemplaza `Answer` anterior en mapa, persiste con `responseRepository`.
- `void deleteResponse(String surveyId, String responseId)`: Elimina respuesta específica del repositorio.
- `List<Response> getMyResponses()`: Devuelve todas respuestas del usuario logueado filtrando por `respondentUsername`.
- `void updateResponseStatus(String surveyId, String responseId, ResponseStatus newStatus)`: Cambia estado de respuesta (`INCOMPLETE` a `SUBMITTED` o viceversa), actualiza fecha de envío si cambia a `SUBMITTED`, persiste cambios.

AnswerService: Servicio que gestiona operaciones específicas sobre respuestas individuales a preguntas (`Answer`), incluyendo validación y formateo de respuestas.

Atributos:

- `answerRepository`: `AnswerRepository` - privado, final - Repositorio que gestiona almacenamiento de respuestas individuales.

Métodos:

- `AnswerService(AnswerRepository answerRepository)`: Constructor que inyecta repositorio de respuestas individuales.
- `TextualAnswer createTextualAnswer(int questionIndex, String text)`: Crea respuesta textual: valida que texto no sea null, crea `TextualAnswer` con índice y texto, marca como respondida (`isAnswered=true`) si texto no está vacío, devuelve respuesta.
- `NumericalAnswer createNumericalAnswer(int questionIndex, Double value)`: Crea respuesta numérica: valida que valor no sea null, crea `NumericalAnswer` con índice y valor, marca como respondida si valor no es null, devuelve respuesta.
- `MultipleChoiceAnswer createMultipleChoiceAnswer(int questionIndex, int numOptions, boolean[] selections)`: Crea respuesta opción múltiple: valida que `selections` tenga longitud correcta, crea `MultipleChoiceAnswer` con array de selecciones, marca como respondida si al menos una opción está seleccionada, devuelve respuesta.
- `boolean validateAnswer(Answer answer, Question question)`: Valida que respuesta sea coherente con pregunta: comprueba que índice coincida, valida según tipo (`TextualAnswer` comprueba máximo de caracteres, `NumericalAnswer` comprueba rango, `MultipleChoiceAnswer` comprueba min/max selecciones), devuelve true si válida, false en caso contrario.
- `String formatAnswerForDisplay(Answer answer)`: Devuelve representación en texto de respuesta formateada para mostrar: para `TextualAnswer` devuelve texto, para `NumericalAnswer` devuelve número como string, para `MultipleChoiceAnswer` devuelve lista de índices o nombres de opciones seleccionadas.

- `void deleteAnswer(int questionIndex)`: Marca Answer como no respondido o lo elimina del repositorio.

QuestionService: Servicio que encapsula operaciones sobre preguntas dentro de encuestas (CRUD y reordenación), delegando en QuestionRepository y verificando permisos del usuario.

Atributos:

- `questionRepository`: QuestionRepository - privado, final - Gestiona almacenamiento y recuperación de preguntas por encuesta.
- `userController`: UserController - privado, final - Verifica qué usuario está logueado y sus permisos.

Métodos:

- `QuestionService(QuestionRepository questionRepository, UserController userController)` - Constructor que inyecta repositorio de preguntas y controlador de usuario.
- `void addQuestion(String surveyId, Question question)` - Añade pregunta a encuesta: valida que usuario actual sea creador, comprueba que pregunta sea válida (texto no vacío, tipo no null), calcula siguiente índice (`getNumQuestions + 1`), establece `questionIndex` y `SURVEYID` en pregunta, delega en `questionRepository.addQuestion` para persistir.
- `void removeQuestion(String surveyId, int questionIndex)` - Elimina pregunta por índice: valida que usuario sea creador, comprueba que índice sea válido, delega en `questionRepository.removeQuestion`, reajusta índices de preguntas siguientes si es necesario.
- `void updateQuestion(String surveyId, int questionIndex, Question updatedQuestion)` - Actualiza pregunta existente: valida que usuario sea creador, obtiene pregunta actual, valida que `updatedQuestion` sea válida, copia campos nuevos (`questionText`, `required`, tipo), si es `MultipleChoiceQuestion` también actualiza opciones y min/max selecciones, persiste con `questionRepository.updateQuestion`.
- `Question getQuestion(String surveyId, int questionIndex)` - Obtiene pregunta específica desde `questionRepository` dados `surveyId` e índice.
- `void reorderQuestions(String surveyId, List<Integer> newOrder)` - Reordena preguntas según lista de índices nuevos: valida que `newOrder` contenga todos índices sin duplicados, reajusta `questionIndex` de cada pregunta según nuevo orden, persiste todos cambios.
- `List<Question> getQuestions(String surveyId)` - Devuelve todas preguntas de encuesta desde `questionRepository`.
- `int getQuestionCount(String surveyId)` - Devuelve número total de preguntas en encuesta.
- `void validateQuestion(Question question)` - Auxiliar que lanza excepción si pregunta inválida: comprueba que `questionText` no sea null ni vacío, que `typeQuestion` no sea null, que si es `MultipleChoiceQuestion` tenga al menos una opción y `minSelections <= maxSelections <= numOptions`.

SurveyService: Servicio que encapsula lógica de negocio de encuestas: validaciones, creación con IDs autoincrementales, actualización, publicación, eliminación, gestión de preguntas y contadores de vistas/rating.

Atributos:

- **surveyRepository:** SurveyRepository - privado, final - Gestiona almacenamiento y recuperación de encuestas.
- **userController:** UserController - privado, final - Verifica usuario logueado y permisos de propietario.
- **userService:** UserService - privado, final - Registra o elimina encuestas del perfil de usuario.

Métodos auxiliares:

- **SurveyService(SurveyRepository surveyRepository, UserController userController, UserService userService)** - Constructor que inyecta repositorio y servicios de usuario.
- **void checkUserLoggedIn()** - Verifica que haya usuario logueado; lanza SurveyException si no.
- **void checkSurveyExists(String surveyId)** - Verifica que encuesta existe; lanza SurveyException si no.
- **void checkQuestionExists(String surveyId, int questionIndex)** - Comprueba que existe encuesta y que índice está en rango [0, numQuestions); lanza SurveyException si no.
- **String generateUniqueSurveyId()** Delega en surveyRepository.generateNextSurveyId para obtener nuevo ID autoincremental único.

Métodos de creación:

- **Survey createSurvey(Survey survey)** - Crea y persiste nueva encuesta: valida usuario logueado, título y descripción no vacíos, al menos una pregunta, que CREATORUSERNAME coincida con usuario logueado; si SURVEYID es null/vacío genera uno nuevo; crea nueva instancia de Survey con ID definitivo y copia profunda de preguntas (si MultipleChoiceQuestion hace copia segura de opciones con ajuste fallback de min/max); comprueba que no exista ya si se pasó ID; guarda en surveyRepository; registra en UserService.addSurveyCreated (capturando excepciones); devuelve encuesta persistida.
- **Survey initializeNewSurvey(String title, String description, String creatorUsername)** - Valida usuario logueado, título y descripción no vacíos; devuelve nueva Survey sin SURVEYID (null) en estado DRAFT aún no almacenada; usado como paso previo en UI.

Métodos de recuperación:

- **List<Survey> getSelectedSurveys()**- Requiere usuario logueado; devuelve surveyRepository.getAllSurveys.
- **List<String> getSurveyIds()**- Requiere usuario logueado; devuelve surveyRepository.getAllSurveyIds.
- **int getNumQuestions(String surveyId)**- Verifica existencia; devuelve surveyRepository.getNumQuestions(surveyId).
- **List<Question> getQuestions(String surveyId)**- Verifica existencia, obtiene surveyRepository.getAllQuestions; si vacía lanza SurveyException; si no, devuelve lista.
- **Question getQuestion(String surveyId, int questionIndex)**- Verifica existencia de pregunta; devuelve surveyRepository.getQuestion(surveyId, questionIndex).
- **Survey getSurvey(String surveyId)** - Verifica existencia; devuelve surveyRepository.getSurvey(surveyId).

- `List<Survey> getMySurveys()` - Requiere usuario logueado; devuelve encuestas del usuario actual mediante `surveyRepository.getSurveysByUsername(userController.getLoggedUser().getUsername())`.

Métodos de validación:

- `boolean existsQuestion(String surveyId, int questionIndex)` - Verifica que existe encuesta, obtiene número de preguntas; devuelve si índice está en rango [0, numQuestions).
- `boolean existsSurvey(String surveyId)` - Devuelve `surveyRepository.existsSurvey(surveyId)`.

Métodos de CRUD completo:

- `Survey updateSurvey(String surveyId, Survey updatedSurvey)` - Valida usuario logueado y existencia, obtiene Survey actual, verifica que usuario actual sea creador, comprueba que estado sea DRAFT, valida título y descripción de `updatedSurvey` no vacíos; crea copia profunda de preguntas (como en `createSurvey`), limpia preguntas actuales y añade nuevas copias, guarda con `surveyRepository.addSurvey`, imprime LOG; devuelve encuesta actualizada.
- `void deleteSurvey(String surveyId)` - Requiere usuario logueado y existencia, verifica que usuario actual sea creador, comprueba que estado sea DRAFT, llama a `surveyRepository.deleteSurvey`, intenta llamar a `userService.removeSurveyCreated` (capturando excepciones y mostrando WARNING si falla), imprime LOG.
- `List<Survey> getSurveysByUser(String username)` - Requiere usuario logueado, verifica que username coincida con usuario logueado (lanza `SurveyException` si no), recorre todos los IDs, filtra por `CREATORUSERNAME`, si lista vacía lanza `SurveyException`, ordena por `CREATEDAT` descendente; devuelve lista.
- `Survey publishSurvey(String surveyId)` - Requiere usuario logueado y existencia, verifica que usuario sea creador, que estado sea DRAFT (si ya publicada lanza excepción), que tenga al menos una pregunta; cambia estado a `PUBLISHED`, establece `PUBLISHEDAT`, guarda en `surveyRepository`, imprime LOG; devuelve encuesta.

Métodos de gestión de preguntas:

- `void deleteQuestion(String surveyId, int questionIndex)` - Verifica existencia de encuesta y pregunta; llama a `surveyRepository.deleteQuestion(surveyId, questionIndex)`.
- `void reorderQuestion(String surveyId, int oldIndex, int newIndex)` - Verifica existencia de encuesta y ambos índices; llama a `surveyRepository.swapQuestions(surveyId, oldIndex, newIndex)`.

Métodos de contadores y rating:

- `void incrementResponseCount(String surveyId)` - Obtiene encuesta con `getSurvey`, obtiene vista actual (`survey.getViews()`), incrementa en 1, actualiza con `survey.setViews`, persiste con `surveyRepository.addSurvey`.
- `void addSurveyRating(String surveyId, double rating)` - Obtiene encuesta con `getSurvey`, lanza `SurveyException` si es null; si existe, llama a `survey.addRating(rating)` para actualizar `ratingCount/totalRating`, persiste con `surveyRepository.updateSurvey`.

UserService: Servicio que gestiona lógica de negocio de usuarios: registro, verificación de credenciales, gestión de encuestas creadas y respuestas, eliminación completa de cuentas.

Atributos:

- **userRepository:** UserRepository - privado, final - Almacena y gestiona usuarios.
- **surveyRepository:** SurveyRepository - privado, final - Accede y elimina encuestas creadas por usuarios.
- **responseRepository:** ResponseRepository - privado, final - Elimina respuestas asociadas a encuestas o usuarios.

Métodos:

- **UserService(UserRepository userRepository, SurveyRepository surveyRepository, ResponseRepository responseRepository)** - Constructor que inyecta repositorios necesarios.
- **private boolean isInputBlank(String text)** - Auxiliar que devuelve true si texto es null o vacío/solo espacios.
- **void registerUser(String username, String email, String password, String securityQuestion, String securityAnswer)** - Valida que todos campos no estén vacíos (usa isInputBlank), que email contenga @, que usuario no exista ya en userRepository; si todo correcto, calcula hash de contraseña con PasswordHasher.hash, crea User y lo guarda en userRepository; si falla, lanza RegisterException con mensajes específicos.
- **void verifyCredentials(String username, String password)** - Valida que usuario y contraseña no sean vacíos, comprueba que usuario exista en userRepository, obtiene User, valida que hash de contraseña introducida coincida con passwordHash guardado usando PasswordHasher.verify; si no coincide, lanza LoginException.
- **void addSurveyCreated(String username, String surveyId)** - Registra que usuario ha creado encuesta: obtiene User del repositorio, si no existe escribe WARNING por stderr y lanza RuntimeException; si existe, llama a userRepository.addSurveyId y registra LOG.
- **void removeSurveyCreated(String username, String surveyId)** - Elimina referencia a encuesta creada: obtiene User, si no existe lanza RuntimeException y escribe WARNING; si existe, llama a userRepository.deleteSurveyId y registra LOG.
- **void addResponseId(String username, String surveyId, String responseId)** - Registra que usuario ha respondido encuesta: obtiene User, crea entrada en respondedSurveysId si no existe, añade responseId a ese conjunto.
- **void loginUser(String username, String password)** - Método heredado no usado en lógica actual; devuelve internamente String ("usernotexists", "incorrectpassword", "success") según checks contra userRepository, lo imprime por consola.
- **String logoutUser(String username)** - Método antiguo que devuelve "usernotexists" o "success" según exista usuario; no gestiona realmente sesiones.
- **boolean deleteUserok(String username)** - Elimina completamente usuario si existe: obtiene User del repositorio, si no existe devuelve false; borra todas encuestas que ha creado (incluyendo sus respuestas mediante responseRepository.deleteResponsesBySurvey), borra todas respuestas emitidas por él según respondedSurveysId, finalmente lo elimina del userRepository; devuelve true si se eliminó, false si no existía.

domain/exception

LoginException: Excepción lanzada cuando ocurre un error en operaciones de inicio de sesión (login) - usuario no existe, contraseña incorrecta, etc.

Atributos:

- Hereda de RuntimeException

Métodos:

- LoginException(String message) - Constructor que acepta mensaje de error descriptivo.

RegisterException: Excepción lanzada cuando ocurre un error en operaciones de registro de usuarios - usuario duplicado, email inválido, campos vacíos, etc.

Atributos:

- Hereda de RuntimeException

Métodos:

- RegisterException(String message) - Constructor que acepta mensaje de error descriptivo.

ResponseException: Excepción lanzada cuando ocurre un error en operaciones con respuestas a encuestas - respuesta inexistente, validación fallida, estado incorrecto, etc.

Atributos:

- Hereda de RuntimeException

Métodos:

- ResponseException(String message) - Constructor que acepta mensaje de error descriptivo.

SurveyException: Excepción lanzada cuando ocurre un error en operaciones con encuestas - encuesta inexistente, validación fallida, permisos insuficientes, etc.

Atributos:

- Hereda de RuntimeException

Métodos:

- SurveyException(String message) - Constructor que acepta mensaje de error descriptivo.

domain/utls

PasswordHasher: Utilidad para hashear y verificar contraseñas de usuario de manera segura usando algoritmos criptográficos. Implementa hashing con salt para protección contra ataques.

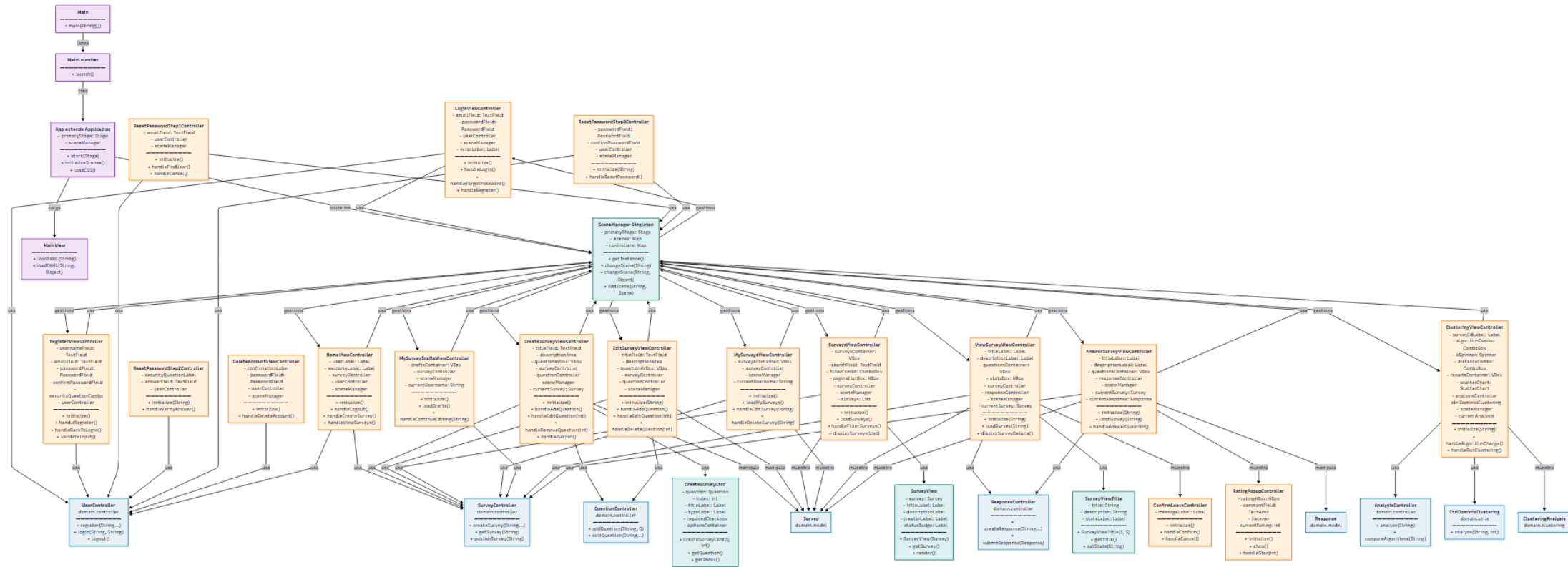
Métodos:

- **static String hash(String password)** - Hashea una contraseña en texto plano usando algoritmo seguro (BCrypt o PBKDF2).
- **static boolean verify(String plainPassword, String hashedPassword)** - Verifica si una contraseña en texto plano coincide con su hash almacenado: extrae el salt del hash, aplica el mismo algoritmo de hashing, compara resultado con hash almacenado, retorna true si coinciden (false en caso contrario).
- **static String generateSalt()** - Genera un salt aleatorio seguro (privado, usado internamente por hash).

- **static int hashIterations()** - Retorna número de iteraciones de hashing configuradas (privado, constante de configuración).

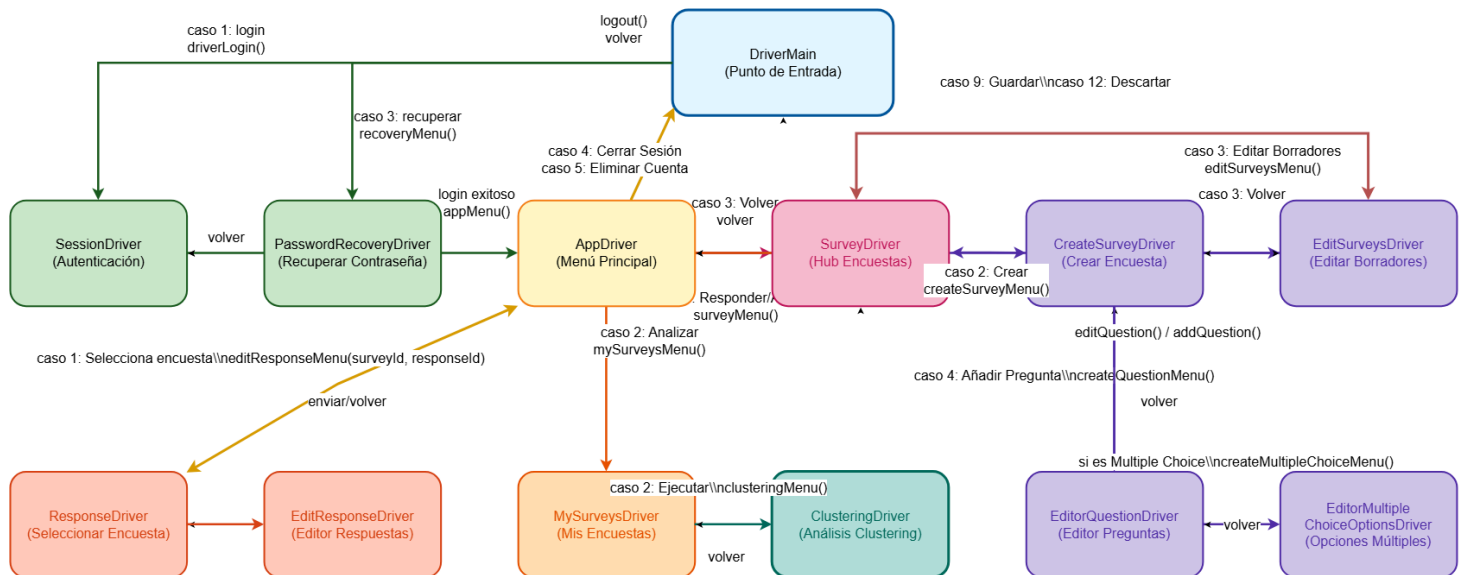
2.1.2. Capa de Presentación

2.1.2.1. Diagrama de navegación entre interfaces



2.1.2.2. Arquitectura de drivers actualizada y Flujos de navegación principales

ARQUITECTURA DE DRIVERS + NAVEGACIÓN JAVAFX - 2ª ENTREGA (ACTUALIZADO)



VISTAS JAVAFX (presentation.views + SceneManager)

1. LoginViewController	2. RegisterViewController	3. HomeViewController	4. SurveysViewController	5. CreateSurveyViewController	6. EditSurveyViewController
7. MySurveysViewController	8. MySurveyDraftsViewController	9. AnswerSurveyViewController	10. ViewSurveyViewController	11. ClusteringViewController	12. ResetPasswordStep1Controller
13. ResetPasswordStep2Controller	14. ResetPasswordStep3Controller	15. RatingPopupController			

CAMBIOS MÁS IMPORTANTES EN 2ª ENTREGA:

1. ClusteringDriver NUEVO: Driver para análisis de clustering. Opciones: ejecutar nuevo análisis, evaluar existente, encontrar K óptima, ver resultados, salir.
2. Arquitectura completamente reestructurada: De drivers línea comandos (1ª) a 15 Vistas JavaFX (2ª) + SceneManager que orquesta transiciones.
3. Patrón MVC con Dependency Injection: Controllers (UserController, SurveyController, etc.) → Services → Repositories → Datos JSON.
4. MySurveysDriver ahora con 3 casos: [1] Listar encuestas, [2] Ejecutar clustering (NUEVO), [3] Volver.
5. CtrlDominioClustering como fachada: Expone métodos runAnalysis(), exportarAnalysis(), obtenerMetricasCalidad(), encontrarKOptima(), getClusterVisualizationData().
6. EditSurveysDriver nuevo: Driver dedicado a editar encuestas en estado DRAFT. Evita conflictos con CreateSurveyDriver.

FLUJO PRINCIPAL:

DriverMain → selectWelcomeMenuOption() → [caso 1: SessionDriver] ↔ [AppDriver] → [caso 1: SurveyDriver | caso 2: CreateSurveyDriver | caso 3: EditSurveysDriver] → Drivers específicos

SessionDriver: caso 1 (login), caso 2 (register), caso 3 (recuperar contraseña) → PasswordRecoveryDriver
 ResponseDriver: caso 1 (selecciona encuesta) → EditResponseDriver, caso 2 (volver)
 MySurveysDriver: caso 1 (listar), caso 2 (ejecutar clustering) → ClusteringDriver [NUEVO], caso 3 (volver)
 CreateSurveyDriver: caso 4 (añadir pregunta) → EditorQuestionDriver → EditorMultipleChoiceOptionsDriver
 EditSurveysDriver: Análogo a CreateSurveyDriver pero para borradores existentes

2.1.2.3. Validaciones en capa de presentación

La aplicación cuenta con validaciones parcialmente implementadas distribuidas entre la capa de presentación y dominio. El principal problema es que son reactivas: el usuario solo recibe feedback al intentar enviar formularios, mediante popups disruptivos que interrumpen el flujo.

Situación por vistas:

- **LoginViewController:** Sin validación visual. El usuario puede escribir datos inválidos sin saberlo.
- **RegisterViewController:** Valida al enviar (email, contraseña débil, usuario duplicado). No hay indicadores de fortaleza de contraseña.
- **CreateSurveyViewController:** Validaciones mínimas. Sin feedback en tiempo real sobre título, descripción o preguntas incompletas.
- **AnswerSurveyViewController:** Valida respuestas antes de enviar, pero sin orientación visual mientras responde.

2.1.2.4. Mejoras en diseño de interfaz (ergonomía, usabilidad, estética)

Ergonomía

- Sistema de espaciado consistente: Normalizar márgenes y paddings a una escala base
- Jerarquía visual clara: Títulos en 28px bold, subtítulos en 18px, texto normal en 14px
- Navegación orientadora: Breadcrumbs mostrando dónde está el usuario

Estética

- Paleta de colores unificada: Usar variables CSS para azul primario, verde (éxito), rojo (error), naranja (advertencia)
- Componentes reutilizables: Crear campo de formulario genérico, indicadores de estado, tarjetas de información
- Contraste accesible: Asegurar que todos los textos cumplan mínimo de visibilidad

Responsividad

- Layouts flexibles: Que se adapten a ventanas redimensionadas
- Tamaño mínimo de controles: Botones y campos mínimo 40x40px (accesibilidad)
- Ancho mínimo y máximo: Aplicación funcional en 800px hasta ultrawide

Accesibilidad

- Navegación por teclado: Tab navega entre campos, Enter confirma, Escape cancela
- Focus visible: Bordes destacados en campos con focus
- Labels asociados: Cada campo vinculado a un label para screen readers

2.1.2.5. Descripción de todas las clases

presentation/driverMain

DriverMain: Clase principal de la capa de presentación encargada de inicializar toda la aplicación K-Survey. Se responsabiliza de:

- Crear e inyectar repositorios, servicios, controladores y drivers.
- Gestionar el menú inicial de bienvenida.
- Controlar el ciclo de vida de la aplicación y la interacción inicial del usuario.

Atributos:

- **sc: Scanner – privado, estático, final** : Scanner compartido por toda la aplicación para gestionar la entrada de datos por consola.

Métodos:

- **static Scanner getScanner()** -Devuelve el Scanner compartido para que pueda ser utilizado por otros drivers de la capa de presentación.
- **static int selectWelcomeMenuOption()** - Muestra el menú inicial de la aplicación con las opciones de iniciar sesión, registrarse, recuperar contraseña, salir. Lee la opción introducida por el usuario y la devuelve como entero.
- **public static void main(String[] args)** - Punto de entrada de la aplicación. Se encarga de; inicializar todos los repositorios (usuarios, encuestas, preguntas, respuestas y respuestas individuales). Crear los servicios de dominio inyectando los repositorios correspondientes. Crear los controladores que encapsulan la lógica de acceso al dominio. Inicializar los distintos drivers de presentación, responsables de la interacción con el usuario. Conectar los drivers entre sí según el flujo de la aplicación. Ejecutar el bucle principal de la aplicación mostrando el menú de bienvenida y gestionando las opciones seleccionadas. Finalizar la ejecución de forma controlada cuando el usuario selecciona la opción de salida o se produce una excepción de entrada.

presentation/drivers

AppDriver: Driver de la capa de presentación que gestiona el menú principal de la aplicación una vez el usuario ha iniciado sesión. Se encarga de coordinar la navegación entre las distintas funcionalidades del sistema y delegar la interacción al driver correspondiente.

Atributos:

- **sc: Scanner – privado, final** - Scanner compartido para la lectura de datos por consola en toda la aplicación.
- **surveyDriver: SurveyDriver – privado, final** - Gestiona las funcionalidades relacionadas con responder encuestas y analizar resultados.
- **createsurveyDriver: CreateSurveyDriver – privado, final** - Controla el flujo de creación de nuevas encuestas por parte del usuario.
- **editSurveysDriver: EditSurveysDriver – privado, final** - Permite modificar encuestas que se encuentran en estado borrador.
- **sessionDriver: SessionDriver – privado, final** - Gestiona la sesión del usuario, incluyendo cierre de sesión y eliminación de cuenta.

Métodos:

- `AppDriver(SurveyDriver surveyDriver, SessionDriver sessionDriver, CreateSurveyDriver createsurveyDriver, EditSurveysDriver editSurveysDriver)` - Constructor que inicializa el driver principal de la aplicación inyectando los drivers necesarios para la gestión del menú principal.
- `private int selectAppMenuOption()` - Muestra el menú principal de la aplicación con las opciones disponibles y devuelve la opción seleccionada por el usuario.
- `public void clearTerminal()` - Limpia la consola imprimiendo líneas en blanco para mejorar la legibilidad de la interfaz por consola.
- `public void appMenu()` - Ejecuta el bucle principal del menú de la aplicación, redirigiendo al usuario a la funcionalidad correspondiente según la opción seleccionada y finalizando al cerrar sesión o eliminar la cuenta.
- `public static void main(String[] args)` - Método auxiliar utilizado para realizar pruebas independientes del menú principal sin necesidad de ejecutar toda la aplicación.

ClusteringDriver: Driver de la capa de presentación que permite ejecutar, evaluar y visualizar análisis de clustering sobre encuestas desde la consola. Actúa como intermediario entre el usuario y el controlador de dominio de clustering.

Atributos:

- `ctrlDominioClustering: CtrlDominioClustering` – `privado, final` - Controlador de dominio encargado de ejecutar los algoritmos de clustering y gestionar los análisis asociados.
- `sc: Scanner` – `privado, final` - Scanner utilizado para la lectura de datos introducidos por el usuario por consola.

Métodos:

- `ClusteringDriver(CtrlDominioClustering ctrlDominioClustering)` - Constructor que inicializa el driver utilizando el `Scanner` compartido de la aplicación.
- `private ClusteringDriver(CtrlDominioClustering ctrlDominioClustering, Scanner scanner)` - Constructor auxiliar pensado para pruebas, que permite inyectar un `Scanner` específico.
- `public void run()` - Ejecuta el menú principal de clustering, permitiendo al usuario lanzar nuevos análisis, evaluar análisis existentes, buscar la K óptima o consultar resultados.
- `private void ejecutarNuevoAnalisis()` - Guía al usuario en el proceso de ejecución de un nuevo análisis de clustering, solicitando los parámetros necesarios y permitiendo la exportación automática de resultados.
- `private void evaluarAnalisisExistente()` - Permite seleccionar un análisis previamente ejecutado y mostrar sus métricas de calidad.
- `private void encontrarKOptima()` - Ejecuta un proceso automático para determinar el número óptimo de clusters (k) para una encuesta dada.
- `private void verResultadosAnalisis()` - Muestra los resultados detallados de un análisis de clustering almacenado en el sistema.
- `private void visualizarResultados(String analysisId)` - Muestra un submenú para consultar métricas de calidad de un análisis recién ejecutado o volver al menú principal.
- `private String selectAlgorithm()` - Permite seleccionar el algoritmo de clustering a utilizar y devuelve su identificador.
- `private String selectDistanceMetric()` - Permite seleccionar la métrica de distancia a utilizar en el análisis de clustering.

- **public static void main(String[] args)** - Método auxiliar para pruebas independientes del driver de clustering, permitiendo ejecutar análisis directamente desde consola.

CreateSurveyDriver: Driver de la capa de presentación encargado de guiar al usuario en la creación de una nueva encuesta desde consola. Permite definir título y descripción, gestionar las preguntas (añadir, editar, eliminar y reordenar) y finalmente guardar la encuesta como borrador o publicarla.

Atributos:

- **surveyController:** `SurveyController` – **privado, final** - Controlador de dominio utilizado para inicializar, guardar y publicar encuestas.
- **userController:** `UserController` – **privado, final** - Controlador de dominio utilizado para obtener información del usuario autenticado (por ejemplo, el creador de la encuesta).
- **editorQuestionDriver:** `EditorQuestionDriver` – **privado, final** - Driver auxiliar encargado de la creación y edición interactiva de preguntas.
- **sc:** `Scanner` – **privado, final** - Scanner compartido para leer entradas del usuario por consola.
- **currentSurvey:** `Survey` – **privado** - Encuesta que se está construyendo durante la ejecución del menú de creación.

Métodos:

- **CreateSurveyDriver(SurveyController surveyController, UserController userController, EditorQuestionDriver editorQuestionDriver)** - Constructor que inicializa el driver inyectando los controladores necesarios y el driver de edición de preguntas.
- **private int selectCreateSurveyMenuOption()** - Muestra el menú de creación de encuestas y devuelve la opción seleccionada por el usuario.
- **public void createSurveyMenu()** - Ejecuta el flujo completo de creación: inicializa una encuesta nueva y mantiene un bucle de opciones para modificarla hasta que se guarda como borrador o se publica.
- **private void initializeNewSurvey()** - Solicita título y descripción, obtiene el usuario creador logueado y crea una encuesta en memoria mediante el `SurveyController`.
- **private void viewSurveyId()** - Muestra por consola la información principal de la encuesta en edición (id si existe, título, descripción, creador, estado, número de preguntas y fecha).
- **private void editTitle()** - Permite modificar el título de la encuesta actual validando que no quede vacío.
- **private void editDescription()** - Permite modificar la descripción de la encuesta actual validando que no quede vacía.
- **private void addQuestion()** - Crea una nueva pregunta usando `EditorQuestionDriver` y la añade a la encuesta actual, asignando su posición según el índice.
- **private void editQuestion()** - Permite seleccionar una pregunta existente por índice y editarla a través de `EditorQuestionDriver`, actualizando la encuesta con el resultado.
- **private void deleteQuestion()** - Permite eliminar una pregunta por índice con confirmación previa, y reajusta los índices de las preguntas restantes.
- **private void reorderQuestions()** - Permite mover una pregunta de una posición a otra y actualiza los índices de las preguntas afectadas.

- **private void viewAllQuestions()** - Lista todas las preguntas de la encuesta mostrando texto, tipo, obligatoriedad y, si aplica, opciones y restricciones de selección.
- **private boolean saveAndPublishSurvey()** - Muestra las opciones finales de guardado (borrador, publicar o cancelar) y redirige al método correspondiente, devolviendo si se ha completado el proceso.
- **private boolean saveSurveyAsDraft()** - Guarda la encuesta como borrador tras confirmación del usuario, asignando un ID si el guardado es correcto.
- **private boolean publishNewSurvey()** - Guarda la encuesta y la publica tras confirmación del usuario, actualizando el estado y mostrando la información de publicación.
- **public static void main(String[] args)** - Método auxiliar para pruebas independientes del driver, simulando un login manual y ejecutando el menú de creación de encuestas.

EditorMultipleChoiceOptionsDriver: Driver de la capa de presentación encargado de crear y editar las opciones de las preguntas de tipo opción múltiple. Permite definir las opciones iniciales, modificar su contenido y orden, y ajustar las restricciones de selección asociadas a la pregunta.

Atributos:

- **sc: Scanner** – **privado, final** - Scanner compartido utilizado para leer la entrada del usuario por consola.
- **questionController: QuestionController** – **privado, final** - Controlador de dominio responsable de crear y modificar las opciones de las preguntas de opción múltiple.

Métodos:

- **EditorMultipleChoiceOptionsDriver(QuestionController questionController)** - Constructor que inicializa el driver inyectando el controlador de preguntas.
- **private int selectEditOptionsMenuOption()** - Muestra el menú de edición de opciones y devuelve la opción seleccionada por el usuario.
- **public MultipleChoiceQuestion createMultipleChoiceQuestion(int questionIndex, String surveyId, String questionText, boolean isRequired)** - Crea una pregunta de opción múltiple solicitando al usuario las opciones iniciales y las restricciones de selección mínima y máxima.
- **public void editOptionsMenu(MultipleChoiceQuestion question)** - Ejecuta el menú principal de edición de opciones, permitiendo añadir, editar, eliminar, reordenar y visualizar las opciones de la pregunta.
- **private void addOption(MultipleChoiceQuestion question)** - Añade una nueva opción a la pregunta de opción múltiple.
- **private void editOption(MultipleChoiceQuestion question)** - Permite modificar el texto de una opción existente seleccionada por índice.
- **private void deleteOption(MultipleChoiceQuestion question)** - Elimina una opción existente, garantizando que se mantenga el número mínimo de opciones requeridas.
- **private void reorderOptions(MultipleChoiceQuestion question)** - Cambia el orden de una opción dentro de la lista de opciones de la pregunta.
- **private void changeMinSelections(MultipleChoiceQuestion question)** - Actualiza el número mínimo de selecciones permitidas para la pregunta.
- **private void changeMaxSelections(MultipleChoiceQuestion question)** - Actualiza el número máximo de selecciones permitidas para la pregunta.

- **private void viewAllOptions(MultipleChoiceQuestion question)** - Muestra por consola todas las opciones de la pregunta junto con las restricciones de selección configuradas.
- **public static void main(String[] args)** - Método auxiliar para pruebas independientes del driver, permitiendo crear y editar preguntas de opción múltiple desde consola.

EditorQuestionDriver: Driver de la capa de presentación encargado de crear y editar preguntas de una encuesta. Permite gestionar preguntas textuales, numéricas y de opción múltiple, delegando la edición de opciones en un driver especializado cuando es necesario.

Atributos:

- **sc: Scanner – privado, final** - Scanner compartido utilizado para la lectura de datos por consola.
- **optionsDriver: EditorMultipleChoiceOptionsDriver – privado, final** - Driver auxiliar encargado de la creación y edición de las opciones de las preguntas de opción múltiple.
- **questionController: QuestionController – privado, final** - Controlador de dominio responsable de crear y modificar preguntas.

Métodos:

- **EditorQuestionDriver(QuestionController questionController)** - Constructor que inicializa el driver inyectando el controlador de preguntas y creando el driver de edición de opciones.
- **private int selectQuestionTypeOption()** - Muestra el menú de selección del tipo de pregunta y devuelve la opción elegida por el usuario.
- **private int selectEditTextualQuestionMenuOption()** - Muestra el menú de edición de preguntas textuales y devuelve la opción seleccionada.
- **private int selectEditNumericalQuestionMenuOption()** - Muestra el menú de edición de preguntas numéricas y devuelve la opción seleccionada.
- **private int selectEditMultipleChoiceQuestionMenuOption()** - Muestra el menú de edición de preguntas de opción múltiple y devuelve la opción seleccionada.
- **public Question createQuestion(String surveyId, int questionIndex)** - Crea una nueva pregunta solicitando el texto, el tipo y si es obligatoria, delegando la creación al controlador o a drivers auxiliares según el tipo seleccionado.
- **public Question editQuestion(Question originalQuestion)** - Edita una pregunta existente redirigiendo al flujo de edición correspondiente según su tipo.
- **private Question editTextualQuestion(Question question)** - Permite editar el texto y la obligatoriedad de una pregunta textual.
- **private Question editNumericalQuestion(Question question)** - Permite editar el texto y la obligatoriedad de una pregunta numérica.
- **private Question editMultipleChoiceQuestion(MultipleChoiceQuestion question)** - Permite editar el texto, las opciones y la obligatoriedad de una pregunta de opción múltiple, validando su consistencia antes de guardar.
- **private void toggleRequiredStatus(Question question)** - Cambia el estado de obligatoriedad de una pregunta previa confirmación del usuario.
- **private void editQuestionText(Question question)** - Modifica el texto de una pregunta existente.
- **private void displayTextualQuestionInfo(Question question)** - Muestra por consola la información básica de una pregunta textual.
- **private void displayNumericalQuestionInfo(Question question)** - Muestra por consola la información básica de una pregunta numérica.

- **private void displayMultipleChoiceQuestionInfo(MultipleChoiceQuestion question)** - Muestra por consola la información básica de una pregunta de opción múltiple.
- **public static void main(String[] args)** - Método auxiliar para pruebas independientes del driver, permitiendo crear y editar preguntas desde consola.

EditResponseDriver: Driver de la capa de presentación encargado de guiar al usuario en la edición y envío de sus respuestas a una encuesta. Permite visualizar las preguntas, responderlas según su tipo, revisar las respuestas introducidas y confirmar el envío final.

Atributos:

- **responseController: ResponseController** – **privado, final** - Controlador de dominio responsable de gestionar las respuestas del usuario y validar su contenido.
- **sc: Scanner** – **privado, final** - Scanner compartido utilizado para la lectura de datos por consola.

Métodos:

- **EditResponseDriver(ResponseController responseController)** - Constructor que inicializa el driver inyectando el controlador de respuestas.
- **public boolean confirmExit()** - Solicita confirmación al usuario antes de enviar la respuesta definitiva y devuelve si la respuesta debe enviarse o seguir editándose.
- **public boolean selectAnswerErrorMessage()** - Muestra un menú de recuperación tras un error al responder, permitiendo reintentar la respuesta o cambiar de pregunta.
- **private int selectEditResponseMenu()** - Muestra el menú principal de edición de respuestas y devuelve la opción seleccionada.
- **public void showQuestion(Question question)** - Muestra por consola una pregunta concreta, incluyendo sus opciones si es de tipo opción múltiple.
- **public void showQuestions(String surveyId)** - Muestra todas las preguntas asociadas a una encuesta determinada.
- **public void showAnswers(String surveyId, String responsed)** - Muestra todas las respuestas introducidas por el usuario para una encuesta, indicando el tipo de respuesta y su contenido.
- **public void answerQuestion(String surveyId, String responsed, int questionIndex)** - Gestiona la introducción de la respuesta a una pregunta concreta, adaptando la entrada al tipo de pregunta (textual, numérica u opción múltiple).
- **public void selectAnswer(String surveyId, String responsed)** - Permite seleccionar una pregunta por índice y responderla, gestionando errores de entrada y validación.
- **public void editResponseMenu(String surveyId, String responsed)** - Ejecuta el flujo completo de edición de respuestas, permitiendo ver preguntas, responderlas, revisar respuestas y confirmar el envío final.

EditSurveysDriver: Driver de la capa de presentación encargado de gestionar la edición de encuestas en estado BORRADOR (DRAFT). Permite listar las encuestas del usuario, seleccionar una encuesta editable y modificar su contenido de forma similar al proceso de creación, incluyendo preguntas, guardado, publicación o eliminación.

Atributos

- **surveyController:** `SurveyController` – **privado, final** - Controlador de dominio responsable de acceder, actualizar, publicar y eliminar encuestas.
- **UserController:** `UserController` – **privado, final** - Controlador de dominio utilizado para identificar al usuario autenticado y validar permisos.
- **editorQuestionDriver:** `EditorQuestionDriver` – **privado, final** - Driver auxiliar encargado de la creación y edición de preguntas dentro de la encuesta.
- **sc:** `Scanner` – **privado, final** - Scanner compartido utilizado para la entrada de datos por consola.
- **currentSurvey:** `Survey` – **privado** - Encuesta actualmente cargada en edición.

Métodos

- **EditSurveysDriver**(`SurveyController surveyController, UserController userController, EditorQuestionDriver editorQuestionDriver`) - Constructor que inicializa el driver inyectando los controladores y drivers necesarios.
- **private int selectEditSurveysMenuOption()** - Muestra el menú principal de edición de encuestas y devuelve la opción seleccionada.
- **public void editSurveysMenu()** - Ejecuta el menú principal que permite listar encuestas del usuario, editar una encuesta en borrador o salir.
- **private void listMySurveys()** - Muestra todas las encuestas del usuario que se encuentran en estado borrador, junto con su información básica.
- **private void editSurvey()** - Permite seleccionar una encuesta por ID, validando su existencia, autoría y estado antes de iniciar su edición.
- **private int selectEditSurveyMenuOption()** - Muestra el menú de edición de una encuesta concreta y devuelve la opción seleccionada.
- **private void editSurveyMenu()** - Ejecuta el flujo completo de edición de una encuesta en borrador, permitiendo modificar su contenido y estado.
- **private void viewSurveyInfo()** - Muestra la información general de la encuesta actualmente en edición.
- **private void editTitle()** - Permite modificar el título de la encuesta y guarda el cambio de forma inmediata.
- **private void editDescription()** - Permite modificar la descripción de la encuesta y persiste el cambio en el sistema.
- **private void addQuestion()** - Añade una nueva pregunta a la encuesta utilizando el `EditorQuestionDriver` y guarda el cambio.
- **private void editQuestion()** - Permite editar una pregunta existente seleccionándola por índice y actualizando la encuesta con los cambios.
- **private void deleteQuestion()** - Elimina una pregunta de la encuesta y actualiza su contenido en el sistema.
- **private void reorderQuestions()** - Permite reordenar las preguntas de la encuesta indicando el índice actual y el nuevo índice.
- **private void viewAllQuestions()** - Muestra todas las preguntas de la encuesta, incluyendo tipo, obligatoriedad y opciones en caso de preguntas de opción múltiple.
- **private boolean saveSurvey()** - Guarda los cambios realizados en la encuesta manteniéndola en estado borrador.
- **private boolean publishSurvey()** - Publica la encuesta tras confirmar que tiene al menos una pregunta y guarda los últimos cambios.
- **private boolean deleteSurvey()** - Elimina definitivamente la encuesta tras confirmación del usuario.

MySurveysDriver: Driver de la capa de presentación que permite al usuario consultar sus propias encuestas y acceder a funcionalidades avanzadas de análisis mediante algoritmos de clustering. Actúa como punto de enlace entre la visualización de encuestas y el módulo de análisis.

Atributos

- **surveyController:** `SurveyController` – `privado`, `final` - Controlador de dominio encargado de recuperar las encuestas asociadas al usuario autenticado.
- **clusteringDriver:** `ClusteringDriver` – `privado`, `final` - Driver responsable de ejecutar y gestionar los análisis de clustering sobre las encuestas.
- **sc:** `Scanner` – `privado`, `final` - Scanner compartido utilizado para la lectura de opciones introducidas por el usuario.

Métodos

- **MySurveysDriver(`SurveyController` `surveyController`, `ClusteringDriver` `clusteringDriver`)** - Constructor que inicializa el driver inyectando el controlador de encuestas y el driver de clustering.
- **public void `mySurveysMenu()`** - Ejecuta el menú principal de gestión de encuestas del usuario, permitiendo listar encuestas propias, ejecutar clustering o volver al menú anterior.
- **private void `listMySurveys()`** - Muestra por consola todas las encuestas creadas por el usuario autenticado, indicando su identificador y título.
- **private void `runClustering()`** - Inicia el flujo de ejecución de algoritmos de clustering a través del `ClusteringDriver`.
- **public static void `main(String[] args)`** - Método auxiliar para pruebas independientes del driver, permitiendo simular el acceso a las encuestas del usuario y la ejecución de análisis de clustering.

PasswordRecoveryDriver: Driver de la capa de presentación encargado de gestionar el proceso de recuperación de contraseña. Guía al usuario a través de la verificación de identidad mediante pregunta secreta y permite establecer una nueva contraseña.

Atributos

- **userController:** `UserController` – `privado`, `final` - Controlador de dominio responsable de validar al usuario y gestionar el proceso de recuperación de contraseña.
- **sc:** `Scanner` – `privado`, `final` - Scanner utilizado para la entrada de datos del usuario durante el proceso de recuperación.

Métodos

- **PasswordRecoveryDriver(`UserController` `userController`)** - Constructor que inicializa el driver inyectando el controlador de usuarios.
- **public void `recoveryMenu()`** - Ejecuta el flujo completo de recuperación de contraseña: solicita el nombre de usuario, muestra la pregunta secreta, valida la respuesta y permite establecer una nueva contraseña.

ResponseDriver: Driver de la capa de presentación encargado de gestionar el flujo de respuesta a encuestas. Permite listar las encuestas disponibles, seleccionar una encuesta para responderla y delegar la edición de respuestas al `EditResponseDriver`.

Atributos

- **surveyController:** `SurveyController` – `privado`, `final` - Controlador de dominio responsable de obtener las encuestas disponibles para responder.

- **responseController:** ResponseController – privado, final - Controlador de dominio encargado de iniciar y gestionar las respuestas de los usuarios.
- **editResponseDriver:** EditResponseDriver – privado, final - Driver auxiliar que permite editar y enviar las respuestas de una encuesta.
- **sc:** Scanner – privado, final - Scanner compartido utilizado para la lectura de datos por consola.

Métodos

- **ResponseDriver(SurveyController surveyController, ResponseController responseController, EditResponseDriver editResponseDriver)** - Constructor que inicializa el driver inyectando los controladores y el driver de edición de respuestas.
- **public int selectResponseMenuOption()** - Muestra el menú principal de respuesta a encuestas y devuelve la opción seleccionada por el usuario.
- **private int selectErrorMenuOption()** - Muestra un menú de recuperación de errores cuando ocurre un problema al seleccionar una encuesta.
- **public void showSurveys(List<Survey> surveys)** - Muestra por consola la información básica de las encuestas disponibles para responder.
- **public void selectSurvey()** - Permite seleccionar una encuesta por su identificador, inicia una nueva respuesta y delega el proceso de edición al **EditResponseDriver**.
- **public void responseMenu()** - Ejecuta el menú principal para responder encuestas, permitiendo acceder a la selección de encuestas o volver al menú anterior.
- **public static void main(String[] args)** - Método auxiliar para pruebas independientes del driver, permitiendo simular el flujo completo de respuesta a encuestas desde consola.

SessionDriver: Driver de la capa de presentación encargado de gestionar la **sesión del usuario**. Controla el registro, inicio de sesión, cierre de sesión, eliminación de cuenta y recuperación de contraseña, actuando como punto de entrada a la aplicación tras la autenticación.

Atributos

- **sc:** Scanner – privado, final - Scanner compartido utilizado para la entrada de datos por consola.
- **userController:** UserController – privado, final - Controlador de dominio responsable de la autenticación, registro y gestión de usuarios.
- **appDriver:** AppDriver – privado - Driver principal de la aplicación al que se redirige al usuario tras un inicio de sesión correcto.
- **passwordRecoveryDriver:** PasswordRecoveryDriver – privado, final - Driver encargado del proceso de recuperación de contraseña.

Métodos

- **SessionDriver(UserController userController, PasswordRecoveryDriver passwordRecoveryDriver)** - Constructor que inicializa el driver inyectando el controlador de usuarios y el driver de recuperación de contraseña.
- **public void setAppDriver(AppDriver appDriver)** - Asigna el **AppDriver** para permitir el acceso al menú principal tras el inicio de sesión.
- **private String displayRegisterErrorMenu()** - Muestra el menú de error durante el registro y devuelve la opción seleccionada por el usuario.
- **private String displayLoginErrorMenu()** - Muestra el menú de error durante el inicio de sesión y devuelve la opción seleccionada.

- **public void clearTerminal()** - Limpia la consola imprimiendo múltiples líneas en blanco para mejorar la legibilidad.
- **public void driverRegister()** - Gestiona el proceso de registro de un nuevo usuario, incluyendo validación de contraseña, pregunta de seguridad y control de errores.
- **public void driverLogin()** - Gestiona el inicio de sesión del usuario, redirigiéndolo al menú principal en caso de éxito o mostrando opciones alternativas en caso de error.
- **public void logout()** - Cierra la sesión del usuario autenticado y muestra un mensaje de despedida.
- **public void driverDeleteAccount()** - Permite al usuario eliminar su cuenta previa confirmación, cerrando la sesión tras la eliminación.

SurveyDriver: Driver de la capa de presentación encargado de gestionar el menú general de encuestas. Actúa como punto de acceso a las funcionalidades de **responder encuestas** y **analizar las encuestas propias**, delegando cada flujo al driver correspondiente.

Atributos

- **sc: Scanner** – **privado, final** - Scanner compartido utilizado para la lectura de opciones por consola.
- **responseDriver: ResponseDriver** – **privado, final** - Driver responsable del flujo de respuesta a encuestas disponibles.
- **mySurveysDriver: MySurveysDriver** – **privado, final** - Driver encargado de la gestión y análisis de las encuestas creadas por el usuario.

Métodos

- **SurveyDriver(ResponseDriver responseDriver, MySurveysDriver mySurveysDriver)** - Constructor que inicializa el driver inyectando los drivers de respuesta y análisis de encuestas.
- **public int selectSurveyMenuOption()** - Muestra el menú principal de encuestas y devuelve la opción seleccionada por el usuario.
- **public void surveyMenu()** - Ejecuta el menú de encuestas, permitiendo al usuario responder encuestas, analizar sus propias encuestas o volver al menú principal.
- **public static void main(String[] args)** - Método auxiliar para pruebas independientes del driver, permitiendo probar el acceso al menú de encuestas y sus funcionalidades asociadas.

presentation/main

Main: Clase principal de la capa de presentación gráfica encargada de inicializar la aplicación JavaFX. Se responsabiliza de crear e inyectar los componentes principales (repositorios, servicios y controladores) y de arrancar la interfaz gráfica a través del gestor de escenas.

Métodos

- **private void iniStage(Stage primaryStage)** - Método auxiliar reservado para la inicialización del **Stage**. Actualmente no implementado, queda preparado para futuras extensiones.
- **public void start(Stage primaryStage)** - Método sobrescrito de **Application** que actúa como punto de inicio de la aplicación JavaFX.
Se encarga de:

- Crear los repositorios para la gestión de datos.
- Inicializar los servicios que contienen la lógica de negocio.
- Crear los controladores de dominio que conectan la lógica con la interfaz.
- Inicializar el SceneManager, responsable de gestionar la navegación entre vistas.
- Configurar el Stage principal y mostrar la pantalla inicial de inicio de sesión.
- `public static void main(String[] args)` - Punto de entrada de la aplicación JavaFX. Lanza la ejecución llamando al método `launch`.

MainLauncher: Clase auxiliar de la capa de presentación encargada de lanzar la aplicación gráfica. Actúa como punto de entrada alternativo que delega la ejecución en la clase `Main`.

Métodos

- `public static void main(String[] args)` - Método de entrada que llama al método `main` de la clase `Main`, permitiendo el arranque correcto de la aplicación JavaFX desde distintos entornos de ejecución.

MainView: Clase de arranque de la capa de presentación gráfica que actúa como equivalente gráfico del antiguo `DriverMain`. Se encarga de inicializar todos los componentes principales de la aplicación (repositorios, servicios y controladores) y lanzar la interfaz JavaFX mediante la clase `App`.

Métodos

- `public static void main(String[] args)` - Punto de entrada de la aplicación gráfica. Se responsabiliza de:
 - Inicializar los repositorios de datos (usuarios, encuestas, preguntas, respuestas y respuestas individuales).
 - Crear los servicios de dominio que encapsulan la lógica de negocio.
 - Inicializar los controladores que conectan la lógica con la capa de presentación.
 - Crear el SceneManager, encargado de la gestión y cambio de escenas JavaFX.
 - Registrar el `SceneManager` como global en la clase `App`.
 - Lanzar la aplicación JavaFX mediante `Application.launch`.

presentation/util

CreateSurveyCard: Clase de utilidad de la capa de presentación gráfica encargada de construir y devolver una card visual de una encuesta utilizando JavaFX. Centraliza la creación de la estructura y el estilo de las tarjetas mostradas en las distintas vistas de la aplicación.

Métodos

- `public static HBox getSurveyCard(SceneManager sceneManager, Survey s, SurveyView surveyView)` - Construye y devuelve la card completa de una encuesta como un `HBox`. Configura el evento de clic según la vista desde la que se muestra (home, mis encuestas o borradores) y organiza la información en columnas (título/autor, vistas, valoración y fecha).

- **private static VBox createMainColumn(Survey s)** - Crea la columna principal de la card que muestra el título de la encuesta y el nombre del creador.
- **private static HBox createViewsColumn(Survey s)** - Crea la columna que muestra el número de visualizaciones de la encuesta junto con un icono representativo.
- **private static HBox createRatingColumn(Survey s)** - Crea la columna que muestra la valoración media de la encuesta mediante estrellas y valor numérico.
- **private static HBox createDateColumn(Survey s)** - Crea la columna que muestra la fecha asociada a la encuesta, utilizando la fecha de publicación si existe o la de creación en caso contrario.
- **private static void addColumnsWithDividers(HBox card, VBox colMain, HBox... columns)** - Añade la columna principal y el resto de columnas al contenedor principal, separándolas mediante divisores verticales.
- **private static Region createVerticalDivider()** - Crea un separador vertical estilizado utilizado para dividir visualmente las columnas de la card.
- **private static HBox createDataCell(String text, String svgPath)** - Construye una celda genérica que combina un icono SVG y un texto, utilizada para vistas, fechas y otros datos.
- **private static HBox createRatingCell(double rating)** - Crea la celda que muestra la valoración media de la encuesta, combinando estrellas gráficas y el valor numérico.
- **private static HBox createStarsBox(double rating)** - Genera el conjunto de estrellas SVG que representan la valoración, marcando estrellas completas o medias según el valor proporcionado.

SurveyView: Enumerado de la capa de presentación que define los distintos contextos de visualización de una encuesta dentro de la interfaz gráfica. Se utiliza para adaptar el comportamiento y la navegación asociada a las cards de encuestas según la vista activa.

Valores

- **HOME** - Vista principal de la aplicación, donde se muestran encuestas disponibles para ser respondidas.
- **DRAFTS** - Vista de encuestas en estado borrador, orientada a su edición.
- **MY_SURVEYS** - Vista de las encuestas creadas por el usuario, destinada a su consulta y análisis.

SurveyViewTitle: Enumerado de la capa de presentación que asocia cada tipo de vista de encuestas con el título textual que debe mostrarse en la interfaz gráfica. Permite centralizar los textos de cabecera y mantener coherencia visual entre las distintas vistas.

Atributos

- **title: String** – **privado, final** - Texto que representa el título de la vista correspondiente.

Valores

- **HOME** - Representa la vista principal de respuesta a encuestas. Título asociado: *"RESPONDE A LAS ENCUESTAS"*.
- **DRAFTS** - Representa la vista de edición de encuestas en borrador. Título asociado: *"EDITA TUS ENCUESTAS"*.

- **MY_SURVEYS** - Representa la vista de análisis de encuestas propias. Título asociado: *"ANALIZA TUS ENCUESTAS"*.

Métodos

- **SurveyViewTitle(String title)** - Constructor que asigna el título asociado a cada valor del enumerado.
- **public String getTitle()** - Devuelve el título textual correspondiente a la vista.

presentation/views

AnswerSurveyViewController: Controlador JavaFX responsable de la **visualización y gestión del proceso de respuesta a una encuesta**. Se encarga de renderizar dinámicamente las preguntas según su tipo, recoger las respuestas del usuario, validarlas y enviarlas al dominio, así como de gestionar la navegación posterior (valoración y retorno a la vista principal).

Atributos

- **surveyTitle:** **Label** - Muestra el título de la encuesta que se está respondiendo.
- **questionsBox:** **VBox** - Contenedor principal donde se renderizan dinámicamente las preguntas.
- **usernameLabel:** **Label** - Muestra el nombre del usuario actualmente autenticado.
- **avatarLabel:** **Label** - Muestra la inicial del usuario como avatar textual.
- **survey:** **Survey** - Encuesta actualmente cargada para ser respondida.
- **responseController:** **ResponseController** - Controlador de dominio encargado de crear y actualizar respuestas.
- **userController:** **UserController** - Controlador de dominio que gestiona la sesión del usuario.
- **surveyController:** **SurveyController** - Controlador de dominio utilizado para actualizar métricas de la encuesta (respuestas, valoración).
- **sceneManager:** **SceneManager** - Gestor de escenas responsable de la navegación entre vistas.
- **answersMap:** **Map<Integer, Object>** - Estructura auxiliar que almacena las respuestas introducidas por el usuario, asociando el índice de la pregunta con su valor.

Métodos

- **AnswerSurveyViewController(...)** - Constructor que inyecta los controladores de dominio y el gestor de escenas necesarios para el funcionamiento de la vista.
- **public void loadSurvey(Survey survey)** - Carga la encuesta a responder, inicializa los datos del usuario en la cabecera y lanza el renderizado dinámico de las preguntas.
- **private void renderQuestions()** - Genera dinámicamente la interfaz de cada pregunta según su tipo:
 - Textual → **TextArea**
 - Numérica → **TextField**
 - Opción múltiple → conjunto de **CheckBox**Registra listeners para almacenar las respuestas en **answersMap**.

- **private void handleSubmit(ActionEvent event)** - Gestiona el envío de la encuesta:
 - Crea una nueva respuesta.
 - Valida preguntas obligatorias.
 - Envía cada respuesta al dominio según su tipo.
 - Incrementa el contador de respuestas.
 - Muestra el popup de valoración y guarda el rating.
 - Redirige al usuario a la vista principal.
- **private void showError(String message)** - Muestra un cuadro de diálogo de error cuando ocurre un problema durante el envío de la encuesta.
- **public void goHome(ActionEvent e)** - Navega a la vista principal previa confirmación de salida.
- **private void goToMySurveys(ActionEvent e)** - Navega a la vista de encuestas propias, solicitando confirmación.
- **private void goToMyDrafts(ActionEvent e)** - Navega a la vista de encuestas en borrador, solicitando confirmación.
- **private void goCreateSurvey(ActionEvent e)** - Navega a la vista de creación de encuestas, solicitando confirmación.
- **public void handleLogout(ActionEvent e)** - Cierra la sesión del usuario y redirige a la vista de login, previa confirmación.

App: Clase principal de arranque de la **interfaz gráfica JavaFX**. Actúa como punto de entrada de la aplicación visual y delega toda la gestión de escenas y navegación al **SceneManager**, manteniendo la clase ligera y desacoplada de la lógica de presentación concreta.

Atributos

- **globalSceneManager:** **SceneManager** (*static*) - Referencia global al gestor de escenas de la aplicación. Se inyecta antes de lanzar JavaFX y permite que **App** inicialice correctamente la interfaz.

Métodos

- **public static void setGlobalSceneManager(SceneManager manager)** - Asigna la instancia de **SceneManager** que será utilizada por la aplicación al iniciarse. Este método debe ejecutarse **antes** de lanzar JavaFX.
- **public void start(Stage stage)** - Método sobrescrito de **Application** que inicializa la aplicación gráfica:
 - Verifica que el **SceneManager** haya sido correctamente inyectado.
 - Asocia el **Stage** principal al gestor de escenas.
 - Inicializa la configuración general del **Stage**.
 - Carga la primera vista de la aplicación (registro).
- **public static void main(String[] args)** - Punto de entrada estándar de JavaFX que lanza la aplicación.

ClusteringViewController: Controlador JavaFX responsable de la **ejecución, visualización y exportación de análisis de clustering** sobre encuestas. Actúa como puente entre la interfaz gráfica avanzada (gráficas, métricas, clusters) y la lógica de dominio encapsulada en **CtrlDominioClustering**.

Atributos

Dependencias

- **UserController:** `UserController` - Proporciona información del usuario autenticado y gestiona sesión.
- **surveyController:** `SurveyController` - Permite actualizar información de la encuesta (ratings, navegación).
- **clusteringController:** `CtrlDominioClustering` - Núcleo del análisis de clustering y proveedor de métricas, resultados y datos visuales.
- **sceneManager:** `SceneManager` - Gestiona la navegación entre vistas.
- **surveyId:** `String` - Identificador de la encuesta que se está analizando.

Estado interno

- **currentAnalysisId:** `String` - ID del último análisis ejecutado.
- **allQuestions:** `List<Map.Entry<Integer, String>>` - Lista de preguntas disponibles para selección de dimensiones.
- **currentScatterSeriesMap:** `Map<String, XYChart.Series<Number, Number>>` - Series activas del ScatterChart agrupadas por cluster.

Constantes

- **CLUSTER_COLORS:** `String[]` - Paleta de colores consistente para representar clusters en gráficas.

Métodos principales

- **initialize(URL, ResourceBundle)** - Inicializa la vista; configura usuario y avatar. Inicializa combos de algoritmo y distancias. Configura ejes de gráficas. Prepara selectores de dimensiones.
- **runAnalysis(ActionEvent)** - Lanza el análisis de clustering de forma **asíncrona**: Valida parámetros introducidos. Ejecuta el clustering en un **Task**. Muestra indicador de carga. Procesa resultados al finalizar.
- **displayResults(String analysisId)** - Procesa y muestra los resultados: Métricas globales (Silhouette, Davies-Bouldin, Calinski-Harabasz). Cards detalladas por cluster. Gráfica de tamaño de clusters. ScatterChart con puntos y centroides. Aplicación de estilos y colores.
- **calculateOptimalK(ActionEvent)** - Calcula automáticamente un valor óptimo de *K* dentro de un rango razonable.
- **updateScatterChart()** - Actualiza la gráfica de dispersión cuando se cambian las dimensiones X/Y: Recalcula coordenadas. Reconstruye series. Mantiene aspecto 1:1 de los ejes.

Exportación

- **exportCSV(ActionEvent)** - Exporta los resultados completos del clustering a CSV.
- **exportBarChartPNG(ActionEvent)** - Exporta la gráfica de tamaños de clusters como imagen PNG.
- **exportScatterChartPNG(ActionEvent)** - Exporta la gráfica de dispersión como PNG.

Utilidades internas

- **configureScatterAxisRanges()** - Ajusta rangos de ejes para mantener proporciones correctas.

- **formatCentroid(String)** - Convierte centroides complejos en una representación legible.
- **createMetricBox(String, String)** - Construye bloques visuales compactos para métricas.

Navegación

Métodos FXML para sidebar:

- **goToHome**
- **goToMySurveys**
- **goToMyDrafts**
- **goToCreateSurvey**
- **handleLogout**
- **handleDeleteAccount**

ConfirmLeaveController: ConfirmLeaveController es el controlador JavaFX encargado de gestionar el cuadro de diálogo que se muestra cuando el usuario intenta abandonar una vista sin haber confirmado previamente la acción. Su función principal es permitir al usuario decidir si desea continuar con la salida o cancelar la acción y permanecer en la vista actual, evitando así pérdidas accidentales de información.

Atributos

- **onConfirm (Runnable)** - Acción que se ejecuta cuando el usuario confirma que desea abandonar la vista. Este atributo permite desacoplar el diálogo de confirmación de la lógica concreta de navegación o acción a realizar, haciendo el controlador reutilizable en distintos contextos.

Métodos

- **setOnConfirm(Runnable onConfirm)** - Permite definir la acción que se ejecutará si el usuario acepta abandonar la vista. Esta acción suele estar relacionada con la navegación a otra pantalla o el cierre de una sesión.
- **handleCancel(ActionEvent event)** - Maneja la acción del botón de cancelar. Cierra el diálogo de confirmación sin ejecutar ninguna acción adicional, manteniendo al usuario en la vista actual.
- **handleLeave(ActionEvent event)** - Maneja la acción del botón de confirmar salida. Ejecuta la acción almacenada en **onConfirm** (si existe) y cierra el diálogo, permitiendo continuar con la operación solicitada por el usuario.

CreateSurveyViewController: CreateSurveyViewController es el controlador JavaFX responsable de la vista de creación y edición de encuestas. Gestiona todo el flujo necesario para que un usuario pueda definir una encuesta completa de forma interactiva: introducir título y descripción, añadir y configurar preguntas de distintos tipos, reordenarlas mediante drag & drop, guardarlas como borrador o publicarlas.

El controlador actúa como intermediario entre la interfaz gráfica y la capa de dominio, delegando en **SurveyController** las operaciones de persistencia y publicación, y en **SceneManager** la navegación entre vistas.

Atributos

- **UserController** (`UserController`) - Controlador de usuarios que permite verificar el estado de sesión, obtener el usuario autenticado y gestionar el cierre de sesión.
- **surveyController** (`SurveyController`) - Controlador encargado de la lógica de negocio relacionada con la creación, actualización y publicación de encuestas.
- **sceneManager** (`SceneManager`) - Gestor de escenas utilizado para navegar entre las distintas vistas de la aplicación.
- **avatarLabel**, **usernameLabel** (`Label`) - Elementos visuales de la barra lateral que muestran la información del usuario autenticado.
- **home**, **createSurvey**, **mySurveys**, **myDrafts** (`Button`) - Botones de navegación del sidebar que permiten desplazarse entre las diferentes secciones de la aplicación.
- **surveyTitleField** (`TextField`) - Campo de texto donde el usuario introduce el título de la encuesta.
- **surveyDescField** (`TextArea`) - Campo de texto destinado a la descripción de la encuesta.
- **questionsContainer** (`VBox`) - Contenedor principal donde se renderizan dinámicamente las tarjetas de preguntas creadas por el usuario.
- **scrollPane** (`ScrollPane`) - Permite el desplazamiento vertical cuando la encuesta contiene un número elevado de preguntas.
- **questionList** (`List<Question>`) - Lista que almacena el modelo de datos de todas las preguntas que forman la encuesta en construcción.
- **currentSurveyId** (`String`) - Identificador de la encuesta actual. Si es `null`, indica que se está creando una encuesta nueva; en caso contrario, se está editando un borrador existente.
- **draggingIndex** (`int`) - Índice temporal utilizado para gestionar el reordenamiento de preguntas mediante drag & drop.

Métodos

- **initialize(URL, ResourceBundle)** - Inicializa la vista comprobando que el usuario esté autenticado, configurando la barra lateral, preparando el `ScrollPane` y creando una pregunta inicial por defecto.
- **addDefaultQuestion()** - Añade automáticamente una pregunta textual inicial para facilitar la creación de la encuesta.
- **addQuestion(TypeQuestion type)** - Crea una nueva pregunta del tipo indicado (textual, numérica u opción múltiple), la añade a la lista de preguntas y actualiza la vista.
- **handleSaveDraft()** - Valida los datos introducidos y guarda la encuesta como borrador, creando una nueva encuesta o actualizando una existente según corresponda.
- **handlePublish()** - Valida la encuesta, solicita confirmación al usuario y publica la encuesta para que pueda recibir respuestas.
- **handleCancel()** - Permite cancelar la creación o edición de la encuesta, mostrando un aviso si existen cambios no guardados.
- **buildSurvey()** - Construye un objeto `Survey` a partir de los datos introducidos en la vista, incluyendo título, descripción y preguntas.
- **validateQuestions()** - Verifica que todas las preguntas estén correctamente definidas, incluyendo textos no vacíos, número mínimo de opciones y coherencia entre selecciones mínimas y máximas.
- **renderQuestions()** - Renderiza dinámicamente todas las preguntas en el contenedor principal, asegurando que los índices estén correctamente actualizados.

- **createQuestionCard(Question, int)** - Genera la tarjeta visual de una pregunta, incluyendo su cabecera, contenido, opciones de configuración y controles de eliminación.
- **renderMultipleChoiceContent(MultipleChoiceQuestion, VBox)** - Renderiza el contenido específico de las preguntas de opción múltiple, permitiendo añadir, eliminar y configurar opciones.
- **changeQuestionType(int, String)** - Cambia el tipo de una pregunta existente, preservando el texto y la obligatoriedad cuando es posible.
- **setupDragAndDrop(Node, int)** - Configura los eventos necesarios para permitir el reordenamiento de preguntas mediante drag & drop.
- **goToHome(), goToCreateSurvey(), goToMySurveys(), goToMyDrafts()** - Métodos de navegación asociados a la barra lateral.
- **handleLogout(ActionEvent)** - Cierra la sesión del usuario y redirige a la pantalla de inicio de sesión.

DeleteAccountViewController: DeleteAccountViewController es el controlador JavaFX encargado de gestionar la vista de confirmación para la eliminación de la cuenta de usuario. Su objetivo principal es ofrecer al usuario una última oportunidad para confirmar o cancelar la eliminación definitiva de su cuenta, garantizando que esta acción crítica se realice de forma consciente. El controlador coordina la operación con la capa de dominio y gestiona la navegación posterior según el resultado de la acción.

Atributos

- **userController (UserController)** - Controlador responsable de la gestión de usuarios, utilizado para ejecutar la eliminación de la cuenta del usuario actualmente autenticado.
- **sceneManager (SceneManager)** - Gestor de escenas encargado de controlar la navegación entre vistas, permitiendo redirigir al usuario a la pantalla de inicio de sesión o a la vista principal según la opción seleccionada.

Métodos

- **confirmDelete(ActionEvent event)** - Maneja la acción de confirmación de eliminación de la cuenta. Invoca la operación de borrado en el **UserController** y, si esta se completa correctamente, cierra la ventana de confirmación y redirige al usuario a la pantalla de inicio de sesión.
- **handleCancel(ActionEvent event)** - Maneja la cancelación de la operación. Cierra la ventana de confirmación sin realizar ningún cambio en la cuenta y devuelve al usuario a la vista principal de la aplicación.

EditSurveyViewController: EditSurveyViewController es el controlador JavaFX responsable de la edición de encuestas que se encuentran en estado borrador. Permite al usuario modificar el título, la descripción y las preguntas de una encuesta existente, así como reordenarlas, cambiar su tipo, ajustar configuraciones específicas y finalmente guardar los cambios, publicar la encuesta o eliminarla. El controlador garantiza que únicamente el creador de la encuesta pueda editarla y que solo las encuestas en estado *DRAFT* sean modificables.

Atributos

- **UserController** (**UserController**) - Controlador encargado de la gestión del usuario autenticado. Se utiliza para verificar el inicio de sesión, obtener el nombre del usuario y gestionar el cierre de sesión.
- **surveyController** (**SurveyController**) - Controlador de dominio responsable de cargar, actualizar, publicar y eliminar encuestas.
- **sceneManager** (**SceneManager**) - Gestor de escenas que permite la navegación entre las distintas vistas de la aplicación.
- **surveyId** (**String**) - Identificador de la encuesta que se está editando.
- **currentSurvey** (**Survey**) - Encuesta actualmente cargada desde el sistema y sobre la cual se aplican las modificaciones.
- **questionList** (**List<Question>**) - Lista de preguntas que actúa como modelo temporal durante la edición, permitiendo añadir, eliminar, modificar y reordenar preguntas antes de guardar los cambios.
- **draggingIndex** (**int**) - Índice auxiliar utilizado para gestionar la funcionalidad de *drag & drop* al reordenar preguntas dentro de la encuesta.
- **Campos FXML** (**Label, Button, TextField, TextArea, VBox, ScrollPane**) - Componentes gráficos de la interfaz que permiten mostrar la información de la encuesta, editar sus campos y gestionar la navegación lateral.

Métodos

- **initialize(URL, ResourceBundle)** - Inicializa la vista comprobando que el usuario esté autenticado, configurando la barra lateral, ajustando el **ScrollPane** y cargando la encuesta existente a partir de su identificador.
- **loadSurvey()** - Carga la encuesta desde el controlador de dominio, verifica que pertenece al usuario actual y que se encuentra en estado borrador, y rellena los campos del formulario con sus datos.
- **addDefaultQuestion()** - Añade una nueva pregunta de tipo textual al final de la encuesta.
- **addQuestion(TypeQuestion)** - Crea y añade una nueva pregunta del tipo indicado (textual, numérica u opción múltiple), inicializando las opciones necesarias cuando corresponde.
- **handleSaveChanges()** - Valida los datos introducidos y guarda los cambios realizados en la encuesta sin publicarla, manteniéndola en estado borrador.
- **handlePublish()** - Valida la encuesta y, tras confirmación del usuario, guarda los cambios y publica la encuesta, dejándola disponible para recibir respuestas.
- **handleDeleteSurvey()** - Solicita confirmación y elimina de forma permanente la encuesta y todas sus preguntas.
- **handleCancel()** - Cancela la edición actual y regresa a la vista de borradores, solicitando confirmación si existen cambios no guardados.
- **updateSurveyFromFields()** - Actualiza el objeto **Survey** con los valores actuales del formulario y la lista de preguntas editadas.
- **validateQuestions()** - Comprueba que todas las preguntas tengan texto válido, que las preguntas de opción múltiple tengan al menos dos opciones y que las configuraciones de mínimo y máximo de selecciones sean coherentes.
- **renderQuestions()** - Renderiza dinámicamente todas las preguntas de la encuesta en la interfaz gráfica, manteniendo el orden actual.
- **createQuestionCard(Question, int)** - Construye visualmente una tarjeta editable para una pregunta concreta, incluyendo su cabecera, contenido y opciones de configuración.
- **renderMultipleChoiceContent(MultipleChoiceQuestion, VBox)** - Genera la interfaz específica para las preguntas de opción múltiple, permitiendo editar opciones, añadir o eliminar opciones y configurar el número mínimo y máximo de selecciones.

- **changeQuestionType(int, String)** - Cambia el tipo de una pregunta existente, conservando su texto y configuración básica cuando es posible.
- **setupDragAndDrop(Node, int)** - Configura la funcionalidad de arrastrar y soltar para permitir la reordenación visual de las preguntas dentro de la encuesta.

HomeController: HomeController es el controlador de la vista principal (*Home*) de la aplicación. Su responsabilidad es mostrar al usuario las encuestas públicas disponibles para responder, aplicando distintos filtros y criterios de búsqueda. Extiende la clase abstracta **SurveyViewController**, reutilizando su lógica común para la visualización de encuestas, y especializándola para el contexto del inicio de la aplicación. Además, gestiona la activación visual del botón de navegación correspondiente y el título central de la vista.

Atributos

- **home (Button)** - Botón de la barra lateral que representa la vista principal. Se marca como activo cuando el usuario se encuentra en la pantalla de inicio.
- **centerContentTitle (Label)** - Etiqueta central que muestra el título descriptivo de la vista, indicando al usuario que puede responder encuestas.
- **TITLE (SurveyViewTitle)** - Constante que define el título asociado a la vista Home, utilizado para mantener coherencia visual y semántica entre las distintas vistas de encuestas.

Métodos

- **HomeController(UserController, SurveyController, SurveyView, SceneManager)** - Constructor que inyecta los controladores de usuario y encuestas, el tipo de vista y el gestor de escenas. Inicializa el controlador base (**SurveyViewController**) y asigna el título correspondiente a la vista Home.
- **setViewActive()** - Marca el botón *Home* como activo en la barra lateral, aplicando el estilo visual que indica la vista actualmente seleccionada.
- **setContentTitle()** - Actualiza el título central de la vista utilizando el valor definido en **SurveyViewTitle.HOME**.
- **getFilter(String filter, String searchText)** - Devuelve un predicado que filtra las encuestas mostradas en función de distintos criterios:
excluye las encuestas en estado borrador, aplica la búsqueda por texto sobre el título y permite filtrar por valoración media o número de visualizaciones.
- **getSelectedSurveys()** - Obtiene desde el controlador de dominio la lista de encuestas disponibles para ser respondidas, que serán posteriormente filtradas y mostradas en la vista.

LoginViewController: LoginViewController es el controlador encargado de gestionar la vista de inicio de sesión de la aplicación. Su función principal es permitir que un usuario introduzca sus credenciales, validar el acceso mediante la capa de dominio y redirigirlo a la vista principal en caso de autenticación correcta. Además, gestiona la visualización de errores de inicio de sesión y ofrece funcionalidades auxiliares como mostrar u ocultar la contraseña y la navegación hacia el registro o la recuperación de contraseña.

Atributos

- **usernameField (TextField)** - Campo de texto donde el usuario introduce su nombre de usuario.

- **passwordField** (**PasswordField**) - Campo de texto protegido utilizado para introducir la contraseña de forma oculta.
- **passwordText** (**TextField**) - Campo alternativo que permite mostrar la contraseña en texto plano cuando el usuario activa la opción de visualización.
- **loginButton** (**Button**) - Botón que desencadena el proceso de inicio de sesión.
- **errorCard** (**HBox**) - Contenedor visual que se muestra cuando ocurre un error durante el inicio de sesión.
- **errorLabel** (**Label**) - Etiqueta donde se muestra el mensaje de error devuelto por la capa de dominio.
- **userController** (**UserController**) - Controlador de la capa de dominio responsable de la autenticación y gestión de la sesión del usuario.
- **sceneManager** (**SceneManager**) - Gestor de escenas utilizado para navegar entre la vista de login, el registro, la recuperación de contraseña y la vista principal.

Métodos

- **LoginViewController(UserController, SceneManager)** - Constructor que recibe las dependencias necesarias para la autenticación del usuario y la navegación entre vistas.
- **handleLogin(ActionEvent event)** - Gestiona el intento de inicio de sesión. Recupera las credenciales introducidas por el usuario, invoca el método de autenticación del **UserController** y, si el proceso es correcto, redirige a la vista principal. En caso de error, muestra el mensaje correspondiente en la interfaz.
- **togglePassword()** - Alterna entre la visualización oculta y visible de la contraseña, sincronizando el contenido entre el **PasswordField** y el **TextField** de texto plano.
- **goToRegister(ActionEvent event)** - Redirige al usuario a la vista de registro para crear una nueva cuenta.
- **goToResetPassword(ActionEvent event)** - Redirige al usuario a la vista de recuperación de contraseña.

MySurveyDraftsViewController: **MySurveyDraftsViewController** es el controlador encargado de gestionar la vista que muestra las encuestas del usuario que se encuentran en estado borrador. Esta vista permite al usuario localizar, filtrar y acceder a sus encuestas no publicadas para editarlas o gestionarlas antes de su publicación. El controlador extiende la clase **SurveysViewController**, reutilizando la lógica común de listado y filtrado de encuestas, y adaptándola específicamente al caso de los borradores.

Atributos

- **myDrafts** (**Button**) - Botón del menú lateral que representa la sección de borradores y se marca visualmente como activa cuando esta vista está cargada.
- **centerContentTitle** (**Label**) - Etiqueta central que muestra el título descriptivo de la vista, indicando al usuario que se encuentra en la sección de edición de borradores.
- **TITLE** (**SurveyViewTitle**) - Enumerado que define el título asociado a la vista de borradores, utilizado para mantener coherencia visual y textual en la interfaz.

Métodos

- **MySurveyDraftsViewController(UserController, SurveyController, SurveyView, SceneManager)** - Constructor que recibe los controladores de dominio y el gestor de escenas necesarios para la gestión de encuestas y la navegación entre vistas. Inicializa el título correspondiente a la vista de borradores.

- **setViewActive()** - Marca el botón de borradores del menú lateral como activo, indicando visualmente al usuario la sección en la que se encuentra.
- **setContentTitle()** - Asigna al título central de la vista el texto correspondiente a la sección de borradores, definido en el enumerado **SurveyViewTitle**.
- **getFilter(String filter, String searchText)** - Define el predicado utilizado para filtrar las encuestas mostradas. En este caso, solo se incluyen aquellas encuestas cuyo estado es **DRAFT**, cuyo título coincide con el texto de búsqueda introducido y que cumplen los criterios adicionales seleccionados por el usuario, como valoración media o número de visualizaciones.
- **getSelectedSurveys()** - Obtiene la lista de encuestas del usuario actual desde el controlador de dominio, que posteriormente será filtrada para mostrar únicamente los borradores.

MySurveysViewController: MySurveysViewController es el controlador encargado de gestionar la vista donde el usuario puede consultar y analizar sus encuestas publicadas. Esta vista permite visualizar únicamente aquellas encuestas que ya no se encuentran en estado borrador, ofreciendo opciones de filtrado por valoración, número de visualizaciones y búsqueda por título. El controlador hereda de SurveysViewController, reutilizando la lógica común de listado y filtrado de encuestas, pero adaptándola al contexto de análisis de encuestas propias ya publicadas.

Atributos

- **mySurveys (Button)** - Botón del menú lateral que representa la sección de encuestas publicadas del usuario y que se resalta visualmente cuando esta vista está activa.
- **centerContentTitle (Label)** - Etiqueta central que muestra el título descriptivo de la vista, indicando al usuario que se encuentra en la sección de análisis de sus encuestas.
- **TITLE (SurveyViewTitle)** - Enumerado que define el título asociado a la vista de encuestas publicadas del usuario, garantizando coherencia visual y semántica en la interfaz.

Métodos

- **MySurveysViewController(UserController, SurveyController, SurveyView, SceneManager)** - Constructor que recibe los controladores de usuario y encuestas, así como el gestor de escenas necesario para la navegación. Inicializa la vista con el título correspondiente a la sección de encuestas propias.
- **setViewActive()** - Marca el botón de "Mis encuestas" del menú lateral como activo, proporcionando una referencia visual clara de la sección en la que se encuentra el usuario.
- **setContentTitle()** - Asigna al título central de la vista el texto definido en el enumerado **SurveyViewTitle.MY_SURVEYS**, indicando que se trata de la sección de análisis de encuestas.
- **getFilter(String filter, String searchText)** - Define el criterio de filtrado aplicado a las encuestas mostradas. Se excluyen las encuestas en estado borrador y se filtran aquellas cuyo título contiene el texto de búsqueda. Además, se aplican filtros adicionales opcionales basados en la valoración media y el número de visualizaciones.

- **getSelectedSurveys()** - Obtiene desde el controlador de encuestas la lista de encuestas pertenecientes al usuario autenticado, que posteriormente será filtrada para mostrar únicamente las encuestas publicadas.

RatingPopupController: RatingPopupController es el controlador responsable de gestionar la ventana emergente de valoración de una encuesta. Su función principal es permitir al usuario asignar una puntuación visual mediante estrellas, admitiendo valores enteros y medias estrellas. La vista ofrece una experiencia interactiva e intuitiva, actualizando en tiempo real la representación gráfica de la puntuación seleccionada y notificando el resultado al componente que haya invocado el popup mediante un listener. Una vez confirmada o cancelada la acción, la ventana se cierra automáticamente.

Atributos

- **starsContainer (HBox)** - Contenedor que agrupa visualmente las estrellas de valoración y permite su distribución horizontal en la interfaz.
- **ratingValueLabel (Label)** - Etiqueta que muestra el valor numérico actual de la valoración seleccionada, actualizándose dinámicamente conforme el usuario interactúa con las estrellas.
- **listener (RatingListener)** - Interfaz de callback utilizada para comunicar la puntuación seleccionada al controlador o vista que haya abierto el popup de valoración.
- **currentRating (double)** - Almacena la puntuación actual seleccionada por el usuario, permitiendo valores con incrementos de 0.5.
- **NUM_STARS (int)** - Constante que define el número total de estrellas mostradas en la interfaz de valoración.
- **STAR_SIZE (double)** - Tamaño visual de cada estrella, utilizado para escalar correctamente los elementos SVG.
- **STAR_PATH (String)** - Definición SVG que describe la forma geométrica de una estrella, reutilizada para las versiones vacía, media y completa.

Métodos

- **setListener(RatingListener listener)** - Asigna el listener encargado de recibir la puntuación final seleccionada por el usuario al confirmar la valoración.
- **initialize()** - Método de inicialización invocado automáticamente por JavaFX. Se encarga de generar las estrellas y establecer su estado visual inicial.
- **createStars()** - Crea dinámicamente el conjunto de estrellas dentro del contenedor, inicializando cada una con su comportamiento interactivo correspondiente.
- **createStarPane(int starIndex)** - Construye una estrella individual compuesta por tres capas (vacía, media y completa), gestionando la detección de clics para diferenciar entre media estrella y estrella completa según la posición del cursor.
- **createStarSVG()** - Genera un objeto **SVGPath** configurado con la forma base de la estrella, reutilizable para las distintas variantes visuales.
- **updateStarsDisplay()** - Actualiza el estado visual de todas las estrellas en función del valor actual de la valoración, mostrando estrellas completas, medias o vacías según corresponda, y refrescando el valor numérico mostrado.
- **handleCancel()** - Maneja la acción de cancelar la valoración, cerrando la ventana sin notificar ningún resultado al listener.
- **handleConfirm()** - Confirma la valoración seleccionada, notificando el valor actual al listener y cerrando posteriormente la ventana emergente.
- **closeWindow()** - Cierra de forma segura la ventana del popup de valoración obteniendo el **Stage** asociado a la escena actual.

RegisterViewController: RegisterViewController es el controlador encargado de gestionar la vista de registro de nuevos usuarios en la aplicación. Su responsabilidad principal es coordinar la entrada de datos del formulario de registro, validar la información proporcionada por el usuario (credenciales, confirmación de contraseña y pregunta de seguridad) y delegar en el UserController la creación efectiva de la cuenta. Además, gestiona la visualización de errores, la navegación entre vistas y la funcionalidad de mostrar u ocultar las contraseñas introducidas para mejorar la experiencia de usuario.

Atributos

- **usernameField (TextField)** - Campo de texto donde el usuario introduce su nombre de usuario.
- **emailField (TextField)** - Campo destinado a la introducción del correo electrónico del usuario.
- **passwordField (PasswordField)** - Campo de contraseña principal que oculta el texto introducido por defecto.
- **passwordText (TextField)** - Campo alternativo para mostrar la contraseña en texto plano cuando el usuario activa la opción de visualización.
- **confirmPasswordField (PasswordField)** - Campo de contraseña para confirmar la contraseña principal, ocultando el texto introducido.
- **confirmPasswordText (TextField)** - Campo alternativo para mostrar en texto plano la confirmación de contraseña.
- **securityQuestionField (TextField)** - Campo donde el usuario introduce la pregunta de seguridad asociada a su cuenta.
- **securityAnswerField (TextField)** - Campo de texto para la respuesta a la pregunta de seguridad.
- **showPasswordCheck (CheckBox)** - Checkbox que permite alternar entre mostrar u ocultar las contraseñas introducidas.
- **errorCard (HBox)** - Contenedor visual que muestra los mensajes de error en caso de fallo durante el registro.
- **errorLabel (Label)** - Etiqueta que presenta el mensaje de error concreto al usuario.
- **userController (UserController)** - Controlador de dominio encargado de validar contraseñas y registrar nuevos usuarios.
- **sceneManager (SceneManager)** - Gestor de escenas responsable de la navegación entre vistas de la aplicación.

Métodos

- **handleRegister(ActionEvent event)** - Gestiona el envío del formulario de registro. Recoge los datos introducidos, valida que ambas contraseñas coincidan, comprueba la fortaleza de la contraseña y delega en el **UserController** la creación del usuario. En caso de éxito, redirige a la vista de inicio de sesión; en caso contrario, muestra el mensaje de error correspondiente.
- **toggleRegisterPasswords()** - Controla la visibilidad de los campos de contraseña y confirmación. Permite alternar entre mostrar las contraseñas en texto plano o mantenerlas ocultas, sincronizando correctamente el contenido entre los distintos campos.
- **goToLogin(ActionEvent event)** - Redirige al usuario a la vista de inicio de sesión, cancelando el proceso de registro actual.

ResetPasswordStep1Controller: ResetPasswordStep1Controller es el controlador encargado de gestionar el primer paso del proceso de recuperación de contraseña. En esta vista, el usuario introduce su nombre de usuario para iniciar el flujo de restablecimiento. El controlador valida la existencia del usuario y obtiene la pregunta de seguridad asociada a la

cuenta a través del UserController. Si el proceso se inicia correctamente, se navega al segundo paso de recuperación; en caso contrario, se informa al usuario del error correspondiente.

Atributos

- **usernameField (TextField)** - Campo de texto donde el usuario introduce su nombre de usuario para comenzar el proceso de recuperación de contraseña.
- **errorCard (HBox)** - Contenedor visual utilizado para mostrar mensajes de error cuando el nombre de usuario introducido no es válido o el proceso falla.
- **errorLabel (Label)** - Etiqueta que muestra el mensaje de error específico devuelto por la capa de dominio.
- **userController (UserController)** - Controlador de la capa de dominio responsable de iniciar el proceso de recuperación de contraseña y validar la existencia del usuario.
- **sceneManager (SceneManager)** - Gestor de escenas encargado de controlar la navegación entre vistas durante el flujo de recuperación.

Métodos

- **handleContinue()** - Gestiona la acción de continuar al siguiente paso del proceso de recuperación. Obtiene el nombre de usuario introducido, solicita al **UserController** el inicio de la recuperación y, si es correcto, navega a la segunda vista de recuperación pasando el nombre de usuario y la pregunta de seguridad. En caso de error, muestra el mensaje correspondiente al usuario.
- **goBack()** - Permite volver a la vista de inicio de sesión, cancelando el proceso de recuperación de contraseña iniciado.

ResetPasswordStep2Controller: ResetPasswordStep2Controller es el controlador responsable del segundo paso del proceso de recuperación de contraseña. En esta vista, el usuario debe responder a la pregunta de seguridad asociada a su cuenta. El controlador muestra dinámicamente la pregunta obtenida en el paso anterior y valida la respuesta introducida mediante el UserController. Si la respuesta es correcta, el flujo continúa hacia el tercer paso de recuperación; en caso contrario, se informa al usuario del error.

Atributos

- **questionLabel (Label)** - Etiqueta que muestra la pregunta de seguridad asociada al usuario cuya contraseña se desea recuperar.
- **answerField (TextField)** - Campo de texto donde el usuario introduce la respuesta a la pregunta de seguridad.
- **errorCard (HBox)** - Contenedor visual utilizado para mostrar mensajes de error cuando la respuesta introducida es incorrecta o se produce un fallo durante la validación.
- **errorLabel (Label)** - Etiqueta que muestra el mensaje de error específico que describe el motivo del fallo.
- **userController (UserController)** - Controlador de la capa de dominio encargado de validar la respuesta de seguridad proporcionada por el usuario.
- **sceneManager (SceneManager)** - Gestor de escenas que controla la navegación entre las distintas vistas del flujo de recuperación de contraseña.
- **username (String)** - Nombre de usuario cuya identidad se está verificando durante el proceso de recuperación.

- **question (String)** - Pregunta de seguridad asociada al usuario, recibida desde el primer paso del flujo y mostrada en la interfaz.

Métodos

- **initialize()** - Inicializa la vista estableciendo el texto de la pregunta de seguridad en la interfaz gráfica, garantizando que el usuario visualiza la pregunta correcta correspondiente a su cuenta.
- **handleValidate()** - Gestiona la validación de la respuesta introducida por el usuario. Comprueba la respuesta mediante el **UserController** y, si es correcta, navega al tercer paso del proceso de recuperación. En caso de error o respuesta incorrecta, muestra un mensaje informativo al usuario.
- **goBack()** - Permite regresar al primer paso del proceso de recuperación de contraseña, manteniendo una navegación controlada y coherente dentro del flujo.

ResetPasswordStep3Controller: ResetPasswordStep3Controller es el controlador encargado del tercer y último paso del proceso de recuperación de contraseña. En esta vista, el usuario introduce y confirma una nueva contraseña para su cuenta. El controlador valida que ambas contraseñas coincidan y que cumplan los criterios de seguridad definidos en la capa de dominio. Si la validación es correcta, la contraseña se actualiza y el usuario es redirigido a la pantalla de inicio de sesión.

Atributos

- **passwordField (PasswordField)** - Campo de entrada donde el usuario introduce la nueva contraseña.
- **confirmPasswordField (PasswordField)** - Campo de entrada donde el usuario confirma la nueva contraseña para evitar errores de escritura.
- **errorCard (HBox)** - Contenedor visual utilizado para mostrar mensajes de error cuando la validación falla o se produce algún problema durante el cambio de contraseña.
- **errorLabel (Label)** - Etiqueta que muestra el mensaje de error específico asociado al fallo producido.
- **userController (UserController)** - Controlador de la capa de dominio responsable de validar la fortaleza de la contraseña y aplicar el cambio definitivo en la cuenta del usuario.
- **sceneManager (SceneManager)** - Gestor de escenas que controla la navegación de la aplicación, permitiendo redirigir al usuario tras completar el proceso.
- **username (String)** - Identificador del usuario cuya contraseña se está restableciendo.

Métodos

- **handleSave()** - Gestiona la acción de guardar la nueva contraseña. Comprueba que ambas contraseñas introducidas coincidan, valida su fortaleza mediante el **UserController** y, si todo es correcto, actualiza la contraseña del usuario. Una vez completado el proceso, redirige al usuario a la vista de inicio de sesión.
- **goBack()** - Cancela el proceso de recuperación y redirige directamente a la pantalla de login, permitiendo al usuario abandonar el flujo en cualquier momento.

SceneManager: SceneManager es la clase central encargada de la **gestión de la navegación y las vistas** de la aplicación JavaFX. Actúa como intermediario entre los controladores de presentación y el Stage principal, permitiendo cambiar de pantalla, abrir ventanas emergentes y mantener una navegación coherente en toda la aplicación.

Esta clase concentra toda la lógica relacionada con la carga de archivos FXML, la inyección manual de controladores y la configuración del escenario principal, evitando que dicha responsabilidad se disperse entre las distintas vistas.

Atributos

- **UserController (UserController)** - Controlador de dominio responsable de la gestión de usuarios y sesión. Se inyecta para que las vistas relacionadas con autenticación y perfil puedan acceder a la lógica de usuario.
- **surveyController (SurveyController)** - Controlador de dominio encargado de la gestión de encuestas. Se utiliza para cargar, editar, visualizar y responder encuestas desde las distintas vistas.
- **clusteringController (CtrlDominioClustering)** - Controlador especializado en análisis de clustering, utilizado para mostrar la vista de análisis de resultados de una encuesta.
- **responseController (ResponseController)** - Controlador responsable de la gestión de respuestas de encuestas, necesario para la vista de respuesta y valoración.
- **primaryStage (Stage)** - Escenario principal de la aplicación JavaFX. Todas las vistas principales se cargan dentro de este **Stage**.

Métodos

- **setPrimaryStage(Stage stage)** - Asigna el **Stage** principal que utilizará la aplicación. Este método debe ejecutarse antes de inicializar cualquier vista.
- **getPrimaryStage()** - Devuelve el **Stage** principal, utilizado principalmente para abrir ventanas modales dependientes.
- **getClusteringController()** - Proporciona acceso al controlador de clustering para aquellas vistas que lo requieran.
- **initStage()** - Inicializa el escenario principal estableciendo tamaño mínimo, centrado en pantalla y visibilidad. Garantiza que la aplicación tenga una configuración base coherente.
- **checkLoaderAddress(FXMLLoader loader)** - Método auxiliar que verifica que la ruta del archivo FXML exista correctamente. En caso contrario, detiene la ejecución para evitar errores silenciosos.
- **finalizeScene(FXMLLoader loader, String title)** - Carga el contenido FXML indicado y lo establece como raíz de la escena actual. Si la escena no existe, la crea; si ya existe, reemplaza su contenido. También actualiza el título de la ventana.

Métodos de navegación principal

Estos métodos cargan vistas completas dentro del Stage principal:

- **showLogin()** - Muestra la vista de inicio de sesión.
- **showRegister()** - Muestra la vista de registro de nuevos usuarios.
- **showHome()** - Carga la vista principal con encuestas públicas disponibles.
- **showMySurveys()** - Muestra las encuestas creadas por el usuario que ya han sido publicadas.
- **showMySurveysDrafts()** - Muestra las encuestas en estado borrador del usuario.
- **showCreateSurvey()** - Abre la vista de creación de encuestas.
- **showEditSurvey(String surveyId)** - Carga la vista de edición de una encuesta en estado borrador.
- **showViewSurvey(String surveyId)** - Muestra una encuesta publicada en modo solo lectura.

- **showAnswerSurvey(String surveyId)** - Carga la vista para responder una encuesta concreta.
- **showClustering(String surveyId)** - Abre la vista de análisis de clustering asociada a una encuesta.

Métodos de recuperación de contraseña

- **showPasswordRecovery()** - Inicia el proceso de recuperación de contraseña (paso 1).
- **showPasswordRecoveryStep2(String username, String question)** - Muestra el segundo paso del proceso, donde el usuario responde a la pregunta de seguridad.
- **showPasswordRecoveryStep3(String username)** - Carga el paso final para introducir una nueva contraseña.

Ventanas emergentes y modales

- **showDeleteAccountConfirm()** - Abre una ventana emergente independiente para confirmar la eliminación de la cuenta del usuario.
- **showConfirmLeave(Runnable onConfirm)** - Muestra un popup de confirmación al abandonar una vista con cambios no guardados.
- **showRatingPopup(RatingPopupController.RatingListener listener)** - Abre una ventana modal para valorar una encuesta mediante estrellas.

SurveysViewController: SurveysViewController es una **clase abstracta de la capa de presentación** que centraliza toda la lógica común relacionada con la **visualización, filtrado, ordenación y paginación de encuestas**. Sirve como base para las vistas de *Inicio*, *Mis Encuestas* y *Borradores*, evitando duplicación de código y garantizando un comportamiento homogéneo en todas ellas.

Esta clase gestiona la interacción del usuario con el buscador, los filtros, el ordenamiento y la paginación, delegando únicamente en las subclases la definición de qué encuestas se muestran y qué criterios de filtrado se aplican según el contexto de la vista.

Atributos

- **UserController (UserController)** - Controlador de dominio encargado de la gestión de usuarios y sesión. Se utiliza para comprobar el estado de autenticación y obtener información del usuario actual.
- **surveyController (SurveyController)** - Controlador de dominio responsable de la gestión de encuestas. Proporciona los datos que se muestran en las distintas vistas.
- **sceneManager (SceneManager)** - Gestor de navegación que permite cambiar entre vistas y abrir ventanas emergentes desde la interfaz.
- **surveyView (SurveyView)** - Indica el tipo de vista activa (HOME, MY_SURVEYS o DRAFTS), utilizado para adaptar el comportamiento y la presentación de las tarjetas de encuesta.
- **MAX_SURVEYS_PER_PAGE (int)** - Número máximo de encuestas que se muestran por página en el sistema de paginación.
- **numSurveys (int)** - Número total de encuestas que cumplen los filtros actuales. Se utiliza para calcular el número de páginas.
- **allSurveys (List<Survey>)** - Lista completa de encuestas disponibles antes de aplicar filtros, búsqueda y ordenación.

Atributos FXML principales

- **usernameLabel / avatarLabel** - Muestran el nombre del usuario autenticado y sus iniciales en el sidebar.
- **searchField** (**TextField**) - Campo de búsqueda para filtrar encuestas por texto.
- **filterCombo** (**ComboBox<String>**) - Selector de filtros predefinidos (rating, visualizaciones, etc.).
- **sortCombo** (**ComboBox<String>**) - Selector de criterio de ordenación de encuestas.
- **surveysContainer** (**VBox**) - Contenedor donde se renderizan dinámicamente las tarjetas de encuestas.
- **emptyStateBox** (**VBox**) - Contenedor que se muestra cuando no hay encuestas disponibles.
- **pagination** (**Pagination**) - Control de paginación para navegar entre páginas de encuestas.

Métodos abstractos

- **setViewActive()** - Define qué botón del sidebar debe marcarse como activo según la vista concreta.
- **setContentTitle()** - Establece el título principal del contenido central de la vista.
- **getSelectedSurveys()** - Devuelve la lista inicial de encuestas que debe manejar la vista concreta.
- **getFilter(String filter, String searchText)** - Define el predicado de filtrado específico que se aplicará según la vista hija.

Métodos principales

- **initialize(URL url, ResourceBundle resourceBundle)** - Método de inicialización que comprueba si el usuario está autenticado, configura el sidebar, inicializa filtros y carga las encuestas correspondientes a la vista.
- **setFilters()** - Configura los ComboBox de filtros y ordenación, así como los listeners del buscador para aplicar los filtros de forma reactiva.
- **applyFilters()** - Aplica de forma conjunta la búsqueda por texto, el filtro seleccionado y el criterio de ordenación, actualizando la lista visible de encuestas y la paginación.
- **renderSurveyList(List<Survey> surveys)** - Renderiza dinámicamente las tarjetas de encuestas en el contenedor principal o muestra el estado vacío si no hay resultados.
- **updatePageCount()** - Actualiza el número total de páginas del control de paginación según el número de encuestas filtradas.

Métodos auxiliares

- **getInitialLetters(String username)** - Obtiene las iniciales del nombre del usuario para mostrarlas en el avatar.
- **getCurrentPageIndex()** - Devuelve el índice de la página actual seleccionada en la paginación.
- **getNumPages()** - Calcula el número total de páginas necesarias para mostrar todas las encuestas filtradas.

Métodos de navegación

- **goToHome(ActionEvent event)** - Navega a la vista principal de encuestas.
- **goToCreateSurvey(ActionEvent event)** - Navega a la vista de creación de encuestas.
- **goToMySurveys(ActionEvent event)** - Navega a la vista de encuestas publicadas del usuario.

- **goToMyDrafts(ActionEvent event)** - Navega a la vista de borradores del usuario.
- **handleLogout(ActionEvent event)** - Cierra la sesión del usuario y redirige a la vista de login.
- **handleDeleteAccount(ActionEvent event)** - Abre la ventana de confirmación para eliminar la cuenta del usuario.

ViewSurveyViewController: ViewSurveyViewController es el controlador de la **vista de visualización de encuestas publicadas** dentro de la capa de presentación. Su función principal es mostrar una encuesta en **modo solo lectura**, incluyendo su información general, estadísticas asociadas y el listado completo de preguntas, sin permitir modificaciones.

Esta vista está pensada para que el creador de la encuesta pueda **consultar el estado actual**, revisar métricas como visualizaciones, valoraciones y número de respuestas, así como ejecutar acciones avanzadas como el **cierre de la encuesta** o el **análisis de clustering** sobre las respuestas recopiladas.

Atributos

- **UserController (UserController)** - Controlador de dominio encargado de la gestión del usuario y de la sesión activa. Se utiliza para verificar permisos y obtener información del usuario autenticado.
- **surveyController (SurveyController)**
Controlador de dominio responsable de acceder y modificar los datos de las encuestas, así como de recuperar la información completa de la encuesta a visualizar.
- **sceneManager (SceneManager)** - Gestor de navegación entre vistas. Permite volver a vistas anteriores, abrir la vista de clustering o redirigir al usuario en caso de error o falta de permisos.
- **surveyId (String)** - Identificador único de la encuesta que se va a visualizar.
- **currentSurvey (Survey)** - Modelo de datos que contiene toda la información de la encuesta actualmente cargada, incluyendo metadatos, preguntas y estadísticas.

Atributos FXML principales

- **avatarLabel / usernameLabel** - Muestran las iniciales y el nombre del usuario autenticado en el sidebar.
- **surveyTitleLabel / surveyDescLabel** - Muestran el título y la descripción de la encuesta.
- **statusBadge** - Indicador visual del estado de la encuesta (publicada o cerrada).
- **viewsCountLabel / responsesCountLabel / ratingLabel** - Muestran estadísticas clave de la encuesta: número de visualizaciones, número de respuestas y valoración media.
- **createdDateLabel / publishedDateLabel** - Muestran las fechas de creación y publicación de la encuesta formateadas para el usuario.
- **questionsContainer (VBox)** - Contenedor donde se renderizan dinámicamente las preguntas de la encuesta en modo lectura.
- **scrollPane (ScrollPane)** - Permite el desplazamiento vertical del contenido cuando el número de preguntas es elevado.

Métodos principales

- **initialize(URL url, ResourceBundle resourceBundle)** - Comprueba que el usuario esté autenticado, configura el sidebar, inicializa el scroll y carga la encuesta asociada al identificador recibido.
- **loadSurvey()** - Recupera la encuesta desde el controlador de dominio, valida que pertenezca al usuario autenticado y rellena todos los campos de la vista con la información correspondiente.
- **updateStatusBadge()** - Actualiza el texto y el estilo visual del indicador de estado según si la encuesta está publicada o cerrada.
- **handleRunClustering(ActionEvent event)** - Lanza la vista de análisis de clustering si la encuesta dispone de respuestas suficientes para realizar el análisis.
- **handleCloseSurvey(ActionEvent event)** - Permite cerrar la encuesta tras confirmación del usuario, impidiendo que se puedan recibir nuevas respuestas.
- **handleBack(ActionEvent event)** - Vuelve a la vista de listado de encuestas del usuario.

Métodos de renderizado

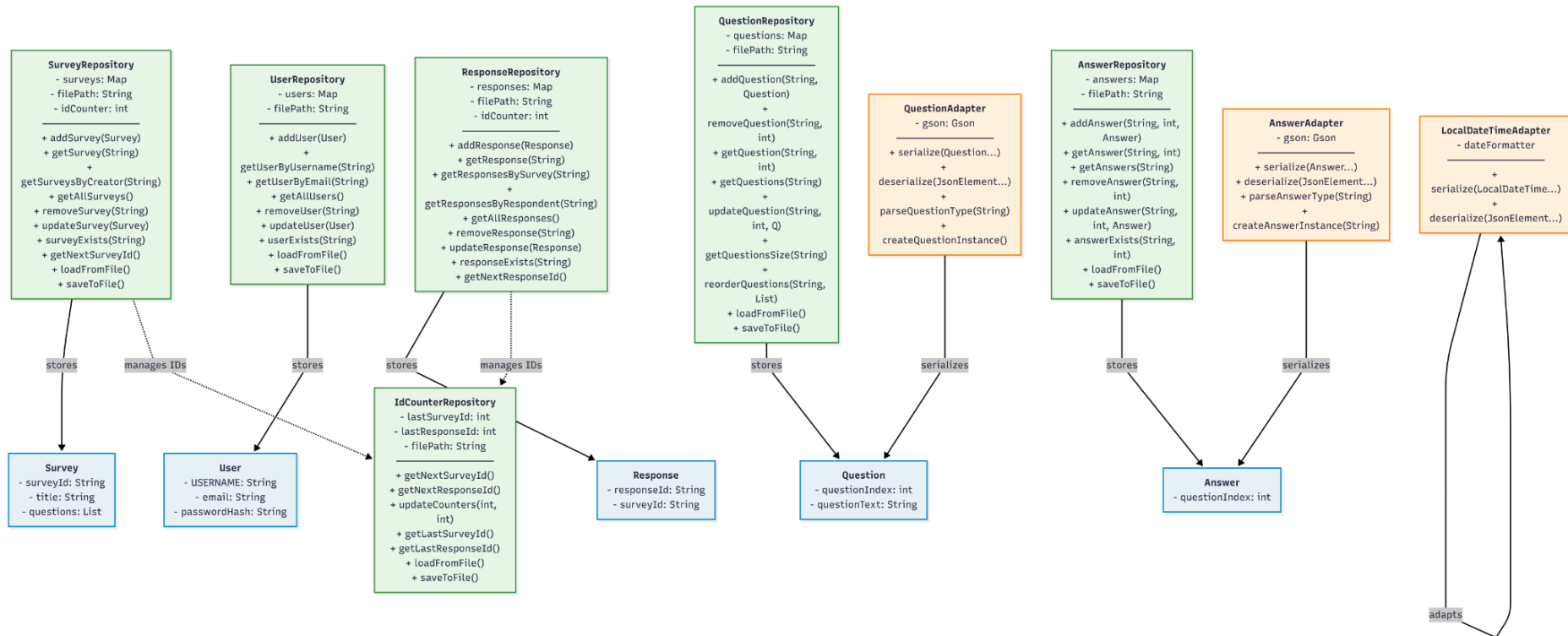
- **renderQuestions()** - Recorre la lista de preguntas de la encuesta y las renderiza en modo solo lectura, mostrando un mensaje informativo si la encuesta no contiene preguntas.
- **createQuestionCard(Question q, int index)** - Genera una tarjeta visual para cada pregunta, mostrando su número, tipo, texto y contenido asociado según el tipo de pregunta.
- **getTypeText(TypeQuestion type)** - Traduce el tipo interno de pregunta a un texto legible para el usuario.

Métodos auxiliares

- **getInitialLetters(String username)** - Obtiene las iniciales del nombre del usuario para mostrarlas en el avatar del sidebar.
- **setViewActive()** - Marca visualmente la opción "Mis Encuestas" como activa en el menú lateral.
- **showAlert(Alert.AlertType type, String title, String content)** - Muestra mensajes informativos, de advertencia o de error al usuario mediante cuadros de diálogo.



2.1.3. [Capa de Persistencia](#)



2.1.3.1. Estructura de repositorios

La capa de datos del proyecto implementa un patrón de repositorio en capas que gestiona la persistencia y recuperación de datos en JSON. Los repositorios están organizados jerárquicamente:

Organización Jerárquica

1. **IdCounterRepository (capa raíz):** Gestiona contadores autoincrement globales para encuestas (lastSurveyId) y respuestas (lastResponseId). Es la fuente única de verdad para IDs únicos.
2. **UserRepository:** Almacena usuarios con sus credenciales (username, email, hash de contraseña, pregunta de seguridad). Mantiene también referencias a encuestas creadas y respuestas realizadas mediante índices dobles.
3. **SurveyRepository:** Gestiona encuestas con sus metadatos (título, descripción, estado, fechas). Delega en IdCounterRepository para obtener nuevos IDs y mantiene un índice inverso por usuario creador.
4. **QuestionRepository:** Gestiona las preguntas asociadas a cada encuesta mediante un doble clave (surveyId → List<Question>). Las operaciones respetan los índices y reajustan automáticamente al eliminar o reordenar.
5. **ResponseRepository:** Almacena respuestas completas con doble indexación: por encuesta (surveyId → responseId → Response) e invertida por usuario (respondentUsername → conjunto de responseIds). Es el más complejo debido a la necesidad de mantener coherencia bidireccional.
6. **AnswerRepository:** Repositorio complementario que gestiona respuestas individuales. En la arquitectura actual, los Answer se mantienen dentro de Response, pero este repositorio existe para futuras extensiones.

2.1.3.2. Modelos de almacenamiento en memoria

Estructura de Datos Base

Cada repositorio mantiene estructuras en memoria que replican archivos JSON:

IdCounterRepository:

- currentSurveyId: int = 201
- currentResponseId: int = 125
- **Persistido en:** DATA/db/last-id.json

UserRepository:

- Map<username, User> → username es clave única
 - └ User { username, email, passwordHash, securityQuestion, securityAnswer }
 - └ surveyIds: Set<String> (encuestas creadas)
 - └ respondedSurveyIds: Map<surveyId, Set<responseId>>
- **Persistido en:** DATA/db/users.json

SurveyRepository:

- Map<surveyId, Survey> → surveyId es clave única
 - └ Survey { id, title, description, creatorUsername, status, createdAt, publishedAt }
 - └ questions: List<Question> (ordenadas por índice)
 - └ views, rating, totalRating (estadísticas)
- **Índice Inverso:**
 - Map<username, Set<surveyId>> → búsqueda rápida por creador
- **Persistido en:** DATA/db/surveys.json

QuestionRepository:

- Map<surveyId, List<Question>> → preguntas indexadas por encuesta
 - └ Question {questionIndex, surveyId, questionText, typeQuestion, required}
 - └ TextualQuestion
 - └ NumericalQuestion
 - └ MultipleChoiceQuestion {options, minSelections, maxSelections}
- **Persistido en:** surveys.json (anidadas dentro de cada Survey)

ResponseRepository:

- Map<surveyId, Map<responselId, Response>> → doble clave
 - └ Response {responselId, surveyId, respondentUsername, status, submittedAt}
 - └ answers: Map<Integer, Answer> → por índice de pregunta
 - └ TextualAnswer {text, isAnswered}
 - └ NumericalAnswer {value, isAnswered}
 - └ MultipleChoiceAnswer {selections[], isAnswered}
- **Índice Inverso (respondentIndex):**
 - Map<username, Set<responselId>> → respuestas por usuario
- **Persistido en:** DATA/db/responses.json

2.1.3.3. Métodos de acceso y gestión de datos

Ciclo de Vida CRUD + Persistencia

- **CREATE:** Valida entrada, genera ID único desde IdCounterRepository, crea objeto en memoria, lo añade al mapa principal, actualiza índices secundarios y persiste a JSON.
- **READ:** Acceso directo O(1) por clave en HashMap, búsquedas O(1) mediante índices secundarios, o listados O(n) completos según necesidad.
- **UPDATE:** Obtiene entidad actual, valida cambios, actualiza campos en memoria, sincroniza índices afectados, persiste a JSON.
- **DELETE:** Verifica existencia, elimina de mapas principales, limpia índices secundarios, aplica cascadas (elimina dependencias directas), persiste a JSON.

Métodos Especializados

Métodos exists*() validan existencia; búsquedas por índice secundario evitan escaneos completos; operaciones en lote (clearAll*()) resetean repositorios; reordenación (reorderQuestions()) mantiene coherencia de índices. Persistencia JSON usa adaptadores especializados para manejar polimorfismo y tipos complejos. Coherencia se garantiza mediante validaciones previas, transacciones lógicas atómicas, cascadas controladas e índices síncronos que se actualizan simultáneamente con estructuras principales.

2.1.3.4. Descripción de todas las clases

data/adapter

AnswerAdapter: Adaptador JSON para deserializar respuestas polimórficas (Answer).

Atributos: (ninguno, es un adaptador)

Métodos:

- `deserialize(JsonElement json, Type type, JsonDeserializationContext context):`
Answer - Convierte un elemento JSON en su correspondiente tipo de Answer (MultipleChoiceAnswer, TextualAnswer o NumericalAnswer) según el campo typeAnswer, usando el contexto para hacer la deserialización delegada correcta.

LocalDateTimeAdapter: Adaptador JSON para serializar/deserializar LocalDateTime.

Atributos:

- `FORMATTER:` DateFormatter - privado, estático, final - Define el formato ISO_LOCAL_DATE_TIME para conversión de fechas.

Métodos:

- `serialize(LocalDateTime localDateTime, Type var2, JsonSerializationContext var3):`
JsonElement - Convierte un LocalDateTime a JsonElement usando el formato FORMATTER.
- `deserialize(JsonElement jsonElement, Type var2, JsonDeserializationContext var3):`
LocalDateTime - Convierte un JsonElement a LocalDateTime parseando con el formato FORMATTER.

QuestionAdapter: Adaptador JSON para deserializar preguntas polimórficas (Question).

Atributos:

- `DELEGATE_GSON:` Gson - privado, estático, final - Instancia de Gson para fallback en deserialización delegada.

Métodos:

- `deserialize(JsonElement json, Type type, JsonDeserializationContext context):`
Question - Convierte un elemento JSON en su correspondiente tipo de Question (MultipleChoiceQuestion u otros) según el campo typeQuestion, manejando compatibilidad con JSON antiguos (optionQuestions → options, transformando strings en objetos con optionText).

data

IdCounterRepository: Repositorio que gestiona los contadores de IDs autoincrement para encuestas y respuestas, persistiendo los valores en JSON para mantener coherencia entre sesiones.

Atributos:

- `currentSurveyId:` int - privado - Almacena el último ID de encuesta generado
- `currentResponseId:` int - privado - Almacena el último ID de respuesta generado
- `FILE_PATH:` String - privado, estático, final - Ruta al archivo JSON que persiste los contadores (DATA/db/last-id.json)

Métodos:

- **IdCounterRepository(int ,startId)** - Constructor que inicializa los contadores; carga desde JSON si existe, sino usa startId como valor inicial
- **String getNextIdAndIncrement()** - Retorna el siguiente ID de respuesta válido; incrementa el contador actual, guarda en JSON, y devuelve el nuevo ID como string
- **String generateNextSurveyId()** - Genera y devuelve el siguiente ID único para encuestas; incrementa currentSurveyId, persiste a JSON, devuelve como string
- **String generateNextResponseId()** - Genera y devuelve el siguiente ID único para respuestas; incrementa currentResponseId, persiste a JSON, devuelve como string (ej: "126")
- **int getCurrentSurveyId()** - Devuelve el valor actual del contador de encuestas sin incrementar
- **int getCurrentResponseId()** - Devuelve el valor actual del contador de respuestas sin incrementar
- **void reset()** - Reinicia ambos contadores a valores iniciales y persiste en JSON (usado en pruebas o limpieza)
- **void loadFromJson()** - Lee el archivo last-id.json y carga los contadores actuales; si el archivo no existe, inicializa con valores por defecto
- **void saveToJson()** - Escribe los contadores actuales al archivo last-id.json en formato {"lastSurveyId": X, "lastResponseId": Y}

AnswerRepository: Repositorio que gestiona el almacenamiento y recuperación de respuestas individuales (Answer). En la arquitectura actual, los Answer se almacenan dentro de Response, por lo que este repositorio tiene funcionalidad limitada pero se mantiene por separación de conceptos.

Atributos:

- **answers: Map<String, Answer>** - privado - Mapa que almacena Answer por ID (aunque en uso real están contenidos en Response)
- **FILE_PATH:** String - privado, estático, final - Ruta reservada para almacenamiento (si se decide persistir separadamente)

Métodos:

- **AnswerRepository()** - Constructor que inicializa el mapa de respuestas
- **void addAnswer(String answerId, Answer answer)** - Añade una respuesta individual al repositorio si se usa de forma independiente
- **Answer getAnswer(String answerId)** - Obtiene una respuesta por su ID
- **void deleteAnswer(String answerId)** - Elimina una respuesta específica
- **boolean existsAnswer(String answerId)** - Verifica si una respuesta existe
- **Map<String, Answer> getAllAnswers()** - Devuelve todas las respuestas almacenadas
- **void clearAnswers()** - Elimina todas las respuestas (usado en tests o limpieza)

QuestionRepository: Repositorio que gestiona operaciones sobre preguntas por encuesta. Las preguntas se almacenan dentro de Survey pero este repositorio proporciona métodos de acceso convenientes.

Atributos:

- **questions: Map<String, List<Question>>** - privado - Mapa que almacena listas de preguntas por SURVEYID

- **FILE_PATH:** String - privado, estático, final - Ruta para persistencia de preguntas (si se decide)

Métodos:

- **QuestionRepository()** - Constructor que inicializa el mapa de preguntas
- **void addQuestion(String surveyId, Question question)** - Añade una pregunta a la lista de preguntas de una encuesta; verifica que surveyId exista, calcula el siguiente índice, establece questionIndex en la pregunta
- **void removeQuestion(String surveyId, int questionIndex)** - Elimina una pregunta por su índice; verifica índice válido, reajusta índices de preguntas posteriores
- **void updateQuestion(String surveyId, int questionIndex, Question updatedQuestion)** - Actualiza una pregunta en su índice; verifica existencia, copia campos nuevos
- **Question getQuestion(String surveyId, int questionIndex)** - Obtiene una pregunta específica por surveyId e índice
- **List<Question> getQuestions(String surveyId)** - Devuelve todas las preguntas de una encuesta
- **int getQuestionCount(String surveyId)** - Devuelve el número total de preguntas en una encuesta
- **boolean existsQuestion(String surveyId, int questionIndex)** - Verifica que una pregunta existe en un índice válido
- **void clearQuestions(String surveyId)** - Elimina todas las preguntas de una encuesta (usado en edición o eliminación)

ResponseRepository: Repositorio que gestiona respuestas completas a encuestas, incluyendo almacenamiento persistente en JSON y un doble índice (por encuesta y por usuario).

Atributos:

- **responses:** Map<String, Map<String, Response>> - privado - Mapa doble: surveyId -> responseId -> Response
- **respondentIndex:** Map<String, Set<String>> - privado - Índice inverso: username -> conjunto de responseIds respondidas
- **surveyIndex:** Set<String> - privado - Conjunto de surveyIds que tienen respuestas
- **responsesFile:** File - privado - Referencia al archivo responses.json
- **FILE_PATH:** String - privado, estático, final - Ruta al archivo de respuestas (DATA/db/responses.json)

Métodos:

- **ResponseRepository()** - Constructor que inicializa mapas e índices, carga respuestas desde JSON si existen
- **void addResponse(String surveyId, Response response)** - Persiste una nueva respuesta: verifica que surveyId exista en responses, añade la respuesta al mapa, persiste a JSON
- **Response getResponse(String surveyId, String responseId)** - Obtiene una respuesta específica dados surveyId y responseId
- **void deleteResponse(String surveyId, String responseId)** - Elimina una respuesta y limpia referencias en índices; lanza ResponseException si no existe
- **List<Response> getResponsesBySurveyId(String surveyId)** - Devuelve todas las respuestas de una encuesta en forma de lista

- `boolean existsResponse(String surveyId, String responseId)` - Verifica si una respuesta existe
- `boolean existsSurveyEntry(String surveyId)` - Verifica que existe entrada para esa encuesta en el mapa de respuestas
- `void addSurveyEntry(String surveyId)` - Crea una entrada vacía para una encuesta nueva en respuestas si no existe
- `void deleteResponsesBySurvey(String surveyId)` - Elimina todas las respuestas de una encuesta (usado al eliminar encuesta); también limpia referencia en surveyIndex
- `Map<String, Response> getAllResponsesForSurvey(String surveyId)` - Devuelve el mapa completo de respuestas de una encuesta
- `void updateResponse(String surveyId, String responseId, Response updatedResponse)` - Actualiza una respuesta existente; verifica que existe, reemplaza datos, persiste
- `void saveResponsesToJson()` - Escribe todas las respuestas a responses.json usando adaptadores JSON para serialización compleja
- `void loadResponsesFromJson()` - Lee responses.json y reconstruye la estructura de mapas; maneja deserialization compleja con adaptadores
- `void clearAllResponses()` - Limpia todos los datos en memoria y en JSON (usado en tests)

SurveyRepository: Repositorio que gestiona encuestas, incluyendo almacenamiento persistente en JSON, gestión de preguntas asociadas, y búsqueda por usuario creador.

Atributos:

- `surveys`: `Map<String, Survey>` - privado - Mapa que almacena encuestas por SURVEYID
- `surveysByUser`: `Map<String, Set<String>>` - privado - Índice inverso: username -> conjunto de surveyIds creadas por ese usuario
- `idCounterRepository`: `IdCounterRepository` - privado, final - Para generar nuevos IDs de encuesta
- `surveysFile`: `File` - privado - Referencia al archivo surveys.json
- `FILE_PATH`: `String` - privado, estático, final - Ruta al archivo de encuestas (DATA/db/surveys.json)

Métodos:

- `SurveyRepository(IdCounterRepository idCounterRepository)` - Constructor que inyecta `IdCounterRepository`, inicializa mapas e índices, carga encuestas desde JSON si existen
- `String generateNextSurveyId()` - Delega en `idCounterRepository.generateNextSurveyId` para obtener nuevo ID único
- `void addSurvey(Survey survey)` - Persiste una encuesta: verifica que no exista ya con ese ID, añade al mapa, actualiza índice de usuario, guarda en JSON
- `Survey getSurvey(String surveyId)` - Obtiene una encuesta por su ID; lanza `SurveyException` si no existe
- `void deleteSurvey(String surveyId)` - Elimina una encuesta: obtiene encuesta, extrae usuario creador, remueve de ambos mapas, persiste a JSON
- `void deleteQuestion(String surveyId, int questionIndex)` - Elimina una pregunta de una encuesta: valida índice, reajusta índices posteriores, persiste

- `void swapQuestions(String surveyId, int oldIndex, int newIndex)` - Intercambia las posiciones de dos preguntas: reajusta todos los índices, persiste
- `List<Question> getAllQuestions(String surveyId)` - Devuelve todas las preguntas de una encuesta; lanza excepción si encuesta no existe
- `Question getQuestion(String surveyId, int questionIndex)` - Obtiene una pregunta específica; valida que ambos parámetros sean válidos
- `int getNumQuestions(String surveyId)` - Devuelve el número total de preguntas en una encuesta
- `List<Survey> getSurveysByUsername(String username)` - Devuelve todas las encuestas creadas por un usuario específico
- `List<Survey> getAllSurveys()` - Devuelve todas las encuestas almacenadas
- `List<String> getAllSurveysId()` - Devuelve todos los IDs de encuestas
- `boolean existsSurvey(String surveyId)` - Verifica si una encuesta existe
- `void updateSurvey(String surveyId, Survey updatedSurvey)` - Actualiza una encuesta existente: verifica que existe, reemplaza datos, persiste a JSON
- `void saveSurveysToJson()` - Escribe todas las encuestas a surveys.json usando adaptadores para preguntas y respuestas
- `void loadSurveysFromJson()` - Lee surveys.json y reconstruye la estructura de mapas e índices; maneja deserialization de preguntas polimórficas
- `void clearAllSurveys()` - Limpia todos los datos en memoria y en JSON (usado en tests)
- `Set<String> getSurveysIdByUser(String username)` - Obtiene el conjunto de IDs de encuestas creadas por un usuario (acceso directo al índice)

UserRepository: Repositorio que gestiona usuarios, incluyendo autenticación, almacenamiento persistente de perfiles y gestión de encuestas creadas y respondidas.

Atributos:

- `users: Map<String, User>` - privado - Mapa que almacena usuarios por username
- `usersFile: File` - privado - Referencia al archivo users.json
- `FILE_PATH: String` - privado, estático, final - Ruta al archivo de usuarios (DATA/db/users.json)

Métodos:

- `UserRepository()` - Constructor que inicializa el mapa de usuarios, carga usuarios desde JSON si existen
- `void addUser(User user)` - Persiste un usuario: verifica que no exista ya, añade al mapa, guarda en JSON
- `boolean existsUser(String username)` - Verifica si un usuario existe en el repositorio
- `User getUser(String username)` - Obtiene un usuario por su nombre; devuelve null si no existe
- `void deleteUser(String username)` - Elimina un usuario: obtiene usuario, elimina del mapa, persiste a JSON
- `void addSurveyId(String username, String surveyId)` - Registra que un usuario ha creado una encuesta: obtiene usuario, añade ID al conjunto de encuestas creadas, persiste
- `void deleteSurveyId(String username, String surveyId)` - Elimina referencia a encuesta creada: obtiene usuario, remueve ID, persiste

- `Set<String> getSurveyIds(String username)` - Obtiene todas las encuestas creadas por un usuario
- `List<User>getAllUsers()` - Devuelve todos los usuarios
- `void updateUser(User user)` - Actualiza los datos de un usuario existente: verifica que existe, reemplaza datos, persiste
- `void saveUsersToJson()` - Escribe todos los usuarios a users.json usando serialización JSON
- `void loadUsersFromJson()` - Lee users.json y reconstruye el mapa de usuarios
- `void clearAllUsers()` - Limpia todos los datos en memoria y en JSON (usado en tests)
- `void addResponseId(String username, String surveyId, String responseId)` - Registra que usuario ha respondido una encuesta: obtiene usuario, crea/actualiza entrada en respondedSurveyId, persiste
- `void deleteResponseId(String username, String surveyId, String responseId)` - Elimina referencia a respuesta: obtiene usuario, remueve responseId del conjunto de respondidas, persiste
- `boolean existsResponseId(String username, String surveyId, String responseId)` - Verifica si un usuario ha respondido una encuesta específica
- `Set<String>getAllResponseIdsFromSurvey(String username, String surveyId)` - Obtiene todos los IDs de respuestas de un usuario para una encuesta
- `Set<String> getAllRespondedSurveyIds(String username)` - Obtiene todos los IDs de encuestas respondidas por un usuario

3. Estructuras de Datos y Algoritmos

3.1 Introducción

Este documento es una ampliación del informe de Estructuras de Datos y Algoritmos original, correspondiente a la segunda entrega del proyecto. En esta fase, se han introducido mejoras significativas que van más allá de las estructuras en memoria y los algoritmos base, enfocándose en la persistencia de datos, la robustez del código y una mayor configurabilidad. A continuación, se detallan las nuevas implementaciones.

3.2 Mejoras Arquitectónicas: Persistencia de Datos

La mejora más sustancial en la arquitectura de la aplicación es la transición de un sistema de almacenamiento en memoria a un modelo de persistencia de datos basado en ficheros JSON. Esto garantiza que toda la información gestionada por la aplicación (usuarios, encuestas y respuestas) se conserve entre sesiones de uso.

Implementación con JSON y Gson

- Tecnología utilizada: Se ha empleado la biblioteca Gson de Google para la serialización y deserialización de los objetos de dominio a formato JSON.
- Estructura de Ficheros: Los datos se almacenan en el directorio DATA/db/, con ficheros dedicados para cada tipo de entidad:
 - users.json: Almacena el mapa de todos los usuarios registrados.
 - surveys.json: Contiene todas las encuestas creadas, con sus preguntas anidadas.
 - responses.json: Guarda todas las respuestas, organizadas por encuesta.
- Beneficios: Este enfoque no solo proporciona durabilidad a los datos, sino que también facilita la depuración y la manipulación externa de los datos si fuera necesario.

3.3 Detalles de Implementación en Repositorios

Los repositorios (UserRepository, SurveyRepository, ResponseRepository) han evolucionado para soportar la persistencia y ofrecer una funcionalidad más completa.

Características Comunes en todos los Repositorios

- Operaciones CRUD completas: Se ha implementado un conjunto completo de operaciones (Crear, Leer, Actualizar, Eliminar) que interactúan directamente con los ficheros JSON, asegurando que cualquier cambio se guarde de forma inmediata.
- Constructores Configurables: Cada repositorio ofrece un constructor por defecto y un constructor parametrizado que acepta la ruta al fichero JSON. Esto facilita la realización de pruebas con conjuntos de datos aislados y añade flexibilidad al sistema.
- Gestión del Sistema de Ficheros: Los repositorios ahora incluyen lógica para crear automáticamente el directorio DATA/db/ si no existe, evitando errores en la primera ejecución de la aplicación.

SurveyRepository: Características Específicas

- Generación de IDs Autoincremental: Gestiona un contador (nextSurveyId) para asignar identificadores únicos y secuenciales a cada nueva encuesta. Este contador también se persiste en el fichero JSON y cuenta con un mecanismo de seguridad para evitar el desbordamiento.
- Adaptadores de Tipo para Polimorfismo: Para manejar la complejidad de las preguntas, se ha implementado un QuestionAdapter. Este adaptador permite a Gson serializar y deserializar correctamente la lista de preguntas de una encuesta, que puede contener distintos tipos de objetos (MultipleChoiceQuestion, TextualQuestion, etc.), conservando su tipo y estructura específicos. También se utiliza un LocalDateTimeAdapter para el manejo de fechas.
- Compatibilidad hacia Atrás: El método de carga de datos está diseñado para ser compatible con versiones anteriores del formato JSON, lo que demuestra la robustez del sistema ante futuras evoluciones.

ResponseRepository: Características Específicas

- Adaptador para Respuestas Polimórficas: De manera similar a las preguntas, se utiliza un AnswerAdapter para gestionar la serialización y deserialización de los diferentes tipos de respuestas (MultipleChoiceAnswer, TextualAnswer, etc.).

3.4 Algoritmos de Agrupamiento: Mejoras y Refinamientos

Los algoritmos de clustering (KMeans, KMeansPlusPlus, KMedoids) han sido refinados para mejorar su usabilidad, robustez y consistencia.

Mejoras Generales en los Algoritmos

- Constructores Configurables: Todos los algoritmos ahora disponen de un constructor por defecto con valores estándar (maxIterations=100, tolerance=1e-4) y un constructor que permite personalizar estos parámetros, ofreciendo un control más fino sobre su ejecución.
- Reproducibilidad: Se ha añadido un método setRandomSeed(long seed) en todos los algoritmos. Esta funcionalidad es crucial para la depuración y las pruebas, ya que permite obtener resultados consistentes y reproducibles en ejecuciones que dependen de la aleatoriedad (como la elección inicial de centroides).

Refinamientos en KMeans

- Manejo Robusto de Preguntas sin Respuesta: La implementación ahora es capaz de gestionar respuestas incompletas. Si una pregunta no ha sido contestada, el algoritmo le asigna un valor por defecto (como una cadena vacía o un cero), lo que evita fallos y permite trabajar con datos del mundo real, que a menudo son dispersos.
- Conciencia del Problema de Clústeres Vacíos: El código fuente incluye anotaciones que demuestran la conciencia sobre el problema de los clústeres vacíos, una consideración práctica importante en implementaciones de K-Means.

Refinamientos en KMeansPlusPlus

- Inicialización con Fallback: El método de inicialización "inteligente" se ha hecho más robusto. En el caso improbable de que el proceso de selección probabilística falle, el

algoritmo recurre a una selección aleatoria para garantizar que no se interrumpa la ejecución.

Refinamientos en KMedoids

- **Criterio de Convergencia Específico:** La convergencia se determina de forma explícita calculando la diferencia en la función de coste total entre una iteración y la siguiente. Si esta diferencia es menor que la tolerancia especificada, el algoritmo se detiene.
- **Estrategia para Clústeres Vacíos:** La implementación cuenta con una estrategia clara para manejar clústeres vacíos: si un clúster pierde todos sus puntos, su medoide se reinicializa eligiendo un punto aleatorio de todo el conjunto de datos, evitando así que el algoritmo falle.
- **Consistencia en la Salida:** Para mantener una estructura de salida uniforme, los medoides finales (que son objetos Response) se convierten en objetos Centroid utilizando un método estático reutilizado de la clase KMeansPlusPlus.

3.5 Nuevas Métricas de Distancia Avanzadas

Una de las incorporaciones más notables en esta segunda entrega es la mejora del sistema de cálculo de distancias, que ahora soporta técnicas más sofisticadas para medir la similitud entre respuestas. Estas nuevas métricas permiten un análisis de clustering más profundo y preciso, especialmente en lo que respecta a los datos textuales.

Cálculo de Distancia Semántica con Embeddings de Texto

Para superar las limitaciones de la distancia de Levenshtein, que se basa únicamente en la estructura sintáctica de los textos, se ha introducido un nuevo método de comparación basado en embeddings de texto.

¿Qué son los Embeddings?

Un embedding de texto es una representación vectorial (una lista de números) de una pieza de texto en un espacio de alta dimensionalidad. La característica principal de estos vectores es que capturan el significado semántico del texto. Palabras o frases con significados similares, como "contento" y "feliz", tendrán representaciones vectoriales muy cercanas en este espacio, aunque su composición de caracteres sea completamente diferente.

Implementación en el Proyecto

- **Modelo Utilizado:** Se ha integrado la biblioteca langchain4j para hacer uso del modelo de embeddings AllMiniLmL6V2EmbeddingModel. Este es un modelo de lenguaje de la familia de "Sentence Transformers" que es eficiente y se ejecuta de forma local, sin necesidad de llamadas a APIs externas.
- **Proceso:** Cuando se elige esta opción, cada respuesta textual se convierte en un vector de 384 dimensiones. Este vector representa la esencia semántica del texto.

Comparación de Vectores con Similitud del Coseno

Una vez que los textos se han convertido en vectores, la distancia entre ellos se calcula utilizando la similitud del coseno.

- Concepto: La similitud del coseno mide el coseno del ángulo entre dos vectores. Esta métrica es especialmente útil para datos de alta dimensionalidad como los embeddings, ya que evalúa la orientación de los vectores en lugar de su magnitud. En el contexto de los textos, esto significa que la similitud se centra en el significado, independientemente de la longitud del texto.
- Fórmula e Interpretación: La similitud del coseno se calcula como:

$$\text{similitud}(A, B) = (A \cdot B) / (\|A\| \|B\|)$$
 El resultado varía entre -1 y 1:
 - 1: Los vectores apuntan en la misma dirección (máxima similitud semántica).
 - 0: Los vectores son ortogonales (sin similitud).
 - -1: Los vectores apuntan en direcciones opuestas (significados contrarios).
- Conversión a Distancia: Para que esta métrica sea compatible con los algoritmos de clustering (que esperan una distancia, no una similitud), el resultado se normaliza a un rango de 0 a 1 con la fórmula $\text{distancia} = (1 - \text{similitud}) / 2$. Así, una distancia de 0 representa textos idénticos en significado, y una distancia de 1 representa textos con significados opuestos.

Beneficios sobre la Distancia de Levenshtein

- Comprensión Semántica: A diferencia de Levenshtein, los embeddings pueden identificar que "Una película excelente" y "Un film magnífico" son respuestas muy similares.
- Robustez ante Sinónimos y Reestructuración: El orden de las palabras y el uso de sinónimos tienen un impacto mucho menor en la distancia, ya que lo que se mide es el significado global.
- Análisis de Sentimiento Implícito: El modelo de embeddings puede capturar matices de sentimiento, agrupando respuestas con connotaciones positivas o negativas de manera más efectiva.

Distancia de Coseno para Agregación de Métricas

Además de la mejora en las distancias textuales, se ha añadido un nuevo DistanceType general denominado COSINE. Este tipo de distancia no se aplica a una pregunta individual, sino que ofrece un método alternativo para agregar las distancias locales de todas las preguntas de una encuesta.

Mientras que la distancia Euclídea suma los cuadrados de las distancias locales y la de Manhattan suma los valores absolutos, la distancia de Coseno en este contexto utiliza una fórmula basada en la similitud del coseno para combinar las puntuaciones de similitud de cada pregunta $(1 - \text{distancia_local})$.

Este enfoque proporciona una visión holística diferente sobre la similitud entre dos respuestas completas. En lugar de penalizar fuertemente las grandes diferencias en preguntas individuales (como hace la distancia Euclídea), se enfoca en el patrón general de similitudes a lo largo de todas las preguntas, ofreciendo así una nueva perspectiva para la experimentación en el análisis de clusters.

3.6 Conclusión

Las implementaciones de esta segunda entrega han transformado la aplicación de un prototipo en memoria a un sistema de software robusto y persistente. La persistencia de datos, la mayor configurabilidad de los algoritmos y el manejo de casos límite (como datos incompletos o clústeres vacíos) contribuyen a una aplicación más fiable, usable y preparada para su uso en un entorno real.